PayCore Code Notes

The explanations that used to sit in the source code as comments, collected by file. Each entry names the class, method, field or setting it describes. Generated 2026-10-09.

- Backend (301 files)
- Backend tests (53 files)
- Frontend (36 files)
- Configuration, build and scripts (8 files)

Backend

account/config

account/config/AccountProperties.java

`private Set<String> supportedCurrencies = Set.of("NGN");`

ISO 4217 codes customers can open accounts in.

`private String institutionCode = "999";`

Three-digit institution code that seeds the NUBAN check digit of every account number. Changing it invalidates existing numbers.

account/controller

account/controller/AccountController.java

`public class AccountController`

Customer self-service. Every endpoint works on the caller's own accounts: the customer id comes from the access token, never from the request, so the permission (RBAC) and ownership are both enforced.

account/controller/AdminAccountController.java

`public class AdminAccountController`

Staff account administration. Support and admins can read any customer's accounts and history; only admins can activate, freeze, unfreeze or close, each with a reason that goes into the audit history.

`public AccountResponse activate(`

PENDING -> ACTIVE. Refused with 403 KYC_VERIFICATION_REQUIRED until the customer's KYC is verified.

`public AccountResponse close(`

POST rather than DELETE: the account is kept (closed, not removed) and the reason travels in the body.

account/dto/request

account/dto/request/AccountStatusChangeRequest.java

`public record AccountStatusChangeRequest(`

Body of a staff freeze, unfreeze or close. The reason is kept in the account's audit history.

account/dto/request/OpenAccountRequest.java

`public record OpenAccountRequest(`

What a customer chooses when opening an account. The owner, account number, status and (later) balance all come from the server, so they are not fields here; any extra fields a client sends are ignored.

account/entity

account/entity/Account.java

`public class Account`

A customer's financial account. It identifies the account; it is not the ledger, so it has no balance. Every monetary movement will be a ledger entry against this account instead.

There are no setters: the account number, owner and currency never change, and the status only moves through the transition methods.

System accounts (see `AccountType#isSystem()`) are PayCore's own side of the ledger. They have no customer and no account number, and are created by the ledger, never through `#open`.

`private Customer customer;`

Null only for system accounts.

`private String accountNumber;`

Null only for system accounts.

`public static Account open(`

Opens a PENDING account. It cannot be used until activated, which is where eligibility (verified KYC) is checked. One open account per type and currency is enforced by the database.

`public void activate()`

PENDING -> ACTIVE, once the customer has passed the eligibility checks.

`public void freeze()`

ACTIVE -> FROZEN. A frozen account keeps its history but will not be allowed to move money.

`public void unfreeze()`

FROZEN -> ACTIVE.

`public void close()`

Any open status -> CLOSED, which is final. Once the ledger exists, closing will also require a zero balance and no pending holds.

account/entity/AccountStatusEvent.java

`public class AccountStatusEvent`

Append-only record of an account being opened or changing status, with who did it and why. Ids are stored as plain UUIDs so reading the history never loads accounts or customers. Rows are never updated or deleted.

account/enums

account/enums/AccountStatus.java

`public enum AccountStatus`

Account lifecycle. Transitions are enforced by `com.fintechplatform.paycore.account.entity.Account`, never by setting a status directly:

```
PENDING --activate--> ACTIVE <--freeze/unfreeze--> FROZEN
   |                    |                           |
   +--------------------close-------------------------+--> CLOSED (final)
```

Accounts open PENDING and become ACTIVE once the customer is eligible, so account, customer and KYC status stay independent.

account/enums/AccountType.java

`public enum AccountType`

Account products, plus PayCore's own system accounts.

PERSONAL is the one customer product for now; BUSINESS, SAVINGS and so on are added when there is a product behind them.

SETTLEMENT is a system account: the pool of real money PayCore holds at its settlement bank, one per currency, on the other side of every deposit and withdrawal. It has no owner and no account number, and customers cannot open one.

account/exception

account/exception/AccountNotFoundException.java

`public class AccountNotFoundException extends RuntimeException`

Also thrown when the account exists but belongs to someone else, so a customer cannot learn which account ids exist.

account/exception/AccountNumberUnavailableException.java

`public class AccountNumberUnavailableException extends RuntimeException`

Every generated account number was already taken. With 10^9 serials this signals a broken generator or an exhausted number range, not bad luck.

account/repository

account/repository/AccountRepository.java

`Optional<UUID> findIdByAccountNumber(@Param("accountNumber") String accountNumber);`

The id only, without loading the account: a caller that goes on to lock it must not already hold an unlocked copy, which Hibernate would keep instead of the state read under the lock.

`Optional<Account> findByIdAndCustomerId(UUID id, UUID customerId);`

Looks an account up only if it belongs to the customer, so ownership is part of the query rather than a check someone can forget.

`INSERT INTO accounts`

Creates the system account of this type and currency unless it already exists. ON CONFLICT against uk_accounts_system_type_currency makes concurrent first uses safe, and unlike a failed INSERT it does not abort the surrounding transaction.

`Optional<Account> findByIdForUpdate(@Param("accountId") UUID accountId);`

SELECT ... FOR UPDATE: holds the row lock until the transaction ends, so two transfers from the same account run one after the other and the second sees the first one's debit.

`Optional<Account> findByIdAndCustomerIdForUpdate(`

As `#findByIdForUpdate`, but only the customer's own account, so the ownership check and the lock happen in one statement.

account/service

account/service/AccountNumberGenerator.java

`public interface AccountNumberGenerator`

Produces candidate account numbers. A candidate is not guaranteed to be unused: `AccountService` checks it and the database unique constraint has the final say.

account/service/AccountService.java

`public AccountResponse openAccount(`

Opens an account for the authenticated customer. The customer id always comes from the access token, never from the request body.

The account opens PENDING; eligibility (verified KYC) is checked when staff activate it, so opening does no reads at all. The normal path is one attempt: INSERT the account and its OPENED event, commit. There are no "does it exist?" SELECTs first: the unique constraints decide, and a violation says which rule was hit. The loop only runs again after an account number collision, about one in 10^9 per existing account.

Not @Transactional on purpose: each attempt runs in its own transaction, because in PostgreSQL a failed INSERT aborts the transaction it ran in, so a collision can only be retried in a fresh one.

`log.warn("Account number collision on attempt {}", attempt);`

Generated number already in use: try a fresh one.

`throw new AccountAlreadyExistsException(request.type(), currency);`

Already has an open account of this type and currency, including one opened by a concurrent request.

`public AccountResponse getOwnAccount(UUID customerId, UUID accountId)`

404 for someone else's account as well as a missing one, so account ids cannot be probed.

`public AccountResponse activate(UUID accountId, UUID staffId, String reason)`

PENDING -> ACTIVE once the customer is eligible. Today eligibility is verified KYC; risk or product checks can be added here without touching account opening.

`Customer customer = customerRepository.getReferenceById(customerId);`

A reference, not a SELECT: this request's access token was just checked against the customer's session and status, and the foreign key guarantees the customer exists.

`Account account =`

Flush now so a unique constraint violation (duplicate account or account number) surfaces inside this attempt, where openAccount can tell which constraint it was.

`private AccountResponse changeStatus(`

Staff status changes. Staff cannot act on their own accounts, so no one can unfreeze themselves. Concurrent changes to the same account are caught by optimistic locking.

`private Account findAccount(UUID accountId)`

Customer accounts only: system accounts are the ledger's, not something staff view, activate, freeze or close here.

account/service/NubanAccountNumberGenerator.java

`public class NubanAccountNumberGenerator implements AccountNumberGenerator`

Generates 10-digit account numbers in the CBN NUBAN format: a 9-digit serial followed by a check digit computed over the institution code and the serial. The check digit catches most mistyped account numbers before any lookup, as Nigerian bank account numbers do.

Serials come from `SecureRandom` so account numbers cannot be predicted or enumerated from one another.

`private static final int[] WEIGHTS = {3, 7, 3, 3, 7, 3, 3, 7, 3, 3, 7, 3};`

NUBAN weights, applied to the 3-digit code then the 9-digit serial.

`public static boolean isValid(String institutionCode, String accountNumber)`

Whether the account number carries the right check digit for the institution code.

authorization/controller

authorization/controller/AdminRoleController.java

`public class AdminRoleController`

Admin-only role management. Every change needs a reason and is written to the role assignment audit trail with the acting admin's id. Admins cannot change their own roles.

`public CustomerRolesResponse revokeRole(`

POST rather than DELETE because the reason travels in the body, and DELETE bodies are dropped by some clients and proxies.

authorization/dto

authorization/dto/RoleAssignmentEventResponse.java

`public record RoleAssignmentEventResponse(`

@param performedBy the acting admin's customer id, or null when the system made the change

authorization/entity

authorization/entity/RoleAssignmentEvent.java

`public class RoleAssignmentEvent`

Append-only record of a role being assigned to or revoked from a customer. Ids are stored as plain UUIDs so reading the history never loads customers or roles. Rows are never updated or deleted.

`private UUID performedBy;`

The customer (admin) who made the change, or null when the system did, e.g. the default CUSTOMER role on registration.

authorization/entity/RoleName.java

`public final class RoleName`

Role names seeded by V6__create_roles_and_permissions.sql.

`public static final String OPERATIONS = "OPERATIONS";`

Added by V24: requests and approves money corrections (maker-checker).

authorization/service

authorization/service/RoleAssignmentService.java

`public class RoleAssignmentService`

The only place roles should be assigned or revoked: every change is recorded in role_assignment_events in the same transaction, so the current state and its history cannot drift apart.

Role changes reach the customer's permissions when their next access token is issued, so a revoked role can stay usable until the current access token expires.

`public CustomerRolesResponse getRoles(UUID customerId)`

ADMIN OPERATIONS

`public CustomerRolesResponse assignRole(`

Admins cannot change their own roles, so no one can grant themselves extra access or remove their own ADMIN by mistake.

`public void assignRole(`

CORE OPERATIONS

The customer must already be persisted (it needs an id).

@param performedBy the acting admin's customer id, or null when the system assigns the role

banktransfer/config

banktransfer/config/RailProperties.java

`public class RailProperties`

`paycore.rails.provider` picks the bank rail ("simulated" or "none"); it is read by the rail's own condition, not here.

`webhookSecret` signs inbound transfer notifications (HMAC-SHA256 of the raw body, hex, in X-PayCore-Signature). Without it the webhook accepts nothing.

`simulatorInboundLimitPerAccount` caps what Test Bank sends to one PayCore account in total, in major units, since its money is not real.

banktransfer/controller

banktransfer/controller/BankRailWebhookController.java

`public class BankRailWebhookController`

Where the bank rail reports money other banks sent to PayCore accounts. Not behind a sign-in: the rail authenticates with the request signature instead (401 without a valid one).

200 ACCEPTED with the ledger reference, also for a repeated delivery of the same session; 422 REJECTED when the account cannot take the money, so the rail returns it to the sending bank.

banktransfer/controller/BankTransferController.java

`public class BankTransferController`

Banks a customer can pay, who holds an account (name enquiry), and transfers to other banks from the caller's own accounts.

`public List<BankResponse> banks()`

PayCore first, then every bank the rail reaches.

`public NameEnquiryResponse nameEnquiry(`

422 BENEFICIARY_NOT_FOUND if the bank has no such account.

`public ResponseEntity<OutboundTransferResponse> send(`

201 with the debit and the rail's verdict. A rejected transfer is still 201: the debit happened and was reversed, and transferStatus says FAILED.

banktransfer/dto

banktransfer/dto/BankResponse.java

`public record BankResponse(String code, String name)`

A bank a customer can send money to: its institution code and name.

banktransfer/dto/InboundTransferAck.java

`public record InboundTransferAck(String status, String reference, String reason)`

PayCore's answer to the rail: ACCEPTED with the ledger reference, or REJECTED with a reason, in which case the rail returns the money to the sending bank.

banktransfer/dto/InboundTransferNotification.java

`public record InboundTransferNotification(`

What the bank rail tells PayCore when another bank sends money to a PayCore account. sessionId identifies the payment on the rail; the same notification may be delivered more than once.

banktransfer/dto/NameEnquiryResponse.java

`public record NameEnquiryResponse(`

Who holds the account, for the customer to confirm before paying.

banktransfer/dto/OutboundTransferRequest.java

`public record OutboundTransferRequest(`

A transfer from one of the caller's accounts (in the path) to an account at another bank. The beneficiary's name is not sent: PayCore looks it up with the bank (name enquiry) and records what the bank says.

Retrying with the same idempotency key returns the original transfer without paying again.

banktransfer/dto/OutboundTransferResponse.java

`public record OutboundTransferResponse(`

The debit for a transfer to another bank, flattened into the same JSON object, plus what happened on the rail:

```
SUCCESSFUL  the beneficiary bank accepted the payment
FAILED      it was rejected; the debit was reversed (reversalReference)
PENDING     the outcome is not known yet; the debit stands until it is
```

paymentId, paymentReference and paymentStatus identify the payment that tracks it (GET /api/v1/payments/{paymentId}): paymentStatus is the detailed state behind transferStatus.

banktransfer/dto/SimulatedInboundRequest.java

`public record SimulatedInboundRequest(`

The signed-in customer sending money from their Test Bank account to a PayCore account (usually their own).

banktransfer/dto/TestBankAccountResponse.java

`public record TestBankAccountResponse(`

The signed-in customer's own (pretend) account at Test Bank, and how much more it may still send to the given PayCore account: the simulator caps what it sends to any one account, since its money is not real.

banktransfer/exception

banktransfer/exception/BeneficiaryNotFoundException.java

`public class BeneficiaryNotFoundException extends RuntimeException`

Name enquiry found no account with that number at that bank.

banktransfer/exception/InboundNotificationLostException.java

`public class InboundNotificationLostException extends RuntimeException`

Simulator only: Test Bank sent the money, but the rail's notification to PayCore was lost (amounts ending in .95), so nothing was credited. Reconciliation finds the session in the rail's settlement report and has the rail deliver it again.

banktransfer/exception/InvalidWebhookSignatureException.java

`public class InvalidWebhookSignatureException extends RuntimeException`

The notification was not signed with PayCore's webhook secret.

banktransfer/exception/SimulatorLimitExceededException.java

`public class SimulatorLimitExceededException extends RuntimeException`

Test Bank has sent the account as much pretend money as it may.

banktransfer/rail

banktransfer/rail/Bank.java

`public record Bank(String code, String name)`

A bank on the rail: its NIP institution code and name.

banktransfer/rail/BankRail.java

`public interface BankRail`

The interbank payment rail (NIP in Nigeria) as PayCore sees it, through a provider: which banks can be reached, who holds an account there (name enquiry), and paying out to it. Money coming in arrives the other way, as a notification to PayCore's webhook.

At most one is active, chosen by `paycore.rails.provider`; with none, customers can only move money inside PayCore. The simulator stands in for a real provider until one is connected.

`String providerName();`

Stored on every transfer it carries, e.g. "SIMULATED".

`List<Bank> banks();`

Banks reachable through the rail.

`Optional<String> nameEnquiry(NameEnquiry enquiry);`

The account holder's name at the other bank, empty if the bank has no such account. Customers confirm the name before they pay.

@throws BankRailException when the rail cannot answer

`RailResult send(RailPayment payment);`

Pays the beneficiary bank. A rejection is an answer (the money must go back to the customer); an exception means the outcome is unknown and must be confirmed with the provider before anything is reversed.

@throws BankRailException when the rail cannot be reached

`RailStatus status(String sessionId);`

What happened to a payment sent under sessionId (a transaction status query). How PayCore settles a transfer whose send gave no answer, or whose answer PayCore never recorded.

@throws BankRailException when the rail cannot answer

`List<RailSettlementLine> settlementReport(Instant from, Instant to);`

The rail's settlement report: every session it carried, either way, created in [from, to). What reconciliation compares PayCore's books with.

@throws BankRailException when the rail cannot answer

`Optional<InboundTransfer> redeliverInbound(String sessionId);`

Asks the rail to deliver an inbound transfer's notification again, when reconciliation found money the rail settled but PayCore never heard of. Empty if the rail has no such inbound session.

@throws BankRailException when the rail cannot answer

banktransfer/rail/BankRailException.java

`public class BankRailException extends RuntimeException`

The rail could not give an answer (outage, timeout). Details are logged, not returned to the customer.

banktransfer/rail/NameEnquiry.java

`public record NameEnquiry(String bankCode, String accountNumber, UUID requestedBy)`

Who holds this account at that bank? requestedBy is the PayCore customer asking; a real rail ignores it, the simulator uses it to recognise the customer's own Test Bank account.

banktransfer/rail/RailPayment.java

`public record RailPayment(`

A payment to another bank. sessionId is PayCore's unique id for it on the rail; sending the same session twice must not pay twice.

banktransfer/rail/RailResult.java

`public record RailResult(boolean accepted, String reason)`

Whether the beneficiary bank accepted the payment, and why not if it did not.

banktransfer/rail/RailSettlementLine.java

`public record RailSettlementLine(`

One line of the rail's settlement report: a session it carried, either way, and what became of it. direction is OUTBOUND (PayCore paid another bank) or INBOUND (another bank paid a PayCore account); status is SUCCESSFUL, FAILED (rejected or returned: nobody kept the money) or RETURNED (inbound money PayCore refused, sent back).

banktransfer/rail/RailStatus.java

`public record RailStatus(Outcome outcome, String reason)`

The rail's answer to a transaction status query (TSQ) for a session.

```
SUCCESSFUL  the beneficiary bank was paid
FAILED      the payment was rejected (reason says why); nobody was paid
PENDING     still in flight; ask again later
NOT_FOUND   the rail has no record of the session: it never arrived
```

banktransfer/rail/SessionIds.java

`public final class SessionIds`

NIP-style session ids: 30 digits, the sending institution's 6-digit code, the Lagos date and time to the second (yyMMddHHmmss), then 12 random digits.

banktransfer/service

banktransfer/service/BankTransferService.java

`public class BankTransferService`

Sending money to other banks, and finding out who holds an account before paying it (name enquiry), for PayCore accounts and accounts at banks on the rail alike.

An outbound transfer works like at any bank:

```
1. retry of a transfer already made with this key? return it, pay nothing
2. name enquiry: the beneficiary must exist; their name goes on the record
3. ledger + payment: DEBIT customer, CREDIT settlement, under the account
   lock, and the payment CREATED, in one database transaction
4. payment PROCESSING, committed before the rail is called
5. rail: pay the beneficiary bank (no database transaction is open)
6. paid: SUCCEEDED
   rejected: the debit is reversed and the payment FAILED, together
   no answer: UNKNOWN. The beneficiary may have been paid, so nothing is
   reversed until a status check with the rail says what happened
   (PaymentRecoveryService)
```

Not transactional itself, so no row lock is held while the rail answers.

`public static final Bank PAYCORE = new Bank("100999", "PayCore");`

PayCore itself, as a bank in the list and in name enquiry.

`public List<BankResponse> banks()`

PayCore first, then every bank the rail reaches (none without a rail).

`public NameEnquiryResponse nameEnquiry(UUID customerId, String bankCode, String accountNumber)`

The name on the account, so the customer can check who they are paying. A PayCore account is looked up here; another bank's through the rail.

`Payment payment =`

The debit and its payment, together: CREATED.

`if (!sessionId.equals(payment.getExternalReference()))`

A concurrent duplicate of this request was debited first; it is the one being paid, not this session.

`RailResult result;`

Committed before the rail is called: from here on PayCore must assume the rail may have acted, even if it never answers.

`log.warn(`

Outcome unknown: the beneficiary may have been paid. Reversing now could pay the customer twice, so the debit stands until a status check with the rail says what happened.

`public Payment settle(Payment payment)`

SETTLING AN UNKNOWN OUTCOME

Asks the rail what happened to a transfer whose answer PayCore does not have (a transaction status query), and acts on it: paid stays debited, rejected or never received is reversed. No answer leaves it UNKNOWN for the next check (or, checks used up, for operations). The check must have been claimed (PaymentService.claimStatusCheck).

`case NOT_FOUND ->`

`payment.getCreatedAt().isBefore(Instant.now().minus(paymentProperties.getNotFoundGrace()))`

The rail may not have recorded a session it just received, so "no such session" is only believed once the grace has passed.

`public Optional<Payment> abandonUnsent(UUID paymentId, Instant staleBefore)`

A transfer PayCore debited but stopped before handing to the rail (still CREATED long after): it was never sent, so nobody was paid. The customer gets their money back rather than a transfer executed long after they asked for it.

`public RailStatus currentStatus(Payment payment)`

The rail's answer about the transfer now (a status query), or null if it gave none (unreachable, or not the rail that carried it).

`private static Payment sameTransfer(Payment payment, UUID sourceAccountId, OutboundTransferRequest request)`

A retry of a transfer that already has a payment: the same request gets that payment's outcome and pays nothing again; a different one with the same key is refused.

`public OutboundTransferResponse toResponse(Payment payment)`

The debit as the customer sees it, and what became of the transfer.

banktransfer/service/InboundWebhookService.java

`public class InboundWebhookService`

Inbound transfer notifications from the bank rail. Only a notification signed with the shared webhook secret is believed: anyone can reach the URL, and an unsigned "money arrived" message would let them mint money.

`public TransactionResponse receive(String rawBody, String signature)`

Verifies the signature over the exact bytes received, then credits the transfer (idempotently, by session id).

`public static String sign(String secret, String body)`

Lower-case hex HMAC-SHA256 of the body: what the rail must send.

banktransfer/simulator

banktransfer/simulator/RailSessionRecords.java

`public class RailSessionRecords`

The simulated rail's own record of every session it carried, in simulator.rail_sessions (V28, V30), as the real rail operator keeps one: what a transaction status query reads, and what its settlement report lists. Each call commits on its own (REQUIRES_NEW), even when PayCore calls the provider from inside one of its transactions.

`public record RailSession(`

One session. direction is OUTBOUND or INBOUND; status SUCCESSFUL, FAILED or RETURNED. For an inbound session, sender* name who paid and notified whether PayCore was told about it.

`public static RailSession outbound(`

An outbound session (PayCore paying another bank).

`public RailSession saveIfAbsent(RailSession session)`

Stores the session unless it already exists, and returns the stored one: a session sent twice is carried once.

`public void markReturned(String sessionId, String reason)`

An inbound session PayCore refused: the money went back to the sender.

`public void markNotified(String sessionId)`

PayCore has now been told about an inbound session.

`public List<RailSession> between(Instant from, Instant to)`

Every session created in [from, to), oldest first.

banktransfer/simulator/SimulatedBankRail.java

`public class SimulatedBankRail implements BankRail`

In-process stand-in for an interbank rail provider, connected to one pretend bank, `TestBank`. It never calls out and no real money moves.

The kobo decide what happens, after the customer was debited, so every path can be seen as easily as a successful transfer:

```
.99  the beneficiary bank rejects it: reversed at once
.98  paid, but the answer is lost (timeout): PENDING until a status
     check finds it was paid
.97  never reaches the rail (timeout): PENDING until a status check
     finds nothing, and then reversed
.96  paid, the answer is lost, and status queries for it never answer
     either: only operations can resolve it (complete or reverse)
.93  accepted (PayCore marks it paid), but the beneficiary bank returns
     it later: the settlement report says FAILED, which only
     reconciliation finds
else paid
```

Like the real rail it deduplicates by session id, answers status queries, produces a settlement report and re-delivers inbound notifications PayCore missed. It keeps its records in its own table (`RailSessionRecords`), so a restart forgets nothing. It must never be enabled where real customers hold accounts.

`public static final long REJECTED_MINOR_ENDING = 99;`

The minor-unit ending the beneficiary bank always rejects.

`public static final long PAID_THEN_TIMEOUT_MINOR_ENDING = 98;`

Paid, but the answer never reaches PayCore.

`public static final long LOST_IN_TRANSIT_MINOR_ENDING = 97;`

Times out before reaching the rail: nobody is paid.

`public static final long STATUS_UNAVAILABLE_MINOR_ENDING = 96;`

Paid, the answer is lost, and its status can never be queried.

`public static final long RETURNED_LATER_MINOR_ENDING = 93;`

Accepted, then returned by the beneficiary bank after PayCore recorded it paid.

`if (returnedLater)`

The answer PayCore gets at the time: accepted. Only the settlement report later shows the return.

banktransfer/simulator/TestBank.java

`public class TestBank`

A pretend bank on the other side of the simulated rail. Every PayCore customer has an account there in their own name, so they can send themselves money "from another bank" and send money back out to it. Any other 10-digit number belongs to a made-up Test Bank customer, except numbers starting 000, which do not exist.

No real bank, account or money is involved.

`public String accountNumberFor(UUID customerId)`

The customer's own Test Bank account number: stable, derived from their customer id, always starting with 7.

`public Optional<String> holderOf(String accountNumber, UUID askingCustomerId)`

Who holds the account: the asking customer if it is their own, otherwise a made-up holder; empty for numbers starting 000.

`private static String digits(String input, int count)`

count decimal digits taken from SHA-256 of the input.

banktransfer/simulator/TestBankController.java

`public class TestBankController`

The simulated other bank, for customers to try money coming in. Only exists while the rail is simulated; it stands in for the customer's banking app at another bank, not for anything PayCore offers.

`public TestBankAccountResponse account(`

The caller's Test Bank account and what it may still send to accountId.

`public ResponseEntity<TransactionResponse> send(`

201 with the PayCore credit, as the destination account sees it.

banktransfer/simulator/TestBankSimulator.java

`public class TestBankSimulator`

Test Bank's side of an inbound transfer: the signed-in customer sends money from their own Test Bank account, in their own name, to a PayCore account. It reaches the ledger exactly the way a real bank's transfer would, as an inbound rail notification with a session id: PayCore does not treat it any differently.

Because the money is pretend, Test Bank sends at most a fixed total to any one PayCore account. That cap belongs to the simulator; PayCore itself never refuses a real inbound transfer for being large.

`public static final long NOTIFICATION_LOST_MINOR_ENDING = 95;`

Test Bank pays, but the rail's notification to PayCore is lost: the money sits in PayCore's settlement bank account, credited to nobody, until reconciliation finds it and the rail delivers it again.

`public TestBankAccountResponse account(UUID customerId, UUID accountId)`

The customer's Test Bank account, and how much more it may send to their PayCore account (accountId, which must be theirs).

`sessions.saveIfAbsent(new RailSession(`

The rail settles the money first, then notifies PayCore: its own record exists whatever PayCore does with the notification.

`sessions.markReturned(sessionId, exception.getMessage());`

PayCore refused it; the rail sends the money back to the sender.

bills/catalog

bills/catalog/BillCatalog.java

`public final class BillCatalog`

The billers PayCore offers, by the aggregator's ids (VTpass serviceIDs). A fixed list rather than the aggregator's whole catalogue: PayCore decides which billers it supports, and the simulator offers the same.

Electricity is paid to a prepaid or postpaid meter; that choice is the product code ("prepaid" / "postpaid").

bills/catalog/BillCategory.java

`public enum BillCategory`

What kind of bill, and what the customer's number is: a phone number for airtime and data, a meter for electricity, a smartcard (IUC) for TV.

bills/catalog/Biller.java

`public record Biller(`

A biller PayCore offers. id is the aggregator's (VTpass serviceID).

@param products the customer picks a product (data bundle, TV bouquet) whose price the aggregator sets @param verified the customer's number is checked with the biller first (meter, smartcard), so they see whose it is @param minAmount for billers without products: the least and most @param maxAmount one payment may be, in naira

bills/config

bills/config/BillConfiguration.java

`RestClient vtpassRestClient(RestClient.Builder builder, BillProperties properties)`

A payment can take a while at VTpass (it waits for the biller), so the read timeout is generous; past it the payment is UNKNOWN and a requery settles it.

bills/config/BillProperties.java

`public class BillProperties`

`paycore.bills.provider` picks the aggregator ("vtpass", "simulated" or "none"); it is read by each provider's own condition.

VTpass authenticates with an API key plus the public key (reads) or the secret key (payments), from the VTpass dashboard. Never commit them.

`private String baseUrl = "https://sandbox.vtpass.com/api";`

https://sandbox.vtpass.com/api for the sandbox, https://vtpass.com/api live.

bills/controller

bills/controller/BillPaymentController.java

`public class BillPaymentController`

Billers, their products, checking a meter or smartcard, and paying a bill from one of the caller's accounts (with the transaction PIN, from a trusted device).

`public ResponseEntity<BillPaymentResponse> pay(`

201 with the debit and the biller's verdict. A refused bill is still 201: the debit happened and was reversed, and billStatus says FAILED.

bills/dto

bills/dto/BillerResponse.java

`public record BillerResponse(`

A biller the customer can pay. products: pick a bundle or bouquet (GET .../products); verified: check the number first (POST /bills/verify); meterTypes: for electricity, the product codes to choose from.

bills/dto/BillPaymentRequest.java

`public record BillPaymentRequest(`

A bill paid from one of the caller's accounts (in the path).

productCode is the bundle or bouquet for data and TV (whose price the provider sets, so amount is ignored), or the meter type for electricity. amount is required for airtime and electricity.

Retrying with the same idempotency key returns the original payment without paying again.

bills/dto/BillPaymentResponse.java

`public record BillPaymentResponse(`

The debit for a bill, flattened into the same JSON object, plus what the biller did:

```
SUCCESSFUL  delivered; token is what the biller gave back (a meter token)
FAILED      refused; the debit was reversed (reversalReference)
PENDING     not known yet; the debit stands until it is
```

bills/dto/VerifyBillCustomerRequest.java

`public record VerifyBillCustomerRequest(`

productCode is the meter type ("prepaid" / "postpaid") for electricity.

bills/exception

bills/exception/BillCustomerNotFoundException.java

`public class BillCustomerNotFoundException extends RuntimeException`

The biller has no such number.

bills/exception/BillPaymentsDisabledException.java

`public class BillPaymentsDisabledException extends RuntimeException`

Bill payments are not available.

bills/exception/InvalidBillPaymentException.java

`public class InvalidBillPaymentException extends RuntimeException`

A bill request that cannot be paid as asked.

bills/exception/UnknownBillerException.java

`public class UnknownBillerException extends RuntimeException`

We do not offer that biller.

bills/exception/UnknownBillProductException.java

`public class UnknownBillProductException extends RuntimeException`

No such product.

bills/provider

bills/provider/BillProvider.java

`public interface BillProvider`

A bills aggregator (VTpass in Nigeria) as PayCore sees it: the products a biller sells, whose number a meter or smartcard is, paying a bill, and asking what happened to a payment (requery).

At most one is active, chosen by `paycore.bills.provider`; with none, bill payments are off. The simulator stands in until a real one is connected.

`String providerName();`

Stored on every bill it carries, e.g. "VTPASS".

`String newRequestId();`

A fresh id for one payment, in the form the aggregator wants. The aggregator deduplicates by it, and it is what a requery asks about.

`List<Product> products(String billerId);`

What the biller sells (data bundles, TV bouquets) and at what price.

@throws BillProviderException when the aggregator cannot answer

`Optional<Customer> verify(String billerId, String productCode, String customerNumber);`

The name (and address) on a meter or smartcard; empty if the biller has no such number.

@param productCode for electricity, "prepaid" or "postpaid" @throws BillProviderException when the aggregator cannot answer

`Result pay(Purchase purchase);`

Pays the bill. FAILED is an answer (nothing was delivered, the money must go back); an exception means the outcome is unknown and a requery decides.

@throws BillProviderException when the aggregator cannot be reached

`Result status(String requestId);`

What happened to the payment made under requestId.

@throws BillProviderException when the aggregator cannot answer

`record Purchase(`

@param productCode bundle or bouquet, the meter type for electricity, null for airtime @param phone the paying customer's phone (the aggregator wants one)

`record Result(Outcome outcome, String reason, String token, String providerReference)`

```
DELIVERED  the biller gave value; token is what it gave back, if any
FAILED     refused, nothing delivered (reason says why)
PENDING    still in flight; ask again later
NOT_FOUND  the aggregator has no such request: it never arrived
```

bills/provider/BillProviderException.java

`public class BillProviderException extends RuntimeException`

The aggregator could not be reached or gave no usable answer. The detail is for logs, never for customers.

bills/provider/RequestIds.java

`public final class RequestIds`

Request ids as VTpass requires them: the current time in Lagos as yyyyMMddHHmm, then something unique (here 64 random bits, hex). 202610081530A1B2C3D4E5F60718

bills/provider/VtpassBillProvider.java

`public class VtpassBillProvider implements BillProvider`

VTpass (https://www.vtpass.com/documentation). Reads send the API key and public key; payments the API key and secret key.

VTpass answers HTTP 200 with a result code in the body:

```
000        processed; content.transactions.status says delivered / pending / failed
099, 089   still processing                       -> PENDING
016, 040   failed, or reversed by the biller      -> FAILED
015        no such request id (requery)           -> NOT_FOUND
011 - 035, 085, 087, 091   refused before anything happened -> FAILED
anything else              unknown: never assumed failed     -> PENDING
```

A PENDING or unreadable answer leaves the payment UNKNOWN; the requery decides. A failure is only believed when VTpass says so.

`private static final Set<String> REFUSED = Set.of(`

Refused before VTpass or the biller did anything.

`JsonNode variations = content.has("variations") ? content.path("variations") : content.path("varations");`

VTpass spells it "varations" in some responses.

`if (category == BillCategory.TV)`

A bouquet bought outright (not a renewal of whatever is on the card).

`private Result result(JsonNode body, boolean requery)`

ANSWERS

`private static String token(JsonNode body)`

A prepaid meter token: mainToken, else "Token : 1234 ..." in purchased_code.

`private JsonNode get(String path, Object... variables)`

HTTP

bills/service

bills/service/BillPaymentService.java

`public class BillPaymentService`

Airtime, data, electricity and TV, through the configured aggregator.

Like a transfer to another bank: the customer is debited first, then the aggregator is asked to deliver. Refused: the debit is reversed. No answer: the payment is UNKNOWN and a requery settles it later (see PaymentRecoveryService), never an assumption.

Prices of bundles and bouquets come from the aggregator, never from the request: the customer pays what the biller charges.

`public List<BillerResponse> billers()`

WHAT CAN BE PAID

`public VerifyBillCustomerResponse verify(String billerId, String productCode, String customerNumber)`

Whose meter or smartcard it is, before the customer pays.

`public BillPaymentResponse pay(UUID customerId, UUID sourceAccountId, BillPaymentRequest request)`

PAYING

`String payee =`

The biller's own name for the number, checked again here rather than taken from the app.

`if (!requestId.equals(payment.getExternalReference()))`

A concurrent duplicate of this request was debited first; it is the one being paid.

`Result result;`

Committed before the aggregator is called: from here on it may have acted, even if it never answers.

`public Payment settle(Payment payment)`

SETTLING AN UNKNOWN OUTCOME

Asks the aggregator what happened (requery) and acts on it. The check must have been claimed (PaymentService.claimStatusCheck).

`public Optional<Payment> abandonUnsent(UUID paymentId, Instant staleBefore)`

A bill debited but never handed to the aggregator (still CREATED long after): nothing was paid, so the money goes back.

`public Result currentStatus(Payment payment)`

The aggregator's answer now, or null if it gave none.

`case NOT_FOUND ->`
`payment.getCreatedAt().isBefore(Instant.now().minus(paymentProperties.getNotFoundGrace()))`

The aggregator may not have recorded a request it just got, so "no such request" is only believed once the grace has passed.

`public BillPaymentResponse toResponse(Payment payment)`

HELPERS

The debit as the customer sees it, and what became of the bill.

`private static Payment sameBill(Payment payment, UUID sourceAccountId, BillPaymentRequest request)`

The same request again gets the first one's outcome; a different one with the key is refused.

`boolean pricedByBiller = BillCatalog.find(request.billerId()).map(Biller::products).orElse(false);`

Bundles and bouquets are priced by the biller, so only an amount the customer typed (airtime, electricity) must match.

`private String phoneOf(UUID customerId, Biller biller, String number)`

The aggregator wants a phone with every purchase: the number being topped up for airtime and data, else the paying customer's own.

bills/simulator

bills/simulator/BillRecords.java

`public class BillRecords`

The simulated aggregator's own record of every bill it was asked to pay, in simulator.bill_payments (V33): what its requery reads. Each call commits on its own, as a real aggregator's database would.

`public BillRecord saveIfAbsent(BillRecord record)`

Stores the payment unless the request id was seen before; returns the stored one.

bills/simulator/SimulatedBillProvider.java

`public class SimulatedBillProvider implements BillProvider`

A stand-in bills aggregator with the same billers as PayCore's catalogue and made-up prices. Like the simulated rail, the kobo pick what happens, so every path can be tried from the app:

```
.99   the biller declines: FAILED, the debit is reversed
.98   delivered, but the answer is lost: UNKNOWN until a requery says DELIVERED
.97   never reaches the aggregator: UNKNOWN, then NOT_FOUND, reversed
else  delivered (electricity comes with a 20-digit token)
```

Meter and smartcard numbers ending in 0000 do not exist.

`private String token()`

1234-5678-9012-3456-7890

common/config

common/config/OpenApiConfig.java

`public class OpenApiConfig`

API description served at /v3/api-docs, browsable at /swagger-ui.html. Every operation is documented as needing a bearer access token; the public ones (login, registration, demo info) simply ignore it. In Swagger UI, call /api/v1/auth/login, then paste the accessToken into "Authorize".

common/config/RequiredSecretsConfiguration.java

`public class RequiredSecretsConfiguration`

Fails startup with a clear message when a required secret is missing.

Spring Boot's property binding does not fail on an unresolvable placeholder: `spring.datasource.password=${PAYCORE_DB_PASSWORD}` silently becomes the literal text "${PAYCORE_DB_PASSWORD}", and startup then dies with a misleading "password authentication failed". This check runs as a BeanFactoryPostProcessor, i.e. before any bean (the connection pool, Flyway) is created.

`if (password == null || password.isBlank())`

Unlike binding, Environment.getProperty resolves nested placeholders strictly and throws when one is missing.

common/config/SecurityConfig.java

`public class SecurityConfig`

Spring Security stays stateless: every API request is authenticated by its bearer access token. The PayCore LoginSession is a separate business/security record and is not a Spring Security HTTP session.

`public CorsConfigurationSource corsConfigurationSource(`

Browser origins allowed to call the API, e.g. the Developer Preview UI on its own domain. Empty (the default) allows none, which is right when the UI shares the API's origin or uses a dev proxy.

`if (!origins.isEmpty())`

With no configuration registered, requests pass through without CORS processing instead of every cross-origin request being rejected, which would also break same-host dev proxies.

`.requestMatchers(`

Needs the caller's identity, so it must be matched before the permitAll on /auth/**.

`.requestMatchers("/error")`

Spring Boot forwards errors (e.g. failed bean validation) here; without this an anonymous caller would get 401 instead of the real 400.

`.requestMatchers(`

API description and Swagger UI. Turn them off with springdoc.api-docs.enabled=false and springdoc.swagger-ui.enabled=false.

Only exists in demo mode (paycore.demo.enabled).

The bank rail authenticates with an HMAC signature over the body, checked by InboundWebhookService.

common/net

common/net/ClientNetwork.java

`public final class ClientNetwork`

Normalizes a client IP address into the network key used for rate limiting and stored for audit.

• IPv4: the address itself, e.g. `203.0.113.7`.

• IPv6: the /64 prefix, e.g. `2001:db8:1:2::/64`. A single subscriber is normally given a whole /64, so keying on individual IPv6 addresses would let one client rotate through billions.

• IPv4-mapped IPv6 (`::ffff:203.0.113.7`): the IPv4 address.

• Missing or unparseable input: `"unknown"`, which is still rate limited as one shared bucket rather than bypassing the limit.

`private static final Pattern IPV4_LITERAL =`

InetAddress.getByName performs a DNS lookup for anything it does not recognise as a literal, including near-misses like "999.1.1.1" or "abc". So it is only called for input that is certainly a literal: a strict dotted-quad IPv4 address, or text containing ':' (which Java always parses as an IPv6 literal and never resolves).

`private static String stripDecorations(String value)`

Removes URI brackets (`[::1]`) and IPv6 zone ids (`fe80::1%eth0`).

common/persistence

common/persistence/ConstraintViolations.java

`public final class ConstraintViolations`

Lets services rely on database constraints instead of "does it exist?" queries before every insert: the insert either succeeds, or fails with the name of the rule it broke. One round trip instead of two, and no race between the check and the insert.

`public static String constraintName(DataIntegrityViolationException exception)`

The name of the violated constraint or unique index, or null if the database did not report one.

common/persistence/DatabasePrivilegeCheck.java

`public class DatabasePrivilegeCheck implements ApplicationRunner`

At startup, checks that the database user the API runs as cannot do what the ledger's triggers cannot stop (V29): own the tables, be a superuser, delete or truncate ledger and payment rows, or rewrite ledger entries.

With `paycore.database.require-least-privilege=true` (production) a violation stops the application; otherwise it is logged as a warning, since local development runs as one all-powerful user.

`static final List<String> PROTECTED_TABLES =`

Tables whose rows must never be deleted by the application.

`public List<String> problems()`

What is wrong, in words; empty when the user is least-privileged.

common/persistence/UuidV7.java

`public @interface UuidV7`

Generates the entity identifier as a time-ordered UUIDv7 (RFC 9562) when the entity is first persisted.

customer/config

customer/config/EmailVerificationProperties.java

`private String publicUrl = "http://localhost:5173";`

Where the web app runs: verification links point to {publicUrl}/verify-email?token=...

`private Duration tokenTtl = Duration.ofHours(24);`

How long a link works.

`private Duration resendCooldown = Duration.ofSeconds(60);`

Minimum wait between two links for the same customer.

`private int maxPerDay = 5;`

Most links one customer can be sent in 24 hours.

customer/controller

customer/controller/AdminCustomerController.java

`public class AdminCustomerController`

Admin customer directory. Actions on a single customer stay on `CustomerController` and the admin role endpoints.

customer/controller/CustomerProfileController.java

`public class CustomerProfileController`

The signed-in customer's own profile. The customer id always comes from the access token.

`public CustomerResponse updateMe(`

Name, email and phone number; send only what changes. A new email is unverified until its link is used.

`public void resendVerificationEmail(`

Sends a new verification link (and retires the old one). At most one a minute and five a day.

customer/controller/EmailVerificationController.java

`public class EmailVerificationController`

Public, like login: the link may be opened on a device where the customer is not signed in. The token itself is the proof.

`public CustomerResponse verifyEmail(@Valid @RequestBody VerifyEmailRequest request)`

Verifies the email and, for a new customer, activates them.

customer/dto/request

customer/dto/request/VerifyEmailRequest.java

`public record VerifyEmailRequest(`

The token from the link in a verification email.

customer/dto/response

customer/dto/response/CustomerSummaryResponse.java

`public record CustomerSummaryResponse(`

One row of the admin customer list.

customer/entity

customer/entity/Customer.java

`private String phoneNumber;`

Null for a customer created through Google sign-in until they add one.

`public void assignRole(Role role)`

Low-level state change only. Application code must go through RoleAssignmentService so the change is recorded in the audit trail.

customer/entity/EmailVerificationToken.java

`public class EmailVerificationToken`

A one-time link proving a customer owns an email address. Only the SHA-256 hash of the token is stored; the raw token exists only in the email that was sent.

`public boolean isUsable(Instant now)`

Usable: not used or replaced, and not expired.

`public void markUsed()`

Used, or replaced by a newer link. Either way it no longer works.

customer/enums

customer/enums/CustomerStatus.java

`public boolean canAuthenticate()`

Whether a customer in this status may sign in, refresh tokens or use an access token they already hold.

customer/event

customer/event/CustomerAccessRevokedEvent.java

`public record CustomerAccessRevokedEvent(`

Published inside the suspend or close transaction once the customer can no longer authenticate. Listeners run in that same transaction, so the status change and ending the customer's sessions commit together.

customer/event/CustomerEmailChangedEvent.java

`public record CustomerEmailChangedEvent(UUID customerId)`

A customer's email address changed and is no longer verified; a new verification link goes to the new address.

customer/event/CustomerRegisteredEvent.java

`public record CustomerRegisteredEvent(`

Published inside the registration transaction once the customer is saved. Listeners run in that same transaction, so a listener failure rolls the registration back.

customer/exception

customer/exception/InvalidVerificationTokenException.java

`public class InvalidVerificationTokenException extends RuntimeException`

One message for unknown, used, replaced, expired and outdated links, so the response says nothing about which tokens exist.

customer/repository

customer/repository/CustomerRepository.java

`select c from Customer c`

The customer with roles and permissions already loaded, in one query, for issuing access tokens. Without it, reading the permissions costs one query for the roles plus one per role.

customer/repository/EmailVerificationTokenRepository.java

`UPDATE EmailVerificationToken t`

Retires every link still open for the customer, so only the newest one ever works.

customer/service

customer/service/CustomerService.java

`public CustomerResponse register(`

Registers a customer. Built for heavy sign-up traffic:

- The password is hashed first, outside any transaction. BCrypt is deliberately slow CPU work, and inside a transaction it would keep a pooled database connection idle for the whole hash, capping registrations at pool size / hash time.
- Duplicate email or phone is detected by the unique constraints, not by "does it exist?" queries first: fewer round trips, and two concurrent sign-ups with the same email get a clean 409 instead of racing past the check.
- Everything else (customer, role, identity, KYC profile) is written in one short transaction.

Not @Transactional: the transaction is opened explicitly after hashing.

`Customer savedCustomer =`

Flush first so a duplicate email or phone fails here, before the role, identity and KYC rows are written.

`roleAssignmentService.assignRole(`

Public registration always gets CUSTOMER; the client never chooses a role. performedBy is null because the system assigns it.

`eventPublisher.publishEvent(`

Creates the KYC profile in this transaction.

`public Customer createVerifiedCustomer(String firstName, String lastName, String email)`

A customer whose email an identity provider has already verified (sign-up with Google): active straight away, with the CUSTOMER role and a KYC profile, but no password and no phone number yet. The caller adds the sign-in identity in the same transaction.

`public CustomerPageResponse listCustomers(`

Admin list, newest first. `search` matches email, first or last name (case-insensitive, substring); blank lists everyone.

`identityService.changePasswordIdentitySubject(customer, newEmail);`

The password identity's subject is the email; left as the old address it would block anyone registering with it.

`eventPublisher.publishEvent(new CustomerEmailChangedEvent(customer.getId()));`

Unverified again: a link goes to the new address.

customer/service/EmailVerificationListener.java

`public class EmailVerificationListener`

Sends a verification link once a registration or email change has committed. A failure here is logged, never thrown: the registration has already succeeded, and the customer can ask for another link.

customer/service/EmailVerificationService.java

`public class EmailVerificationService`

Proves a customer owns their email address, and activates a new customer once they do (PENDING_VERIFICATION -> ACTIVE).

A link holds a 256-bit random token; only its SHA-256 hash is stored. It works once, for 24 hours, only for the address it was sent to, and only while it is the customer's newest link.

`public void sendLink(UUID customerId)`

Sends a link after registration or an email change, unless the address is already verified (e.g. a seeded demo customer). Runs in its own transaction: it is called after the registration committed.

`public void resend(UUID customerId)`

The customer asks for another link: at most one a minute and five a day, so the endpoint cannot be used to flood a mailbox.

`public CustomerResponse verify(String rawToken)`

Uses a link. Every kind of bad link gets the same answer.

`.filter(found -> found.getEmail().equals(token.getEmail()))`

Sent before an email change: it proves nothing about the current address.

`TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization()`

Only once the token is committed: a rolled-back request sends nothing, and a slow mail server holds no database connection.

demo

demo/DemoAdminGuard.java

`public class DemoAdminGuard implements HandlerInterceptor`

The demo admin is shared by every visitor of the public preview, so no visitor may lock the others out of it: it cannot be edited, suspended, closed or have its roles changed, and nobody can end every session of it at once. The demo recipient, which every visitor sends money to, is protected the same way, and its account cannot be frozen, closed or drained. Admin actions on any other customer work normally.

`private static final List<String[]> PROTECTED = List.of(`

Method and path pattern of every request that changes the customer named by {customerId}.

`private static final List<String[]> PROTECTED_RECIPIENT_ACCOUNT = List.of(`

Requests that would take the demo recipient's account out of use for every visitor: freezing or closing it, or draining it (which would make transfers into it impossible to reverse).

`if ("PATCH".equals(method)`

Changing its own email would lock every visitor out of it.

`private boolean targetsDemoRecipient(String method, String path)`

Whether the request changes the demo recipient or its account. The recipient is only looked up for requests that match a rule, and never cached, since the seeder may recreate it.

`private Set<UUID> demoStaffIds()`

The demo admin and the demo operations officers. Cached once all are found; they are seeded at startup and never deleted.

demo/DemoAdminSeeder.java

`public class DemoAdminSeeder`

Creates the shared demo staff once the application is ready, or repairs them: after every restart the admin is ACTIVE with the CUSTOMER and ADMIN roles, each operations officer (`DemoStaff`) is ACTIVE with the OPERATIONS role, and all accept the configured password.

`String password = properties.getAdminPassword();`

Fail startup, not the first login, when the password is missing.

`public void seed()`

Runs before DemoRecipientSeeder, which records the admin as activating its account.

`private void seedOperationsOfficer(DemoStaff staff)`

The same treatment for an operations officer: ACTIVE, the OPERATIONS role only (no customer app, no admin powers), the demo password.

`Customer officer =`

Not a registration: officers are employees, not customers, so they get no KYC profile (and no place in the KYC review queue).

`eventPublisher.publishEvent(new CustomerRegisteredEvent(admin.getId()));`

Same side effects as a normal registration (its KYC profile).

demo/DemoConfiguration.java

`public class DemoConfiguration implements WebMvcConfigurer`

Everything in this package is active only when `paycore.demo.enabled=true`.

demo/DemoController.java

`public class DemoController`

Tells the demo UI which shared admin account visitors can sign in with, and which demo account they can send money to. Publishing a password is deliberate here and only acceptable because the endpoint exists solely in demo mode, on sandbox data. Without demo mode the path does not exist.

`public List<DemoOutbox.DemoEmail> outbox(@AuthenticationPrincipal CurrentUser currentUser)`

The emails sent to the signed-in visitor's own address, newest first: how a demo visitor with a made-up address opens their verification link. Needs a sign-in (every path here but GET /api/v1/demo does).

`public record DemoInfoResponse(`

@param kycProvider which identity provider answers BVN/NIN checks, so the UI can say when results are simulated @param recipientName who the demo recipient is, or null before it has been seeded @param recipientAccountNumber an active NGN account visitors can transfer to, or null @param fundingProvider which payment provider takes customer top-ups, or null if card top-ups are off @param bankRailProvider which rail carries transfers to and from other banks, or null if none @param operationsStaff the shared maker and checker officers

`public record DemoStaffInfo(String email, String name, String duty)`

A shared operations officer: signs in with the demo admin password.

demo/DemoOutbox.java

`public class DemoOutbox`

Keeps the emails PayCore sent so Developer Preview visitors, who sign up with made-up addresses, can still open their verification link. Each signed-in visitor only ever sees mail addressed to their own email.

In memory and bounded: the latest few emails for the most recent addresses. A restart empties it; the visitor then asks for a new link.

`private final Map<String, Deque<DemoEmail>> byAddress =`

Least recently written address first, evicted beyond the limit.

`public synchronized List<DemoEmail> emailsTo(String address)`

Newest first.

demo/DemoProperties.java

`public class DemoProperties`

Developer Preview mode. When enabled, a shared admin account is seeded so visitors of the public demo can walk through the admin side of the flow. Its credentials are published by `DemoController`, so demo mode must never be enabled on an environment holding real customer data.

`public String normalizedAdminEmail()`

Stored the way registration stores emails.

demo/DemoRecipientSeeder.java

`public class DemoRecipientSeeder`

A fixed customer with an active NGN account that Developer Preview visitors can send money to; each visitor otherwise has only their own account. It has no password, so nobody can sign in as it, and it is not staff, so an admin can reverse a transfer into it.

Created or repaired after every restart, after the demo admin (who is recorded as having activated its account). Its account is opened directly rather than through KYC review: it stands in for "someone at another customer", not for a real applicant.

`kycProfileRepository`

Every visitor sends money here, so it gets the tier with no balance cap; it has no applicant to put through review.

`public Optional<Account> findAccount()`

The recipient's NGN account, if one is open, for the UI to name as a transfer destination.

`eventPublisher.publishEvent(new CustomerRegisteredEvent(recipient.getId()));`

Same side effects as a normal registration (its KYC profile).

`String accountNumber = accountNumberGenerator.generate();`

Skips numbers already in use; the unique constraint still has the final say, and startup runs one seeder at a time.

`private void ensureActive(Account account, Customer admin)`

PENDING or FROZEN -> ACTIVE, audited as done by the demo admin.

demo/DemoStaff.java

`public record DemoStaff(`

The shared operations officers of the Developer Preview, seeded next to the demo admin with the same (public) password. There are two because corrections need two people: one requests, the other approves.

device/config

device/config/DeviceProperties.java

`private Duration coolingOff = Duration.ofHours(24);`

How long spending stays capped after a new device is confirmed.

`private BigDecimal coolingOffLimit = new BigDecimal("20000");`

The most a customer can send, in total, during a cooling-off period (major units of the account's currency, e.g. 20000 naira).

device/controller

device/controller/DeviceController.java

`public class DeviceController`

The caller's own devices. "current" is the device of the login session the access token belongs to.

`public CodeSentResponse sendConfirmationCode(@AuthenticationPrincipal CurrentUser currentUser)`

Emails a code that confirms this device. At most one a minute.

`public void revoke(`

Removes a device and signs it out. Only from a trusted device.

device/dto

device/dto/CodeSentResponse.java

`public record CodeSentResponse(String sentTo, Instant expiresAt)`

A code went to the customer's email; it works until expiresAt.

device/dto/DeviceResponse.java

`public record DeviceResponse(`

A device as its owner sees it. current marks the one this request came from; coolingOffUntil is set while spending is capped after confirming it.

device/entity

device/entity/CustomerDevice.java

`public class CustomerDevice`

A browser or phone a customer signs in from, known by the SHA-256 of the random id the client generated for itself.

`public static CustomerDevice firstDevice(UUID customerId, String deviceHash, String label)`

The first device a customer ever uses: trusted, no cooling-off.

`public static CustomerDevice pending(UUID customerId, String deviceHash, String label)`

Any later device: it must be confirmed before it moves money.

`public void trust(Instant coolingOffUntil)`

Confirmed with an emailed code. Until coolingOffUntil the customer's spending is capped, in case it was not the customer who confirmed it.

`public void signedIn(String label)`

Signed in again. A revoked device has to be confirmed again.

device/enums

device/enums/DeviceStatus.java

`public enum DeviceStatus`

```
PENDING --confirm with emailed code--> TRUSTED --revoke--> REVOKED
```

Only a TRUSTED device can move money. A customer's very first device is TRUSTED at once: there is nothing older to confirm it from. A REVOKED device that signs in again starts over as PENDING.

device/exception

device/exception/DeviceNotTrustedException.java

`public class DeviceNotTrustedException extends RuntimeException`

Money can only move from a device the customer confirmed.

device/repository

device/repository/CustomerDeviceRepository.java

`boolean existsByCustomerIdAndTrustedAtIsNotNull(UUID customerId);`

Whether the customer has ever had a trusted device, revoked or not.

`Optional<CustomerDevice> findBySessionId(@Param("sessionId") UUID sessionId);`

The device a login session was opened on, if it named one.

`SELECT MAX(d.coolingOffUntil) FROM CustomerDevice d`

The latest end of a cooling-off period on any of the customer's devices, revoked ones included: revoking a newly added device does not lift the cap it brought.

`SELECT 1 FROM (SELECT pg_advisory_xact_lock(hashtextextended(CAST(:customerId AS text), 0))) AS locked`

Serializes device registration per customer for the rest of the transaction, so two new devices signing in at once cannot both be taken for the customer's first.

device/service

device/service/DeviceLabels.java

`final class DeviceLabels`

A name a customer recognises their device by ("Chrome on Windows"), guessed from the User-Agent. Only for display.

device/service/DeviceService.java

`public class DeviceService`

Device binding. Each login names the device it came from; money moves only from a TRUSTED one. A customer's first device is trusted at once; every later one is confirmed with a code sent to their email, and then caps their spending for a cooling-off period, so someone who learned the password and got into the mailbox still cannot empty the account.

`private static final Pattern DEVICE_ID = Pattern.compile("[A-Za-z0-9_-]{16,128}");`

What a client may send as X-Device-Id: a random id it made itself.

`public CustomerDevice recognize(UUID customerId, String rawDeviceId, String userAgent)`

The device a login comes from, registered on first sight. Null when the client sent no usable device id: the session then works for everything except moving money. Runs in the login's transaction.

`public CustomerDevice requireTrusted(UUID sessionId)`

The trusted device a session was opened on.

@throws DeviceNotTrustedException when the session named no device, or the device is not trusted

`public DeviceResponse current(UUID sessionId)`

The device this session runs on; 404 when the client named none.

`public CodeSentResponse sendConfirmationCode(UUID customerId, UUID sessionId)`

Emails a code that confirms the device this session runs on.

`public DeviceResponse confirm(UUID customerId, UUID sessionId, String code)`

Confirms the device this session runs on with the emailed code. A wrong code counts against the code's attempts.

`public void revoke(UUID customerId, UUID sessionId, UUID deviceId)`

Removes a device: its sessions end at once, and it has to be confirmed again before it can move money. Only a trusted device can remove devices, so a newly added one cannot lock its owner out.

`public Optional<Instant> coolingOffUntil(UUID customerId)`

When the customer's spending cap from a newly confirmed device ends; empty when no cooling-off is running.

`public static String maskEmail(String email)`

esther@example.com -> e*****@example.com

funding/config

funding/config/FundingProperties.java

`public class FundingProperties`

`paycore.funding.provider` picks the payment provider ("simulated" or "none"); it is read by the provider's own condition, not here.

`maxTotalPerAccount` caps what provider top-ups may add to one account in total, in major units of its currency (500000 = NGN 500,000). Reversed top-ups free their share again.

funding/controller

funding/controller/FundingController.java

`public class FundingController`

Customers adding money to their own accounts through the payment provider. Only ever the caller's own accounts (404 otherwise).

`public ResponseEntity<?> fund(`

201 with the deposit once the provider confirmed the payment; 422 PAYMENT_DECLINED if it did not, with nothing credited. 202 with the payment when the provider gave no answer: the customer may have been charged, and PayCore is finding out (GET the Location to follow it). A retry with the same idempotency key returns the same outcome and charges nothing.

`public FundingAllowanceResponse allowance(`

Whether top-ups are available, and how much more can be added.

funding/exception

funding/exception/FundingDeclinedException.java

`public class FundingDeclinedException extends RuntimeException`

The provider answered and refused the payment. Nothing was credited.

funding/exception/FundingProviderException.java

`public class FundingProviderException extends RuntimeException`

The provider could not give an answer (outage, bad credentials), as opposed to declining the payment. Details are logged, not returned.

funding/provider

funding/provider/CardPaymentRecord.java

`public record CardPaymentRecord(`

The simulated card processor's own record of one charge, as a real processor keeps it: keyed by PayCore's merchant reference, with the processor's reference and what became of it. status is CONFIRMED, DECLINED or REFUNDED; settled is whether it is in the processor's settlement report (V30).

funding/provider/CardPaymentRecords.java

`public interface CardPaymentRecords`

Where the simulated card processor keeps its records. In the running application they are in the database (simulator.card_payments, V28), so a restart forgets nothing; unit tests use an in-memory copy.

`CardPaymentRecord saveIfAbsent(CardPaymentRecord record);`

Stores the record unless one already exists for its merchant reference, and returns the one that is stored: a repeated or concurrent charge for the same payment gets the first answer.

`CardPaymentRecord markRefunded(String merchantReference, String refundReference, Instant at);`

Marks a CONFIRMED charge REFUNDED under refundReference; a charge already refunded keeps its first refund. Returns the stored record.

`List<CardPaymentRecord> between(Instant from, Instant to);`

Every charge created in [from, to), oldest first.

funding/provider/CardSettlementLine.java

`public record CardSettlementLine(`

One line of the card processor's settlement report: a charge, keyed by PayCore's merchant reference, the amount the processor actually took, and what became of it (CONFIRMED, DECLINED or REFUNDED).

funding/provider/FundingCollection.java

`public record FundingCollection(`

A payment to take from the customer. merchantReference is PayCore's own reference for it, derived from the customer's idempotency key, so a provider that deduplicates by reference never charges a retry twice.

funding/provider/FundingProvider.java

`public interface FundingProvider`

The only payment-provider contract the domain knows: something that takes a customer's payment into PayCore's settlement bank account and says whether it succeeded. Implementations (the demo simulator today; Paystack, Flutterwave, Monnify, ... later) translate vendor responses into `FundingResult`. At most one is active, chosen by `paycore.funding.provider`; with none, customers cannot top up.

The simulator answers at once. A real provider usually confirms later, through a webhook; that webhook would end in the same ledger posting, keyed by the provider's payment reference.

`FundingResult collect(FundingCollection collection);`

@throws com.fintechplatform.paycore.funding.exception.FundingProviderException when the provider cannot give an answer (outage, bad credentials), as opposed to a declined payment

`FundingStatus status(String merchantReference);`

What happened to the payment PayCore asked for under merchantReference: how a top-up whose collect gave no answer (or whose answer PayCore never recorded) is settled.

@throws com.fintechplatform.paycore.funding.exception.FundingProviderException when the provider cannot answer

`String refund(String merchantReference);`

Gives a confirmed charge back to the customer's card, and returns the provider's refund reference. Refunding a charge already refunded returns the first refund: a payment is refunded at most once.

@throws com.fintechplatform.paycore.funding.exception.FundingProviderException when the provider cannot answer, or has no charge to refund

`List<CardSettlementLine> settlementReport(Instant from, Instant to);`

The processor's settlement report: every charge it settled to PayCore's bank account, created in [from, to). What reconciliation compares PayCore's books with.

@throws com.fintechplatform.paycore.funding.exception.FundingProviderException when the provider cannot answer

`String providerName();`

Stored on every deposit it confirms, e.g. "SIMULATED".

`String narration();`

What customers read on their statement for a top-up.

funding/provider/FundingResult.java

`public record FundingResult(`

The provider's answer. providerReference identifies the payment at the provider; reason says why a payment was declined (null when confirmed).

funding/provider/FundingStatus.java

`public record FundingStatus(Outcome outcome, String providerReference, String reason, String refundReference)`

The provider's answer when asked what happened to a payment, by PayCore's merchant reference.

```
CONFIRMED  the customer was charged (providerReference identifies it)
DECLINED   the charge was refused; nothing was taken
REFUNDED   the customer was charged, then refunded (refundReference)
PENDING    not settled yet; ask again later
NOT_FOUND  the provider has no such payment: the request never arrived
```

funding/provider/JdbcCardPaymentRecords.java

`public class JdbcCardPaymentRecords implements CardPaymentRecords`

The simulated processor's records in simulator.card_payments. Each call commits on its own (REQUIRES_NEW), even when PayCore calls the provider from inside one of its transactions: the processor is another system, and what it did stands whatever PayCore does next.

funding/provider/SimulatedFundingProvider.java

`public class SimulatedFundingProvider implements FundingProvider`

In-process stand-in for a payment provider, for the public Developer Preview and local development. It never calls out and no real money moves: it pretends the customer paid by card and confirms at once.

The kobo decide what happens, so every path can be tried:

```
.99  declined
.98  charged, but the answer is lost (timeout): PayCore must find out
     with a status check and then credit the customer
.97  the request never reaches the provider (timeout): a status check
     finds nothing, and nothing was charged
.96  charged, the answer is lost, and status queries for it never
     answer either: only operations can resolve it (credit or refund)
.94  confirmed, but the processor charged N1.00 more than asked: only
     reconciliation sees the difference
.92  confirmed, but missing from the processor's settlement report (it
     never settled the money to PayCore's bank): only reconciliation
     sees it
else confirmed
```

Like a real provider it deduplicates by merchant reference (asking again for the same payment returns the first answer), answers status queries and refunds. It keeps its records in its own table (`CardPaymentRecords`), so a restart forgets nothing. It must never be enabled where real customers hold accounts: it creates balances nothing pays for.

`public static final long DECLINED_MINOR_ENDING = 99;`

The minor-unit ending that is always declined.

`public static final long CHARGED_THEN_TIMEOUT_MINOR_ENDING = 98;`

Charged, but the answer never reaches PayCore.

`public static final long LOST_IN_TRANSIT_MINOR_ENDING = 97;`

Times out before reaching the provider: nothing is charged.

`public static final long STATUS_UNAVAILABLE_MINOR_ENDING = 96;`

Charged, the answer is lost, and its status can never be queried.

`public static final long OVERCHARGED_MINOR_ENDING = 94;`

Confirmed, but N1.00 more is charged than PayCore asked for.

`public static final long UNSETTLED_MINOR_ENDING = 92;`

Confirmed, but never settled: missing from the settlement report.

funding/service

funding/service/FundingOutcome.java

`public record FundingOutcome(TransactionResponse transaction, PaymentResponse payment)`

What a top-up request came to: the deposit once it is credited, or the payment while the provider's answer is still unknown (then transaction is null and the payment says what PayCore is doing about it).

funding/service/FundingService.java

`public class FundingService`

Customers adding money to their own accounts through the payment provider. The customer only asks; the money is credited because the provider confirmed the payment, never because the customer said so.

```
1. ledger   a retry of a top-up that already posted? return it, charge nothing
            otherwise: own account, active, right currency, within limit?
2. payment  a retry of one still being settled? return it, charge nothing
            otherwise record it, PROCESSING: one key, one payment
3. provider take the payment (no database transaction is open meanwhile)
4. confirmed: credit it and mark it SUCCEEDED, together
   declined:  FAILED, nothing credited
   no answer: UNKNOWN. The customer may have been charged, so this is
              never reported as "not charged": a status check with the
              provider settles it (PaymentRecoveryService)
```

Not transactional itself: each step is its own transaction, so no row lock is held while the provider answers.

`Money amount = Money.ofMajor(request.amount(), Currency.of(request.currency()));`

Valid by now: checkFunding parsed it.

`if (ConstraintViolations.violates(exception, PAYMENT_KEY_CONSTRAINT))`

A concurrent duplicate of this request recorded its payment first; it is the one being charged.

`public Payment settle(Payment payment)`

SETTLING AN UNKNOWN OUTCOME

Asks the provider what happened to a top-up whose answer PayCore does not have, and acts on it: credits a confirmed charge, fails a declined or never-received one. No answer leaves it UNKNOWN for the next check (or, checks used up, for operations). The check must have been claimed (PaymentService.claimStatusCheck).

`(end of block)`

Logged and sent to review by credit().

`public FundingStatus currentStatus(Payment payment)`

The provider's answer about the payment now, or null if it gave none (unreachable, or not the provider that took the payment).

`private TransactionResponse credit(Payment payment, FundingTerms terms, String providerReference)`

HELPERS

Credits a charge the provider confirmed. If the credit is refused (the account was frozen meanwhile, the limit was reached by another top-up), the customer has paid and nothing was credited: that must not disappear into a 4xx, so the payment goes to review for operations to credit or refund.

`private FundingOutcome replay(Payment payment, UUID accountId, Money amount)`

A retry of a top-up that already has a payment: the same request gets that payment's outcome and is never charged again.

`public FundingTerms terms()`

The terms top-ups through the configured provider are credited on.

`public String refund(Payment payment)`

Gives the payment's charge back to the card, through the provider that took it. Returns the refund reference; refunding twice returns the first.

`public static String merchantReference(UUID customerId, String idempotencyKey)`

PC-<32 hex>: stable for one customer's idempotency key, so a provider that deduplicates by merchant reference sees a retry as the same payment. Hashed so the key itself never leaves PayCore. V26 computes the same value in SQL for the payments it backfilled.

globalexception

globalexception/GlobalExceptionHandler.java

`public ResponseEntity<Map<String, Object>> handleKycProvider(`

Provider details (status codes, credential problems) are logged, not returned to the client.

`public ResponseEntity<Map<String, Object>> handleInvalidPaymentTransition(`

A lost race or a bug, never the customer's doing; the payment is unchanged.

`public ResponseEntity<Map<String, Object>> handleFundingDeclined(`

The provider's payment reference is returned so the customer can quote it; nothing was credited.

`public ResponseEntity<Map<String, Object>> handleFundingProvider(`

Provider details are logged, not returned to the client.

`public ResponseEntity<Map<String, Object>> handleNotificationLost(InboundNotificationLostException exception)`

Rail details are logged, not returned. Nothing was debited. Simulator: the money left Test Bank but PayCore was never told. 202: it is in flight.

`public ResponseEntity<Map<String, Object>> handleResolutionContradicted(ResolutionContradictsProviderException exception)`

The provider's definite answer wins over a staff decision.

`public ResponseEntity<Map<String, Object>> handleConcurrentModification(`

Two requests changed the same record at once (optimistic locking). Nothing was saved from the losing request; the client can reload and retry.

`public ResponseEntity<Map<String, Object>> handleDeviceNotTrusted(DeviceNotTrustedException exception)`
DEVICES, TRANSACTION PIN, ONE-TIME CODES, SPENDING LIMITS

`public ResponseEntity<Map<String, Object>> handleInvalidPin(InvalidTransactionPinException exception)`

422, not 401: the access token is fine, so clients must not refresh it.

`public ResponseEntity<Map<String, Object>> handleBillsDisabled(BillPaymentsDisabledException exception)`
BILL PAYMENTS

`public ResponseEntity<Map<String, Object>> handleBillProvider(BillProviderException exception)`

Details are in the logs; the customer is told to try again.

identity/config

identity/config/SessionProperties.java

`public class SessionProperties`

Login session lifetime, following common banking practice:

• `expiration`: absolute limit. After it the customer must sign in again however active they are.

• `idleTimeout`: the session ends when it has not been used to issue tokens (login or refresh) for this long. Active clients refresh at least once per access-token lifetime, so keep the access token well below this.

identity/controller

identity/controller/AuthenticationController.java

`public static final String DEVICE_ID_HEADER = "X-Device-Id";`

The random id the client keeps for itself; see DeviceService.

`public ResponseEntity<LoginResponse> signInWithGoogle(`

Sign in, or sign up, with a Google ID token. The same session and tokens as a password login.

`public AuthProvidersResponse providers()`

Which sign-in methods the UI should offer.

identity/dto

identity/dto/AuthenticationContext.java

`public record AuthenticationContext(`

Where a sign-in comes from. deviceId is the random id the client keeps for itself (the X-Device-Id header); null when it sent none.

identity/dto/AuthProvidersResponse.java

`public record AuthProvidersResponse(`

Which ways to sign in this server offers, so the UI only shows what works. The Google client id is public by design: it is in every page that shows a Google button.

identity/dto/GoogleSignInRequest.java

`public record GoogleSignInRequest(`

@param credential the ID token Google Identity Services handed the browser

identity/dto/IssuedRefreshToken.java

`public record IssuedRefreshToken(`

The raw token goes to the client exactly once; only its hash is stored.

identity/dto/SessionAccess.java

`public record SessionAccess(`

What the per-request access token check needs about a login session and its customer, read in one query.

`public boolean grantsAccessTo(UUID customerId, Instant now)`

Whether an access token naming this session may be used by the given customer: the session is theirs, not revoked, within its absolute lifetime, and the customer can still authenticate. The idle timeout needs no check here, because an access token expires well before a session it was issued for can go idle.

identity/entity

identity/entity/Identity.java

`public static Identity createGoogleIdentity(`

Signs in with Google. The subject is Google's stable account id ("sub"), never the email, which a Google user can change.

`public void changeProviderSubject(String providerSubject)`

For a password identity, whose subject is the customer's email.

identity/entity/LoginSession.java

`private CustomerDevice device;`

The device the session was opened on; null if the client named none.

`public boolean isIdle(Duration idleTimeout)`

True when the session has not been used for longer than `idleTimeout`.

identity/entity/RefreshToken.java

`private UUID familyId;`

Every token produced by rotating the same login's refresh token shares one family, so reuse of any old token can revoke them all.

identity/exception

identity/exception/GoogleSignInDisabledException.java

`public class GoogleSignInDisabledException extends RuntimeException`

No Google client id is configured (paycore.google.client-id).

identity/exception/RefreshTokenReuseException.java

`public class RefreshTokenReuseException`

A refresh token that was already rotated or revoked was presented again. The whole token family and its session have been revoked, so the customer must re-authenticate.

identity/exception/SessionTimedOutException.java

`public class SessionTimedOutException extends RuntimeException`

The login session can no longer issue tokens: it was idle too long or reached its absolute lifetime. The client must sign in again.

identity/google

identity/google/GoogleConfiguration.java

`public GoogleIdTokenVerifier googleIdTokenVerifier(GoogleProperties properties)`

Deliberately not a JwtDecoder bean: the application's own JwtDecoder (for PayCore access tokens) must stay the only one.

identity/google/GoogleIdentity.java

`public record GoogleIdentity(`

The verified claims of a Google ID token that PayCore uses.

@param subject Google's stable, never-reused account id ("sub")

identity/google/GoogleIdTokenVerifier.java

`public interface GoogleIdTokenVerifier`

Checks that an ID token was issued by Google, for PayCore, and has not expired, and returns its claims.

`GoogleIdentity verify(String idToken);`

@throws com.fintechplatform.paycore.identity.exception.InvalidGoogleTokenException if the token is forged, expired, for another app or malformed

identity/google/GoogleProperties.java

`public class GoogleProperties`

Sign in with Google. Set paycore.google.client-id (PAYCORE_GOOGLE_CLIENT_ID) to the OAuth client id of a "Web application" client in Google Cloud Console to turn it on; empty (the default) leaves it off. No client secret is needed: PayCore only verifies ID tokens.

`private String jwkSetUri = "https://www.googleapis.com/oauth2/v3/certs";`

Google's signing keys, fetched once and cached by the decoder.

identity/google/GoogleSignInService.java

`public class GoogleSignInService`

Sign in (and sign up) with Google.

The browser gets an ID token from Google; PayCore verifies it, then:

• a Google account seen before signs in to its customer;

• otherwise, a customer with the same email is linked to it;

• otherwise, a new customer is created, active with a verified email but no phone number yet.

Linking is where the risk is. If the existing customer never verified their email, whoever set their password never proved they own the address; an attacker could have registered it first, waiting for the real owner to arrive ("pre-account takeover"). Google has now proved ownership, so that password is disabled and every session of the customer is ended, and the real owner continues with Google.

`public LoginResult signIn(String idToken, AuthenticationContext context)`

Not @Transactional: verifying the token may fetch Google's keys, which must not hold a database connection. The writes run in one transaction afterwards.

`return transactionTemplate.execute(status -> signInVerified(google, context));`

The same Google account signing in twice at once: the other request created the customer or identity first. Use it.

`eventPublisher.publishEvent(new CustomerAccessRevokedEvent(customer.getId()));`

Ends every session and refresh token, in this transaction.

identity/google/JwtGoogleIdTokenVerifier.java

`public class JwtGoogleIdTokenVerifier implements GoogleIdTokenVerifier`

Verifies Google ID tokens with Spring Security's JWT support: the RS256 signature against Google's published keys, then the issuer, the audience (PayCore's client id) and the expiry. This is what Google documents for back-end sign-in; no call to Google per sign-in beyond the cached keys.

`public static JwtGoogleIdTokenVerifier forGoogle(GoogleProperties properties)`

The production verifier, reading keys from Google.

`public static OAuth2TokenValidator<Jwt> validator(String clientId)`

Issuer, audience and expiry checks; public so tests can apply them to tokens signed with a local key.

`OAuth2TokenValidator<Jwt> issuer = jwt ->`

Read as a string: Google also issues "accounts.google.com" without a scheme, which Jwt.getIssuer() fails to parse as a URL.

identity/repository

identity/repository/LoginSessionRepository.java

`select new com.fintechplatform.paycore.identity.dto.SessionAccess(`

The session and customer state checked on every authenticated request, in one query.

identity/service

identity/service/AuthenticationService.java

`public LoginResult login(`

Built for heavy login traffic:

- The customer is read together with their roles and permissions in one query, so issuing the access token needs no further reads.
- The BCrypt check runs outside any transaction. It is deliberately slow CPU work; inside a transaction it would keep a pooled database connection idle for the whole comparison.
- Only the writes (session and refresh token) run in a transaction, and only once the password is known to be right.

Not @Transactional: the transaction is opened explicitly after the password check.

`public LoginResult startSessionFor(`

For a customer who proved who they are another way (Google sign-in): the same status check and the same session and tokens as a password login. Runs in the caller's transaction.

`Customer customer =`

Refresh runs every few minutes for every active user, so the customer, roles and permissions come back in one query rather than one per role.

`throw new InvalidRefreshTokenException();`

Rolls back the rotation; the customer simply cannot refresh.

`public void logout(String rawToken)`

Revokes the session identified by the raw session token together with its refresh tokens. A missing token is a no-op so logout stays idempotent for clients that already discarded it.

identity/service/CustomerAccessRevocationListener.java

`public class CustomerAccessRevocationListener`

Ends every session and refresh token of a customer who was suspended or closed, so reactivating the account later does not bring old sessions back: the customer has to sign in again.

identity/service/IdentityService.java

`public Identity createPasswordIdentity(`

One password identity per customer is enforced by the unique index uk_identity_customer_provider (V3), not by a query before the insert.

`public void changePasswordIdentitySubject(Customer customer, String email)`

Keeps a password identity's subject equal to the customer's email. A customer without one (Google only) has nothing to update.

identity/service/LoginSessionService.java

`public void ensureNotTimedOut(LoginSession session)`

Refuses a session that passed its absolute lifetime or sat idle for longer than the idle timeout, saying which, so the client can tell the user why they were signed out. Revoked sessions are not checked here.

identity/service/RefreshTokenService.java

`public IssuedRefreshToken issue(`

Starts a new token family for a freshly created login session.

`public IssuedRefreshToken rotate(String rawToken)`

Exchanges a refresh token for a new one in the same family. Presenting a token that was already rotated revokes the whole family and its session. The revocation must be committed even though the exception propagates, hence noRollbackFor.

`if (current.getReplacedBy() != null)`

Rotated tokens are handed out exactly once; seeing one again means it was copied. Tokens revoked by logout are just stale.

`if (!session.isRevoked())`

Before the token's own expiry, so the client learns whether the session idled out or hit its absolute limit.

`Instant expiresAt =`

Never outlives its session's absolute limit.

kyc/config

kyc/config/KycProperties.java

`public class KycProperties`

Limits failed BVN checks so the endpoint cannot be used to guess other people's BVNs (or burn provider credit): at most `maxFailedAttempts` FAILED results within any rolling `attemptWindow`.

`private int ipMaxAttempts = 10;`

Per client network: at most `ipMaxAttempts` BVN checks (of any result) within any rolling `ipAttemptWindow`, across all customers. Stops one network from spreading guesses over many accounts.

kyc/controller

kyc/controller/KycController.java

`public class KycController`

Customer endpoints act on the caller's own KYC (resolved from the access token), so one customer can never touch another's profile. They need KYC_READ or KYC_SUBMIT. Review endpoints need KYC_REVIEW and refuse self-review.

`public ResponseEntity<KycDocumentResponse> uploadDocument(`

Multipart upload of a PDF, PNG or JPEG (max 5 MB). Only metadata and the storage key are kept in the database.

`public KycVerificationResponse verifyBvn(`

The client IP comes from the servlet request's remote address, never from a raw X-Forwarded-For header (which clients can forge). Behind a reverse proxy, the prod profile (application-prod.properties) makes Tomcat set the remote address from X-Forwarded-For, but only for requests arriving from trusted proxies.

`public KycVerificationResponse verifyNin(`

National Identification Number check; an alternative to the BVN. Same client-IP handling and limits as `#verifyBvn`, counted separately per check type.

`public KycReviewQueueResponse listProfiles(`

Review queue, most recently changed first. Omit `status` to list profiles in every state.

`public BvnAttemptsResponse getBvnAttempts(`

Admin view of the customer's BVN retry limit and attempt history. Read-only, so unlike the review actions it is also allowed on the admin's own profile.

History is paginated (`page` is zero-based, newest first); the limit summary describes the current state on every page.

`public BvnAttemptsResponse getNinAttempts(`

Same view as `#getBvnAttempts` for NIN checks.

`public BvnAttemptsResetResponse resetBvnAttempts(`

Gives the customer a fresh set of BVN (and NIN) attempts. Failed attempts are kept as history; only the retry-limit count starts over.

kyc/controller/KycTierController.java

`public class KycTierController`

KYC tiers: the caller's tier and limits, asking for tier 3 with a proof of address, and compliance deciding those requests.

`public KycTierResponse requestUpgrade(`

A proof of address (PDF, PNG or JPEG, max 5 MB) for tier 3.

kyc/dto

kyc/dto/BvnAttemptResponse.java

`public record BvnAttemptResponse(`

One recorded BVN check. The BVN itself was never stored, so it cannot appear here. `countsTowardLimit` is true for FAILED checks inside the current window (and after any reset). `ipAddress` is the normalized client network (IPv6 as its /64); null for attempts recorded before networks were tracked.

kyc/dto/BvnAttemptsResponse.java

`public record BvnAttemptsResponse(`

Admin view of a customer's BVN retry limit and attempt history.

The limit summary (everything except `attempts` and `page`) always describes the customer's current state and is the same on every page; only the history is paginated.

@param attemptWindow ISO-8601 duration of the rolling window, e.g. "PT24H" @param countingSince failures after this instant count toward the limit (window start, or the last reset if later) @param retryAfter when the customer may try again; null unless limited @param attempts one page of BVN checks, newest first @param page paging metadata for `attempts`

kyc/dto/KycDocumentResponse.java

`public record KycDocumentResponse(`

Deliberately omits the document number.

kyc/dto/KycResponse.java

`public record KycResponse(`

`reviewReason` is the latest reviewer's explanation for a rejection or an information request; null otherwise.

kyc/dto/KycReviewItemResponse.java

`public record KycReviewItemResponse(`

One profile in the reviewer's queue: enough to decide what to open, without document numbers or any provider identity data.

@param bvnPassed whether any BVN check on the profile PASSED @param ninPassed whether any NIN check on the profile PASSED @param documentTypes types of the uploaded documents, oldest first

kyc/dto/KycReviewRequest.java

`public record KycReviewRequest(`

Reviewer's explanation for a rejection or an information request. The customer sees it.

kyc/dto/KycTierResponse.java

`public record KycTierResponse(`

A customer's KYC tier and what it allows.

@param dailySend what the tier may send per day @param sentToday sent so far today @param maxBalance what the account may hold; null for no cap @param resetsAt when "today" ends @param nextTier the tier above, null at the top @param nextTierNeeds what the customer must do to reach it @param upgrade the latest request for a higher tier, if any

kyc/dto/KycTierUpgradeResponse.java

`public record KycTierUpgradeResponse(`

A tier upgrade request as a compliance reviewer sees it.

kyc/dto/KycVerificationResponse.java

`public record KycVerificationResponse(`

Never echoes the identity data the provider returned. `remainingAttempts` is how many more FAILED BVN checks are allowed in the current window before checks are blocked.

kyc/dto/PageInfo.java

`public record PageInfo(`

Paging metadata for a list response. `number` is zero-based.

kyc/entity

kyc/entity/KycDocument.java

`public class KycDocument`

Metadata for an uploaded document. The file itself lives in object storage; only its storage key is kept here.

kyc/entity/KycProfile.java

`public class KycProfile`

The KYC aggregate. Its status is independent of Customer.status and only changes through the transition methods below, never a setter.

```
NOT_STARTED -> IN_PROGRESS -> SUBMITTED -> UNDER_REVIEW
UNDER_REVIEW -> VERIFIED | REJECTED | ADDITIONAL_INFO_REQUIRED
ADDITIONAL_INFO_REQUIRED -> IN_PROGRESS
```

Verification results (e.g. a passed BVN check) never move the status: the customer submits the package and a reviewer decides.

The tier (0-3) says how much is known about the customer, and with it how much they may send and hold. It is 0 until approval and only ever goes up: see `#approve(UUID, int)` and `#raiseTier(int)`.

`private Instant bvnAttemptsResetAt;`

Failed BVN checks before this instant no longer count toward the retry limit. The attempts themselves are kept as history.

`private UUID bvnAttemptsResetBy;`

Customer id of the admin who performed the most recent reset.

`private String reviewReason;`

Why the most recent reviewer rejected the profile or asked for more information; null after an approval. Shown to the customer.

`private UUID reviewedBy;`

Customer id of the reviewer who made the most recent decision.

`public void approve(UUID reviewerId, int tier)`

@param tier what the approved evidence earns, 1 to 3

`public void raiseTier(int tier)`

New evidence after approval (a second identity number, an approved proof of address). A tier never goes down here.

`public void verifyForSeededAccount(int tier)`

Verified without a review: only for accounts PayCore seeds itself (the Developer Preview recipient), which have no applicant behind them.

`public void resetBvnAttempts(UUID adminId)`

Gives the customer a fresh set of BVN attempts. Only meaningful while the customer can still run BVN checks.

kyc/entity/KycTierUpgrade.java

`public class KycTierUpgrade`

A verified customer asking for a higher KYC tier with new evidence (a proof of address for tier 3). A compliance reviewer decides, never the customer themselves.

kyc/entity/KycVerification.java

`public class KycVerification`

Immutable record of one provider verification attempt. Attempts are appended, never overwritten, and never store the raw identifier (BVN/NIN) that was checked.

`private String ipAddress;`

Normalized client network the attempt came from (see `com.fintechplatform.paycore.common.net.ClientNetwork`).

`public static KycVerification create(`

An attempt with no known client network.

kyc/enums

kyc/enums/KycTierUpgradeStatus.java

`public enum KycTierUpgradeStatus`

```
PENDING --approve--> APPROVED (the profile moves up a tier)
PENDING --reject---> REJECTED (the customer may ask again)
```

kyc/enums/VerificationResult.java

`public enum VerificationResult`

What the verification provider returned. Distinct from `KycStatus`, which tracks where the customer is in our process.

kyc/exception

kyc/exception/BvnAttemptLimitExceededException.java

`public class BvnAttemptLimitExceededException extends RuntimeException`

Too many FAILED identity-number checks (BVN, or NIN) in the rolling window. `retryAfter` is when the oldest counted failure leaves the window.

kyc/exception/BvnIpRateLimitExceededException.java

`public class BvnIpRateLimitExceededException extends RuntimeException`

Too many identity-number checks of one type (BVN, or NIN) from one client network in the rolling window, across all customers. `retryAfter` is when the oldest counted check from that network leaves the window.

kyc/exception/KycIncompleteException.java

`public class KycIncompleteException extends RuntimeException`

The KYC package is missing something it needs before it can be submitted for review. `missing` names each requirement.

kyc/provider

kyc/provider/DojahKycProvider.java

`public class DojahKycProvider implements KycProvider`

Dojah "Validate BVN" adapter: GET /api/v1/kyc/bvn with the customer's claimed names and date of birth. Dojah answers with a per-field match, e.g. {"entity":{"bvn":{"status":true},"first_name":{"status":true}}}.

A match on every supplied field is PASSED; an unknown BVN or any mismatch is FAILED. Outages, bad credentials, empty wallet and rate limits are provider errors, not verification results.

`return failed(root, describeError(root, "BVN could not be found"));`

Dojah reports an unknown BVN as {"error":"BVN not found"}.

`private String extractReference(JsonNode root)`

The documented Validate BVN response carries no reference; keep whichever one Dojah sends if that changes.

`public KycProviderResult verifyNin(`

Not wired to Dojah yet: its NIN response has not been verified against the sandbox, and guessing a vendor's response format would risk wrong PASSED/FAILED verdicts. Reported as a provider error.

kyc/provider/DojahProperties.java

`public record DojahProperties(`

Dojah credentials come from DOJAH_APP_ID / DOJAH_SECRET_KEY and must never be committed. Base URL defaults to the free Sandbox.

kyc/provider/KycProvider.java

`public interface KycProvider`

The only KYC vendor contract the domain knows. Implementations (Dojah and the demo simulator today; Smile Identity, Youverify, ... later) translate vendor responses into `KycProviderResult`. Exactly one is active, chosen by `paycore.kyc.provider`.

`KycProviderResult verifyBvn(`

@throws com.fintechplatform.paycore.kyc.exception.KycProviderException when the provider cannot give an answer (outage, bad credentials, rate limit), as opposed to a failed check

`KycProviderResult verifyNin(`

Same contract as `#verifyBvn`, for a National Identification Number.

kyc/provider/KycVerificationRequest.java

`public record KycVerificationRequest(`

Provider-neutral identity-number check (a BVN or a NIN). The number is held only for the duration of the provider call and is masked in toString so it cannot leak into logs.

kyc/provider/SimulatedKycProvider.java

`public class SimulatedKycProvider implements KycProvider`

In-process stand-in for a KYC vendor, for the public Developer Preview and local development without vendor credentials. It knows one test record per number type and never calls out, so it cannot break on a vendor outage, an expired key or an empty sandbox wallet.

Only a known test number whose names and date of birth all match the record PASSES; any other number or a mismatch FAILS with a reason, so retry limits and the review workflow behave as they would with a real provider. It verifies nobody's real identity, so it must never be enabled where real customers onboard.

kyc/repository

kyc/repository/KycProfileRepository.java

`Optional<KycProfile> findByCustomerForUpdate(`

Locks the profile row (SELECT ... FOR UPDATE) until the transaction ends, so concurrent BVN checks for one customer run one at a time and cannot both slip under the retry limit.

`Optional<KycProfile> findByIdForUpdate(@Param("id") UUID id);`

Same row lock by profile id, used by admin actions that must not interleave with an in-flight BVN check (e.g. resetting attempts).

`Optional<Short> findTierByCustomerId(@Param("customerId") UUID customerId);`

The customer's KYC tier; empty if they have no KYC profile.

kyc/repository/KycTierUpgradeRepository.java

`List<KycTierUpgrade> findByStatusOrderByRequestedAtAsc(KycTierUpgradeStatus status);`

Oldest first, with the customer and document, for the review queue.

kyc/repository/KycVerificationRepository.java

`Page<KycVerification> findByKycProfileAndVerificationType(`

One page of a profile's history for a verification type. Callers pass the sort in the `Pageable` (newest first for the admin view).

`long countByVerificationTypeAndIpAddressAndCreatedAtAfter(`

Attempts of one type from a client network after `since`, across all customers. Backs the per-IP BVN rate limit.

`Optional<KycVerification>`

Oldest counted attempt from a client network, which decides when that network may try again.

`Integer lockClientNetwork(@Param("networkKey") String networkKey);`

Transaction-scoped PostgreSQL advisory lock on a client network, so concurrent BVN checks from one network (even for different customers) are counted one at a time. Released automatically at commit/rollback. The two-int form with its own namespace (7201) cannot collide with Flyway's single-bigint advisory lock.

`List<KycVerification>`

Attempts of one type and result made after `since`, oldest first. Used to count failed BVN checks inside the retry window.

kyc/service

kyc/service/KycDocumentService.java

`public class KycDocumentService`

Uploads KYC document files. The bytes go to `KycDocumentStorage`; the database keeps only metadata and the storage key.

`public KycDocumentResponse upload(`

Only while KYC is IN_PROGRESS. The file type is detected from the content itself (PDF, PNG or JPEG); the client's file name and declared content type are not trusted. Uploading the same document type again keeps both, so a customer can send a clearer copy after an information request.

`requireValid(documentNumber, content);`

Before any lookup: a bad file is refused whatever the profile.

`public KycDocument store(`

Stores a document for a profile the caller has locked and checked may take one. The file goes away again if the transaction rolls back.

`private String requireValid(String documentNumber, byte[] content)`

@return the file extension its content shows

`private void deleteFileIfTransactionRollsBack(String storageKey)`

The file is written before the row is committed; if the commit fails, remove the file so storage holds no orphans.

kyc/service/KycRegistrationListener.java

`public class KycRegistrationListener`

Every customer needs KYC before they can transact, so the profile is created with the customer (NOT_STARTED) instead of on a separate call. The customer module only publishes the event and knows nothing of KYC.

kyc/service/KycService.java

`private static final Sort NEWEST_FIRST =`

id breaks ties between attempts saved in the same instant; UUIDv7 ids are time-ordered, so this stays chronological.

`public void createInitialProfile(UUID customerId)`

CUSTOMER

The NOT_STARTED profile of a customer who has just registered. No "already exists?" query: a brand-new customer cannot have one, and uk_kyc_profile_customer would reject a second anyway.

`public KycResponse startKyc(UUID customerId)`

NOT_STARTED or ADDITIONAL_INFO_REQUIRED -> IN_PROGRESS.

`public KycResponse submitKyc(UUID customerId)`

IN_PROGRESS -> SUBMITTED: the customer hands the package over for compliance review.

Requires a PASSED identity-number check (BVN or NIN) and at least one uploaded document. Both may come from an earlier round: after an information request the customer only adds what the reviewer asked for.

`if (!profile.isInProgress())`

State first, so e.g. a VERIFIED profile reports its state rather than what it would be missing.

`if (profile.getCustomer().getPhoneNumber() == null)`

Customers who signed up with Google have none until they add one.

`private KycVerificationResponse verifyIdentityNumber(`

Checks a BVN or NIN with the provider and records the attempt. The number itself is never persisted. Only allowed while KYC is IN_PROGRESS.

The result is one verification signal and never moves the KYC status: a PASSED check does not mean the whole package is ready. The customer submits separately and a reviewer decides. FAILED and PENDING results can be retried with corrected details.

A provider outage throws and rolls back, leaving no attempt record.

Retry limit, per check type: once the customer has `maxFailedAttempts` FAILED checks of that type inside the rolling `attemptWindow`, further checks are refused before the provider is called, until the oldest of those failures leaves the window. The profile row is locked for the whole check so parallel requests cannot race past the limit.

Network limit, per check type: the client network (`ClientNetwork`) may make at most `ipMaxAttempts` checks of that type of any result, for any customer, within `ipAttemptWindow`. A per-network advisory lock serializes checks from one network across customers.

`kycVerificationRepository.lockClientNetwork(networkKey);`

Lock order is always network, then profile, so two requests can never hold them in opposite orders and deadlock.

`boolean upgrading = profile.isVerified() && !hasPassed(profile, type);`

After approval a customer may still prove the identity number they did not use, which moves them up to tier 2.

`private int tierEarned(KycProfile profile)`

The tier the evidence on file earns: both identity numbers make 2, and with a proof of address in the reviewed package, 3. Otherwise 1. (A proof of address added after approval goes through KycTierService instead, which a reviewer decides.)

`if (profile.isVerified())`

Already approved: a new proof of address needs its own review.

`private List<KycVerification> recentFailedAttempts(`

FAILED checks of one type that count toward the limit: those inside the rolling window and, if an admin reset the limit, after that reset.

Counted failures, oldest first. Takes `since` explicitly so a caller that also marks individual attempts uses the same instant.

`private void enforceNetworkLimit(`

Refuses the check if this network already made `ipMaxAttempts` checks of this type (any result, any customer) within the rolling `ipAttemptWindow`. The caller must hold the network lock so concurrent requests cannot race past the count.

`private Instant countingSince(KycProfile profile)`

FAILED BVN checks after this instant count toward the limit: the start of the rolling window, or the last admin reset if later.

`private Instant retryAfter(Instant oldestCountedFailure)`

When a limited customer may try again: once the oldest counted failure leaves the window.

`public KycReviewQueueResponse listProfiles(`

COMPLIANCE REVIEW

Reviewer's queue, most recently changed first. A null status lists profiles in every state.

`public KycResponse startReview(UUID kycId, UUID reviewerId)`

SUBMITTED -> UNDER_REVIEW: a reviewer picks up the package.

`public KycResponse requestAdditionalInformation(`

UNDER_REVIEW -> ADDITIONAL_INFO_REQUIRED. The customer sees the reason, restarts KYC, adds what is missing and submits again.

`public BvnAttemptsResetResponse resetBvnAttempts(`

Gives the customer a fresh set of BVN attempts, e.g. after support confirms they were genuine typos. Earlier failures stay in the history but stop counting. Locks the profile so an in-flight BVN check cannot overwrite the reset.

`public BvnAttemptsResponse getAttempts(`

Read-only admin view of one check type's retry limit and full history, using exactly the counting rule the verify methods enforce.

`List<KycVerification> counted = recentFailedAttempts(profile, type, since);`

Summary: the same query the verify methods enforce with, independent of which page of history is requested. Oldest first.

`private boolean countsTowardLimit(KycVerification attempt, Instant since)`

Same predicate as the repository query behind the limit: FAILED and strictly after the counting start.

`private KycProfile findReviewableProfile(UUID kycId, UUID reviewerId)`

HELPERS

kyc/service/KycTierService.java

`public class KycTierService`

KYC tiers: what a customer's tier allows, and moving up to tier 3 with a proof of address that a compliance reviewer approves. (Tier 2 needs no reviewer: a passed check of the second identity number earns it, see KycService.)

`public KycTierResponse requestAddressUpgrade(UUID customerId, byte[] content)`

A tier 2 customer sends a proof of address (PDF, PNG or JPEG) and asks for tier 3. One open request at a time.

`public List<KycTierUpgradeResponse> pending()`

Open requests, oldest first, for compliance.

kyc/storage

kyc/storage/KycDocumentStorage.java

`public interface KycDocumentStorage`

Where uploaded KYC document files live. The database keeps only the returned storage key, so swapping this for S3 or Azure Blob does not touch KYC business logic.

`String store(String prefix, String extension, byte[] content);`

Stores the file and returns the key it can later be found under.

@param prefix groups files, e.g. by KYC profile; must be a server-generated value, never client input @param extension file extension without the dot, e.g. "pdf"

`void delete(String storageKey);`

Removes a stored file. Missing files are ignored.

kyc/storage/LocalKycDocumentStorage.java

`public class LocalKycDocumentStorage implements KycDocumentStorage`

Development storage on the local file system. File names are always generated here; the client's original file name is never used, so it cannot influence the path.

ledger/controller

ledger/controller/AdminLedgerController.java

`public class AdminLedgerController`

Staff reads of the ledger: any transaction (with its staff note), any customer account's balance and statement, and PayCore's own system accounts for reconciliation.

Staff cannot move money here. Money enters and leaves through payment providers and bank transfers; corrections go through maker-checker requests (OperationsController).

`public StatementResponse getStatement(`

Any customer account's statement; same shape and defaults as the customer's own.

ledger/controller/LedgerController.java

`public class LedgerController`

Customer money movement and balances. The customer id always comes from the access token: transfers only leave the caller's own accounts, and balances and transactions are only visible to the customers involved.

`public ResponseEntity<TransactionResponse> transfer(`

201 with the transaction. A retry with the same idempotency key returns the same transaction again.

`public StatementResponse getStatement(`

Opening and closing balance, totals and the lines with running balances for whole days (ISO dates, e.g. 2026-09-01) in PayCore's statement time zone. Defaults to the month to date.

ledger/domain

ledger/domain/Counterparty.java

`public record Counterparty(`

The account at another bank on the other side of a bank transfer: the sender of an inbound transfer, the beneficiary of an outbound one.

ledger/domain/Currency.java

`public record Currency(`

An ISO 4217 currency and how many minor units it has (2 for NGN: 100 kobo to the naira). Which currencies accounts may hold is decided when the account is opened (paycore.accounts.supported-currencies); the ledger only needs to know how to count them.

`public static Currency of(String code)`

The minor unit comes from the JDK's ISO 4217 data, so it is never hand-maintained here.

`if (iso.getDefaultFractionDigits() < 0)`

-1 for things like gold (XAU) that have no minor unit.

ledger/domain/Money.java

`public record Money(`

A non-negative amount in whole minor units: ₦20,000.50 is 2,000,050 kobo. No floating point anywhere, and arithmetic fails loudly on overflow or a currency mismatch instead of producing a wrong number.

`public static Money ofMajor(BigDecimal amount, Currency currency)`

Converts an amount as people write it (20000.50) into minor units. Never rounds: 20000.505 NGN is rejected, not silently changed.

ledger/dto/request

ledger/dto/request/FundAccountRequest.java

`public record FundAccountRequest(`

A customer adding money to one of their own accounts (in the path) through the configured payment provider.

Retrying with the same idempotency key and the same details returns the original top-up without charging the customer again.

ledger/dto/request/TransferRequest.java

`public record TransferRequest(`

A transfer from one of the caller's accounts (in the path) to any PayCore account, identified the way a sender knows it: by its 10-digit account number.

The amount is in major units as people write it (20000.50); the ledger converts it to minor units exactly and rejects extra decimal places.

Retrying with the same idempotency key and the same details returns the original transaction instead of moving the money again.

ledger/dto/response

ledger/dto/response/BalanceResponse.java

`public record BalanceResponse(`

Derived from the ledger at the time of the request: credits minus debits across all of the account's entries.

ledger/dto/response/FundingAllowanceResponse.java

`public record FundingAllowanceResponse(`

How much more the customer can add to the account through a payment provider. funded counts provider deposits that still stand (reversed ones do not); staff deposits and incoming transfers never count.

enabled is false, and provider null, when no funding provider is configured; the amounts are then still reported.

ledger/dto/response/LedgerEntryResponse.java

`public record LedgerEntryResponse(`

accountName is the account holder as a receipt names them ("PayCore" for PayCore's own settlement account, which has no account number).

ledger/dto/response/StaffTransactionResponse.java

`public record StaffTransactionResponse(`

A transaction as staff see it: everything the customer sees, flattened into the same JSON object, plus the internal staff note, who initiated it (null for an inbound transfer) and, for a staff correction, the officer who approved it.

A separate type on purpose. `TransactionResponse`, which customer endpoints return, has no field that could carry a staff note, so a mapping mistake cannot leak one.

ledger/dto/response/StatementLineResponse.java

`public record StatementLineResponse(`

One movement on the account: direction is CREDIT for money in and DEBIT for money out, and balanceAfter is the running balance once it was applied. transactionStatus shows REVERSED on a transaction that was later undone; the reversal is its own line.

counterpartyName and counterpartyBank name the other side: the other customer for a transfer inside PayCore ("PayCore"), the other bank's account holder for a transfer in or out, null for top-ups and adjustments.

ledger/dto/response/StatementResponse.java

`public record StatementResponse(`

An account statement for the calendar days from..to (inclusive) in timeZone. openingBalance + totalCredits - totalDebits = closingBalance always holds, whichever page of lines this is.

ledger/dto/response/SystemAccountResponse.java

`public record SystemAccountResponse(`

One of PayCore's own accounts. For SETTLEMENT the balance is the money PayCore holds for customers in that currency: deposits minus withdrawals. It should always equal the sum of customer balances in the same currency, which is what reconciliation checks against the bank.

ledger/dto/response/TransactionResponse.java

`public record TransactionResponse(`

amount is the total debited, which equals the total credited.

counterparty is the account at another bank for a transfer in or out; provider and providerReference name the card processor or bank rail that carried the money and its own reference for it (a NIP session id for bank transfers), which is what customers quote to their bank.

reversedAt is set once the transaction has been reversed; reversalOf is set on a REVERSAL and names the transaction it undid.

ledger/entity

ledger/entity/LedgerEntry.java

`public class LedgerEntry`

One accounting side of a transaction: a positive amount debited from or credited to one account. Entries are historical evidence, so there are no setters and every column is insert-only; a mistake is corrected with a new, compensating transaction.

ledger/entity/LedgerTransaction.java

`public class LedgerTransaction`

One financial event, e.g. a transfer. The money itself is in its `LedgerEntry entries`; this records what happened, who asked for it and where it is in its lifecycle.

Every kind of transaction has its own factory, which sets exactly the fields that kind needs (the database checks the same rules):

```
TRANSFER           customer to customer inside PayCore
DEPOSIT            card top-up a payment provider confirmed
INBOUND_TRANSFER   from another bank, reported by the bank rail
OUTBOUND_TRANSFER  to another bank, sent through the bank rail
ADJUSTMENT         manual correction, requested and approved by two officers
REVERSAL           undoes one of the above
```

There are no setters. The status only moves through `#post` and `#markReversed`, and posting is where the double-entry rules are enforced.

`private UUID initiatedBy;`

Who asked for it: the customer for their own transfers and top-ups, the officer who made the request for staff corrections. Null only for an inbound transfer, which nobody at PayCore initiated.

`private String description;`

Customer-facing narration, shown on transactions and statements.

`private String staffNote;`

Why staff moved money. Internal: never part of a customer response. Required for adjustments and staff reversals.

`private UUID approvedBy;`

For a staff correction, the officer who approved it (never the initiator).

`private String provider;`

For anything a payment provider or bank rail carried: which one, and its own reference (payment reference, NIP session id).

`private String counterpartyName;`

For transfers to or from another bank: the other side.

`private UUID reversesTransactionId;`

For a REVERSAL, the transaction it undoes; null otherwise.

`public static LedgerTransaction createTransfer(`

FACTORIES

A customer's transfer to another PayCore account.

`public static LedgerTransaction createProviderDeposit(`

A card top-up the customer asked for and a payment provider confirmed. The provider and its payment reference record where the money came from.

`public static LedgerTransaction createInboundTransfer(`

Money another bank sent to a PayCore account, as the bank rail reported it. Nobody at PayCore initiated it; the rail's session id is both its provider reference and its idempotency key.

`public static LedgerTransaction createOutboundTransfer(`

A customer's transfer to an account at another bank, sent through the bank rail under the given session id.

`public static LedgerTransaction createBillPayment(`

A bill paid through an aggregator: the biller (and the customer's number with it) is the counterparty; requestId is what the aggregator is asked to deliver under.

`public static LedgerTransaction createAdjustment(`

A manual correction: requested by one operations officer, approved by another, with the reason as the staff note.

`public static LedgerTransaction createReversal(`

A staff REVERSAL of a posted transaction: requested by one officer, approved by another. Its entries must be the original's with the sides swapped; posting it and marking the original reversed are separate steps.

`public static LedgerTransaction createRailReversal(`

The automatic REVERSAL of an outbound transfer the bank rail rejected: the money goes back to the customer who sent it, with no staff involved.

`private void setApproval(UUID requestedBy, UUID approvedBy, String staffNote)`

Four eyes, as the database checks too.

`public void post(List<LedgerEntry> entries)`

LIFECYCLE

INITIATED -> POSTED, but only with a complete, balanced set of entries: at least two, all belonging to this transaction and in its currency, and total debits equal to total credits.

The service always builds balanced entries, so a failure here is a bug, not bad input: it throws IllegalStateException (a 500) and the database transaction rolls back without moving any money.

`public void markReversed()`

POSTED -> REVERSED. Only records the fact; the compensating transaction that undoes the money is posted separately.

`public UUID getId()`

GETTERS

`public Counterparty getCounterparty()`

The other bank's side of a bank transfer; null for anything else.

ledger/enums

ledger/enums/LedgerEntryType.java

`public enum LedgerEntryType`

The side of an entry. Amounts are always positive; the type carries the direction. For a customer account a CREDIT adds to the balance and a DEBIT takes from it.

ledger/enums/LedgerTransactionStatus.java

`public enum LedgerTransactionStatus`

```
INITIATED --post--> POSTED --reverse--> REVERSED
```

INITIATED does not mean money moved. POSTED means the balanced entries are committed. REVERSED means a later, compensating transaction undid the effect; the original entries stay untouched.

ledger/enums/LedgerTransactionType.java

`public enum LedgerTransactionType`

What a ledger transaction is. Customer accounts are always balanced against another customer account (TRANSFER) or against PayCore's settlement account, the real money PayCore holds at its bank:

```
TRANSFER           DEBIT sender customer    CREDIT recipient customer
DEPOSIT            DEBIT settlement         CREDIT customer      card top-up
INBOUND_TRANSFER   DEBIT settlement         CREDIT customer      from another bank
OUTBOUND_TRANSFER  DEBIT customer           CREDIT settlement    to another bank
BILL_PAYMENT       DEBIT customer           CREDIT settlement    airtime, data, power, TV
ADJUSTMENT         either way against settlement, maker-checker approved
REVERSAL           the original's entries with the sides swapped
```

WITHDRAWAL is no longer posted: it remains for transactions recorded while staff could still pay money out by hand.

ledger/exception

ledger/exception/IdempotencyConflictException.java

`public class IdempotencyConflictException extends RuntimeException`

The idempotency key was already used for a different request. A retry must repeat the original request exactly; anything else needs a new key.

ledger/exception/InboundTransferRejectedException.java

`public class InboundTransferRejectedException extends RuntimeException`

PayCore cannot accept an inbound transfer (unknown or inactive account, wrong currency). Nothing was credited; the rail returns the money to the sending bank.

ledger/exception/InvalidLedgerTransactionException.java

`public class InvalidLedgerTransactionException extends RuntimeException`

A transfer the client asked for that can never be valid, such as one from an account to itself.

ledger/exception/LedgerTransactionNotFoundException.java

`public class LedgerTransactionNotFoundException extends RuntimeException`

Also thrown when the transaction exists but touches none of the caller's accounts, so transaction ids and references cannot be probed.

ledger/exception/TransactionNotReversibleException.java

`public class TransactionNotReversibleException extends RuntimeException`

The transaction is already reversed, or is itself a reversal. A reversal is never reversed: if one was a mistake, post a new transaction instead.

ledger/repository

ledger/repository/LedgerEntryRepository.java

`List<LedgerEntry> findByTransactionIdOrderByIdAsc(UUID transactionId);`

In insertion order (ids are UUIDv7), with each entry's account so the response can show account numbers without a query per entry.

`boolean existsByTransactionIdAndAccountCustomerId(UUID transactionId, UUID customerId);`

Whether the transaction touches any of the customer's accounts: the ownership check for reading it.

`SELECT COALESCE(`

CREDITS - DEBITS of everything before the instant: a statement's opening balance.

`SELECT new com.fintechplatform.paycore.ledger.repository.StatementTotals(`

Credits, debits and line count for [from, to).

`SELECT e`

One page of statement lines in [from, to), oldest first, with their transactions. (createdAt, id) is a total order, so pages never overlap or skip entries created in the same instant.

`SELECT COALESCE(`

CREDITS - DEBITS of the period's entries that come before the given entry in statement order: carries the running balance onto a later page without reading the earlier pages.

`SELECT e.account.id`

Ids of the customer (not system) accounts a transaction touched. Ids only, so no account is loaded before the caller locks it.

`SELECT COALESCE(SUM(e.amountMinor), 0L)`

Minor units credited to the account by provider deposits that still stand: what counts towards the account's funding limit. Reversed ones do not count; staff deposits and transfers never do.

Minor units credited to the account by inbound transfers from the provider that still stand.

Minor units the customer sent out of the account since the given moment, by payments of the given types that still stand: a reversed payment gave the money back, so it no longer counts.

`List<LedgerEntry> findByTransactionIdIn(List<UUID> transactionIds);`

Every entry of the given transactions, with their accounts and account holders: names the other side of each statement line in one query.

`SELECT COALESCE(`

CREDITS - DEBITS in minor units: the balance is derived from the ledger, never stored. Answered from idx_ledger_entries_account_created alone (its INCLUDE columns), without reading table rows.

ledger/repository/LedgerTransactionRepository.java

`Optional<LedgerTransaction> findByIdForUpdate(@Param("transactionId") UUID transactionId);`

SELECT ... FOR UPDATE on the original of a reversal, so two reversals of the same transaction run one after the other and the second sees it already REVERSED.

`Optional<LedgerTransaction> findByInitiatedByAndIdempotencyKey(`

Idempotency keys are scoped to the customer who used them.

`Optional<LedgerTransaction> findByProviderAndProviderReference(String provider, String providerReference);`

A provider payment or rail session already recorded; unique, so a payment reported twice is found rather than credited again.

`List<LedgerTransaction> findByTypeAndProviderAndCreatedAtGreaterThanEqualAndCreatedAtLessThan(`

A provider's transactions of one type created in [from, to), for reconciliation.

`Optional<LedgerTransaction> findByReversesTransactionId(UUID reversesTransactionId);`

The reversal of a transaction, if it has one (at most one).

ledger/repository/StatementTotals.java

`public record StatementTotals(`

A statement period's totals in minor units, computed in one query.

ledger/service

ledger/service/BillDebit.java

`public record BillDebit(`

A customer's bill, ready to post: the biller and the customer's number as the counterparty, and the request id the aggregator will be asked to deliver under.

ledger/service/FundingTerms.java

`public record FundingTerms(`

What the ledger needs to know about the funding provider in use: its name (stored on every deposit it confirms), the narration customers see on their statement, and the most provider deposits may add to one account in total, in major units of the account's currency.

ledger/service/InboundTransfer.java

`public record InboundTransfer(`

A transfer another bank sent to a PayCore account, as the bank rail reported it. sessionId is the rail's unique id for the payment.

ledger/service/LedgerService.java

`public class LedgerService`

Moves money through the double-entry ledger.

Every operation is two entries of the same amount, a debit and a credit. Customer money only ever moves between two customer accounts or between a customer account and PayCore's settlement account, the real money PayCore holds at its bank:

```
transfer           DEBIT sender customer   CREDIT recipient customer
card top-up        DEBIT settlement        CREDIT customer
inbound transfer   DEBIT settlement        CREDIT customer
outbound transfer  DEBIT customer          CREDIT settlement
adjustment         either way against settlement (maker-checker)
reversal           the original with the sides swapped
```

Nobody can put money into an account by typing an amount: it arrives because a provider or another bank says it did, and staff corrections need two officers.

PostgreSQL is the consistency boundary: an operation is one database transaction that locks the customer accounts involved, checks balances under that lock, and inserts the transaction and its entries. Either all of it commits or none of it does.

`public TransactionResponse transfer(`

CUSTOMER TRANSFERS INSIDE PAYCORE

Transfers from one of the customer's accounts to any active PayCore customer account in the same currency.

Retrying with the same idempotency key and identical details returns the original transaction and moves no money; the same key with different details is refused.

`UUID destinationAccountId =`

The id only: the account itself is first read under its lock.

`Optional<Posted> replay = replay(posting);`

Fast path for a retry of a transfer that already went through.

`if (replay.isPresent())`

Checked again under the lock: a concurrent duplicate of this request that locked the source first has committed by now, and this one must return its result rather than move money again.

`public Optional<TransactionResponse> checkFunding(`

CARD TOP-UPS THROUGH A PAYMENT PROVIDER

The caller charges the provider between the two steps, outside any database transaction, so no row lock is held while the provider answers:

checkFunding repeat of an earlier top-up? else may this one go ahead? (provider) takes the payment, or declines it postFunding credits the account, checking everything again

The top-up this key already paid for, if this is an exact repeat; otherwise empty once the top-up is allowed: the account is the customer's own, active, in the currency, and within its limit. Nothing is locked; postFunding checks it all again under the lock.

`public TransactionResponse postFunding(`

Credits the payment the provider confirmed. A retry of a top-up that already posted returns it; the same provider payment is never credited twice (the database refuses a second one).

`Account account =`

The lock also serializes top-ups to the account, so two at once cannot both fit under the limit.

```
public FundingAllowanceResponse getFundingAllowance(
```

How much more the customer can add to their own account by card. provider is null when no card provider is configured.

```
private void requireCanFund(Account account, Money amount, FundingTerms terms)
```

Active, in the currency, and with room under the card limit. On the locked account, the funded total cannot change before commit.

```
private static Money remaining(Money limit, Money funded)
```

Zero, not negative, if the limit was lowered below what was funded.

```
public TransactionResponse receiveInboundTransfer(InboundTransfer inbound)
```

TRANSFERS IN FROM OTHER BANKS

Credits a transfer another bank sent, as the bank rail reported it (DEBIT settlement, CREDIT customer). The rail's session id makes it idempotent: a notification delivered twice is credited once and the second delivery gets the same transaction back.

@throws InboundTransferRejectedException when the account cannot take the money (unknown, not active, wrong currency), so the rail returns it to the sending bank

```
if (previous.isPresent())
```

A concurrent delivery of the same session committed while this one waited for the lock.

```
public long inboundTotal(UUID accountId, String provider)
```

Total minor units credited to the account by inbound transfers from the given provider that still stand. For a simulator's own limit.

```
private Posted sameInbound(LedgerTransaction previous, InboundTransfer inbound, Money amount)
```

The same session reported again must carry the same transfer.

```
public TransactionResponse postOutbound(
```

TRANSFERS OUT TO OTHER BANKS

Like any bank: the customer is debited first, then the rail is asked to pay the other bank. If the rail rejects the payment (or never received it), the debit is reversed and the customer has their money back. The payment that tracks it (PaymentService) decides when.

postOutbound DEBIT customer, CREDIT settlement, under the lock (rail) pays the beneficiary bank, or rejects reverseRejectedOutbound only once the rail said no: the money goes back

Debits the customer for a transfer to another bank. The beneficiary is the one name enquiry confirmed; sessionId is what the rail will be asked to pay under.

```
public TransactionResponse postBillPayment(UUID customerId, UUID sourceAccountId, BillDebit bill)
```

Debits the customer for a bill (airtime, data, electricity, TV) before the aggregator is asked to deliver it; reversed like a rejected transfer if it refuses.

```
private TransactionResponse postProviderDebit(
```

DEBIT customer, CREDIT settlement, under the source account lock.

```
public TransactionResponse reverseRejectedOutbound(UUID transactionId, String reason)
```

Gives the customer their money back after the rail rejected an outbound transfer: a REVERSAL with the sides swapped, made by the rail rather than staff, so it works on any account the money left. Reversing twice returns the first reversal.

`public Account requireAdjustable(UUID officerId, UUID accountId, String currencyCode)`

STAFF CORRECTIONS (MAKER-CHECKER)

Staff never move money on their own. An operations officer requests an adjustment or a reversal; a different officer approves it, and only then is it posted, recording both. Neither may hold an account the correction touches.

Whether an adjustment to the account could be requested at all, so a maker cannot queue one that is bound to fail. Checked again in full when it is approved.

`public LedgerTransaction requireReversible(UUID officerId, UUID transactionId)`

Whether a reversal of the transaction could be requested.

`public StaffTransactionResponse postAdjustment(`

Posts an approved adjustment: CREDIT gives the customer money from settlement, DEBIT takes it back. The idempotency key is the request it came from, so approving twice posts once.

`public StaffTransactionResponse reverse(`

Posts an approved reversal: the original's accounts and amount with the sides swapped. The original's entries are never touched; the original moves to REVERSED.

Refused if the transaction is already reversed or is itself a reversal, if it touches either officer's own account, if an account involved is closed or still pending, or if the account being debited no longer holds the money. Frozen accounts may take part: pulling a fraudulent credit back out of a frozen account is what reversals are for.

`LedgerTransaction original =`

Serializes reversals of the same transaction.

`List<Account> locked =`

Customer accounts in id order, as transfers lock them, so reversals and transfers cannot deadlock. Settlement is not locked.

`Account debitAccount = originalCredit.getAccount();`

The swap: whoever was credited is now debited, and vice versa.

`private static void requireNotOwnedBy(Account account, UUID officerId, String action)`

Segregation of duties: nobody corrects their own money.

`private static void requireCorrectable(Account account)`

Active or frozen: closed and pending accounts hold no money to correct.

`public BalanceResponse getBalance(UUID customerId, UUID accountId)`

BALANCES AND READS

404 unless the account is the customer's own.

`public BalanceResponse getCustomerAccountBalance(UUID accountId)`

Staff view of any customer account's balance.

`public List<SystemAccountResponse> getSystemAccounts()`

PayCore's own accounts with their balances, for reconciliation. A settlement account only appears once money has moved in its currency.

`public TransactionResponse getTransaction(UUID customerId, UUID transactionId)`

404 unless the transaction touches one of the customer's accounts, as sender or recipient.

`public StaffTransactionResponse getAnyTransaction(UUID transactionId)`

Staff view of any transaction, including its staff note, e.g. to find one to reverse.

`private record Posting(`

POSTING

What an operation asks for: its type, amount, who asked, their idempotency key, the customer-facing description, the internal staff note, the accounts to debit and credit, for a reversal the transaction it undoes, and the provider that carried it. Compared field by field when a key is reused (a provider's own reference is not: a retry is recognised before the provider is asked again).

create builds the transaction for the reference it is given.

`private record Posted(LedgerTransaction transaction, List<LedgerEntry> entries)`

A stored transaction with its entries, before it becomes a response.

`private Posted record(`

Writes the transaction and its two entries, and for a reversal marks the original REVERSED in the same database transaction. The accounts have been validated and the customer ones locked by the caller.

`if (ConstraintViolations.violates(exception, IDEMPOTENCY_CONSTRAINT))`

The same key raced in through a different lock (another account). The database kept only one.

`if (ConstraintViolations.violates(exception, REVERSAL_CONSTRAINT))`

Unreachable while the original is locked; the database's own guarantee that nothing is reversed twice.

`if (ConstraintViolations.violates(exception, PROVIDER_PAYMENT_CONSTRAINT))`

The same provider payment reported twice; it was credited once.

`private Optional<Posted> replay(Posting posting)`

The original transaction if this key was already used by the same person, provided the request is an exact repeat; otherwise refused.

`private UUID settlementIdOrNull(Currency currency)`

Looked up, never created: a request in a currency no account uses must not leave a settlement account behind.

`private Account settlementAccount(Currency currency)`

The settlement account for the currency, created on first use. The currency has already been checked against a customer account, so only currencies PayCore supports ever get one.

`private AccountPair lockAccounts(UUID customerId, UUID sourceAccountId, UUID destinationAccountId)`

LOCKING AND CHECKS

Locks both accounts, always in the same (id) order whichever way the money flows, so A -> B and B -> A running at once cannot deadlock. The source is only found if it belongs to the customer.

`private static void requireOwnActive(Account account, String action)`

For the customer's own account, whose status they may see.

`private void requireCanReceive(Account account, Currency currency)`

Does not say what state someone else's account is in.

`private void requireFunds(Account account, Money amount)`

Every debit from a customer account needs that account's lock, which the caller holds, so the balance cannot drop before commit.

`private long balanceOf(Account account)`

HELPERS

Balance in minor units on the account's normal side. Customer accounts are what PayCore owes the customer: credits add, debits subtract. The settlement account is money PayCore holds: debits add, credits subtract. Either way a healthy balance is never negative.

`private String generateReference()`

TXN-20260930-8F3A2C91D0B47E15: the UTC date for people, then 64 random bits. At a million transfers a day the chance of any same-day collision is about 1 in 37 million per day; the unique constraint is the backstop if it ever happens.

`private StaffTransactionResponse toStaffResponse(Posted posted)`

Staff only: adds the internal staff note, who initiated the transaction and who approved it.

`private TransactionResponse toResponse(Posted posted)`

What customers see. Built only from customer-facing fields; the staff note has no place in TransactionResponse.

`static String accountName(Account account)`

The account holder's name, as a receipt shows it; "PayCore" for its own accounts.

ledger/service/OutboundTransfer.java

`public record OutboundTransfer(`

A customer's transfer to another bank, ready to post: the beneficiary as name enquiry confirmed it, and the session id the rail will be asked to pay under.

ledger/service/StatementService.java

`public class StatementService`

Account statements, derived from the ledger like every balance.

A statement covers whole calendar days in PayCore's statement time zone (Africa/Lagos by default), so a transfer at 23:30 UTC on 30 September appears on 1 October, as the customer experienced it.

Every figure comes from the database: the opening balance is everything before the period, and each page's running balance starts from the opening balance plus the period's earlier entries, so any page can be read on its own.

Statements are for customer accounts; system accounts are reconciled through their balances instead.

`public StatementResponse getStatement(`

404 unless the account is the customer's own.

`public StatementResponse getCustomerAccountStatement(`

Staff view of any customer account's statement.

`private StatementResponse statement(`

Without dates, the month to date. With only a start date, from then to today.

`Instant start = from.atStartOfDay(timeZone).toInstant();`

[start, end): from's first instant up to the day after to.

`long running =`

Running balance at the start of this page.

`Map<UUID, List<LedgerEntry>> siblings =`

The other side of every line on the page, in one query.

`private static Counterparty counterparty(LedgerEntry entry, List<LedgerEntry> transactionEntries)`

Who is on the other side of the line, as a banking app names them: the other bank's account for a transfer in or out, the other customer for a transfer inside PayCore, nobody for money PayCore itself moved (top-ups, adjustments).

limits/config

limits/config/TierLimitProperties.java

`public class TierLimitProperties`

What each KYC tier may send per day and hold, in major units of the account's currency. Defaults follow the CBN's three-tier KYC for naira accounts. A null maxBalance means no cap.

`public TierLimit forTier(int tier)`

Tier 0 (not verified) may neither send nor hold anything.

`private BigDecimal dailySend;`

Total a customer may send in a day (statement time zone).

`private BigDecimal maxBalance;`

Most the account may hold; null for no cap.

limits/exception

limits/exception/BalanceLimitExceededException.java

`public class BalanceLimitExceededException extends RuntimeException`

A credit would take an account past what its holder's KYC tier may hold. The message never reveals another customer's balance.

limits/exception/SpendingLimitExceededException.java

`public class SpendingLimitExceededException extends RuntimeException`

A payment would take the customer past one of their spending limits. remaining is what can still be sent before resetsAt.

`public String getLimit()`

Which limit, e.g. NEW_DEVICE.

limits/service

limits/service/SpendingLimitService.java

`public class SpendingLimitService`

How much a customer may send and hold. The ledger calls this while it holds the account's lock, so two payments at once cannot both fit under a limit that only one of them fits under.

```
DAILY        what the customer's KYC tier may send per day
NEW_DEVICE   for a while after a new device is confirmed, a small total
BALANCE      what the KYC tier may hold, checked on money coming in
```

Reversals and staff adjustments are never limited: money going back where it came from must always be possible.

`public static final Set<LedgerTransactionType> CUSTOMER_PAYMENTS =`

Payments the customer makes; all count against their limits.

`public void requireWithinLimits(Account source, Money amount)`

Before money leaves a customer account.

@param source the account paying, locked by the caller @throws SpendingLimitExceededException when amount does not fit

`public void requireCanHold(Account destination, long balanceMinor, Money amount, boolean ownAccount)`

Before money comes into a customer account.

@param destination the account credited, locked by the caller @param balanceMinor its balance now, in minor units @param ownAccount whether the person paying holds it (a top-up), so the message can speak to them directly @throws BalanceLimitExceededException when the credit would take the account past its tier's cap

`public int tierOf(UUID customerId)`

The customer's KYC tier, 0 if not verified.

`public long sentToday(UUID accountId)`

What the customer sent today from the account, in minor units.

`public Instant endOfToday()`

When "today" (statement time zone) ends.

notification

notification/EmailSender.java

`public interface EmailSender`

Delivers an email. SMTP when spring.mail.host is configured (Mailpit locally, a real provider in production); otherwise the email is only logged, so development works without a mail server.

notification/EmailSentEvent.java

`public record EmailSentEvent(OutgoingEmail email)`

Published after an email has been handed to the EmailSender, so other modules (the demo outbox, tests) can see what was sent.

notification/LoggingEmailSender.java

`public class LoggingEmailSender implements EmailSender`

Used when no mail server is configured: writes the email to the log so a developer can still follow a verification link. Never used in production, where spring.mail.host is set.

notification/NotificationConfiguration.java

`public class NotificationConfiguration`

SMTP when Spring Boot has configured a JavaMailSender (it does whenever spring.mail.host is set), otherwise the logging sender.

notification/NotificationProperties.java

`private String from = "PayCore <no-reply@paycore.local>";`

The From address of every email PayCore sends.

notification/NotificationService.java

`public class NotificationService`

Sends email for the rest of the application. A delivery failure is logged, not thrown: the action that triggered the email (a registration, say) has already happened, and the customer can ask for another one.

`public boolean send(OutgoingEmail email)`

@return whether the email was handed over for delivery

notification/OutgoingEmail.java

`public record OutgoingEmail(`

A plain-text email. Plain text on purpose: verification links are the only mail PayCore sends, and they need no markup.

notification/SmtpEmailSender.java

`public class SmtpEmailSender implements EmailSender`

Sends through the JavaMailSender that Spring Boot configures from the spring.mail.* properties.

operations/controller

operations/controller/OperationsController.java

`public class OperationsController`

The operations desk: makers file corrections, checkers decide them. Filing needs LEDGER_REQUEST, deciding LEDGER_APPROVE, and the same officer can never decide their own request (403 FOUR_EYES_REQUIRED).

`public ResponseEntity<OperationsRequestResponse> requestPaymentResolution(`

Settle a payment in NEEDS_REVIEW (see /admin/payments): COMPLETE or REVERSE a transfer; CREDIT, REFUND or CLOSE a top-up. 409 if the provider now says otherwise, or one is already waiting.

`public List<OperationsRequestResponse> list(@RequestParam(defaultValue = "PENDING") String status)`

status=PENDING (default) for the queue, DECIDED for recent decisions.

operations/dto

operations/dto/AdjustmentRequestBody.java

`public record AdjustmentRequestBody(`

A request to correct a customer account by hand: CREDIT gives the customer money, DEBIT takes it, against PayCore's settlement account. reason is for staff only; customerDescription is what the customer reads (default "Account adjustment").

operations/dto/DecisionBody.java

`public record DecisionBody(`

The checker's note: optional when approving, required when rejecting.

operations/dto/OperationsRequestResponse.java

`public record OperationsRequestResponse(`

A maker-checker request as the operations queue shows it. For an ADJUSTMENT, account and direction say what changes; for a REVERSAL, transaction names what is undone. amount and currency are the money either way. For a PAYMENT_RESOLUTION, payment and resolution say how a payment in review is settled (account and amount are the payment's). resultReference is the ledger transaction an approval posted, if any.

operations/dto/PaymentResolutionRequestBody.java

`public record PaymentResolutionRequestBody(`

A request to settle a payment in NEEDS_REVIEW. reason is for staff only and should name the evidence (e.g. "Rail support ticket 4411: session paid at 14:02"); customerDescription is what the customer reads when money moves back (a reversal), default "Reversal of TXN-...".

operations/dto/ReversalRequestBody.java

`public record ReversalRequestBody(`

A request to undo a posted transaction. reason is for staff only; customerDescription is what the customers involved read (default "Reversal of TXN-...").

operations/entity

operations/entity/OperationsRequest.java

`public class OperationsRequest`

A correction one operations officer (the maker) asks for and a different one (the checker) approves or rejects. Nothing moves until it is approved; the approval posts the ledger transaction and records it here.

```
PENDING --approve--> APPROVED (result transaction set; for a payment
                    resolution only when money moved)
        --reject---> REJECTED
```

`public static OperationsRequest adjustment(`

Credit (or debit) the account by the amount, against settlement.

`public static OperationsRequest reversal(`

Undo the transaction.

`public static OperationsRequest paymentResolution(`

Settle a payment in NEEDS_REVIEW the given way.

`public void requireDecidableBy(UUID officerId)`

Checks that this officer may decide it: still pending, and not the officer who asked for it.

`this.resultTransactionId =`

A payment resolution posts to the ledger only when money moves.

`public String ledgerKey()`

The ledger idempotency key of the transaction an approval posts: one request can only ever post one transaction.

operations/exception

operations/exception/DuplicatePaymentResolutionException.java

`public class DuplicatePaymentResolutionException extends RuntimeException`

A resolution of this payment is already waiting for approval.

operations/exception/FourEyesViolationException.java

`public class FourEyesViolationException extends RuntimeException`

The officer who made a request tried to decide it.

operations/exception/ResolutionContradictsProviderException.java

`public class ResolutionContradictsProviderException extends RuntimeException`

The provider gave a definite answer about the payment that the requested resolution contradicts (e.g. COMPLETE a transfer the rail says it rejected). The provider's answer wins: run a status check instead, which applies it. The request stays pending, for the checker to reject.

operations/repository

operations/repository/OperationsRequestRepository.java

`List<OperationsRequest> findByStatusOrderByRequestedAtAsc(OperationsRequestStatus status, Pageable pageable);`

The queue: oldest first, so nothing waits forever.

`List<OperationsRequest> findByStatusNotOrderByDecidedAtDesc(OperationsRequestStatus status, Pageable pageable);`

Recently decided ones, newest first.

`Optional<OperationsRequest> findByIdForUpdate(@Param("id") UUID id);`

SELECT ... FOR UPDATE: two checkers approving the same request at once are served one after the other, and the second finds it decided.

operations/service

operations/service/OperationsService.java

`public class OperationsService`

Maker-checker for every money correction staff can make. An operations officer (maker) files a request with a reason; nothing moves. A different officer (checker) approves it, which posts the ledger transaction recording both of them, or rejects it with a note.

The rules a bank would apply, enforced here and in the database:

• the checker is never the maker (four eyes)

• neither may hold an account the correction touches

• a request is decided once; approving twice posts once

• at most one reversal of a transaction waits at a time

`public OperationsRequestResponse requestAdjustment(UUID makerId, AdjustmentRequestBody body)`

MAKER

`ledgerService.requireAdjustable(makerId, body.accountId(), amount.currency().code());`

Fails now rather than at approval if it could never be posted.

`public OperationsRequestResponse requestPaymentResolution(UUID makerId, PaymentResolutionRequestBody body)`

Settle a payment in NEEDS_REVIEW (see PaymentResolutionService).

`public OperationsRequestResponse approve(UUID checkerId, UUID requestId, String note)`

CHECKER

Posts the correction and marks the request approved, in one database transaction: if the ledger refuses (the money is gone, the account closed), nothing is posted and the request stays pending.

`public List<OperationsRequestResponse> list(boolean pending)`

READS

Pending requests oldest first, or decided ones newest first.

operations/service/PaymentResolutionService.java

`public class PaymentResolutionService`

Settling a payment in NEEDS_REVIEW, under maker-checker: one officer requests a resolution with the evidence, a different one approves it, and only the approval changes the payment (and moves money, if any).

Staff decide only what the provider cannot tell PayCore. Whenever the provider gives a definite answer that contradicts the resolution (it says the transfer was rejected, staff say COMPLETE), the request is refused, at filing and again at approval: the provider's answer wins, and a status check applies it.

Approval is one database transaction holding the request's and the payment's row locks while the provider is asked once more (and, for a refund, asked to refund). Holding locks during a provider call is avoided everywhere else; here it is deliberate: the action is rare and human-triggered, and nothing else (a status check, a second approval) may settle the payment in between.

`public OperationsRequest request(UUID makerId, PaymentResolutionRequestBody body)`

MAKER

`public Optional<UUID> apply(OperationsRequest request, UUID checkerId)`

CHECKER

Applies the resolution and approves the request, together; any refusal leaves both unchanged and the request pending. Runs inside the caller's transaction (OperationsService.approve), which holds the request's lock. Returns the ledger transaction it posted, if any.

`Payment payment = paymentService.lockForResolution(request.getPaymentId());`

Locked until commit: nothing else settles it meanwhile.

`private static void requireResolvable(Payment payment, PaymentResolution resolution)`

RULES

`private static void requireNotOwnPayment(Payment payment, UUID officerId)`

Segregation of duties: nobody settles a payment of their own.

`private Optional<String> requireNoContradiction(Payment payment, PaymentResolution resolution)`

Asks the provider now and refuses a resolution its definite answer contradicts. No answer (unreachable, still pending) is what staff resolve on their own evidence. For CREDIT, returns the reference to credit under: the provider's own if it gave one, else PayCore's merchant reference.

payment/config

payment/config/PaymentProperties.java

`public class PaymentProperties`

How PayCore settles payments whose outcome it does not know.

```
stale-after       a payment still CREATED or PROCESSING this long after
                  its last change was abandoned mid-request (crash,
                  restart). Must exceed the longest provider call.
first-check-after the first status check after a provider gave no answer
max-backoff       checks back off exponentially up to this interval
max-checks        then NEEDS_REVIEW: operations take over
not-found-grace   how old a payment must be before a provider saying
                  "no such payment" is believed (it may not have
                  recorded it yet)
batch-size        payments handled per sweep
```

The sweep runs every `paycore.payments.recovery.interval` while `paycore.payments.recovery.enabled` is true.

`public Duration backoffAfter(int checksMade)`

When the check after this one is due: first-check-after, doubled for every check already made, capped at max-backoff.

payment/controller

payment/controller/AdminPaymentController.java

`public class AdminPaymentController`

Staff view of payments: the queue of ones in trouble (UNKNOWN, NEEDS_REVIEW), a payment's full history, and asking the provider again.

Nobody on staff decides a payment's outcome: a status check asks the provider, and only its answer moves the payment (and any money).

`public PaymentPageResponse list(`

Newest first; ?status=NEEDS_REVIEW for the ones operations must resolve.

`public List<PaymentEventResponse> events(@PathVariable UUID paymentId)`

Every status the payment went through, oldest first.

`public PaymentResponse escalate(@PathVariable UUID paymentId)`

Takes an UNKNOWN payment out of automation into NEEDS_REVIEW, e.g. on a customer complaint, so it can be resolved under maker-checker.

`public PaymentResponse checkStatus(@PathVariable UUID paymentId)`

Asks the provider now what happened, for an UNKNOWN or NEEDS_REVIEW payment, and applies its answer. 409 while the payment is still in its original request.

payment/controller/PaymentController.java

`public class PaymentController`

Customers following their own payments, e.g. a top-up or transfer whose outcome was not known when they made it. 404 for anyone else's.

payment/dto

payment/dto/PaymentEventResponse.java

`public record PaymentEventResponse(`

One step in a payment's history; fromStatus is null for its creation.

payment/dto/PaymentResponse.java

`public record PaymentResponse(`

A payment and where it stands. ledgerTransactionId is the posting it made (the debit of a transfer, the credit of a top-up); reversalTransactionId the one that returned a failed transfer's money; refundReference the provider's refund of a top-up given back to the card. failureReason is written for customers. bill is set for bill payments: the biller, the product, and what the biller gave back (a meter token).

payment/entity

payment/entity/Payment.java

`public class Payment`

The lifecycle of one payment through an external provider: a transfer to another bank or a card top-up. The ledger records what money moved; this records what PayCore asked the provider for, what it answered, and whether the answer is still outstanding.

Every change goes through a method here that checks the state machine in `PaymentStatus`; the database checks it again.

`private String billToken;`

What the biller gave back, e.g. a prepaid meter token.

`public static Payment outboundTransfer(`

A transfer to another bank, recorded in the same database transaction as the customer's debit (ledgerTransactionId), before the rail is asked to pay. sessionId is what the rail pays it under.

`public static Payment billPayment(`

A bill, recorded in the same database transaction as the customer's debit, before the aggregator is asked to deliver it. requestId is what the aggregator knows it by; biller names the biller (name), its category (bank) and the customer's number (accountNumber).

`public static Payment cardFunding(`

A card top-up, recorded before the provider is asked to charge. Nothing has been credited yet, so it starts PROCESSING: the next thing to happen is the provider call. merchantReference is what the provider knows it by.

`public void startProcessing()`

TRANSITIONS

About to call the provider.

`public void succeed(UUID ledgerTransactionId, String providerReference)`

The provider confirmed it. ledgerTransactionId is the posting it made (a top-up's credit; a transfer already has its debit).

`public void fail(String reason, UUID reversalTransactionId, String providerReference)`

The provider refused it, or it never reached the provider. A transfer must say which reversal returned the money.

`public void refund(String refundReference, String reason)`

A top-up whose charge was given back to the card: FAILED, nothing credited, with the provider's refund reference.

`public void recordToken(String token)`

The biller's token, kept once it arrives.

`public void markUnknown(Instant nextCheckAt)`

No answer from the provider. Not failed: the first status check is due at nextCheckAt.

`public void recordStatusCheck(Instant nextCheckAt)`

Claims the status check that is due: counts it and pushes the next one to nextCheckAt, so another instance does not ask at the same time and, if this check gets no answer, the next one is scheduled.

`public void requireReview(String reason)`

Automation stops here; operations take over. reason is what the customer and staff read; the detail is in the logs.

`public Money amount()`

QUERIES

`public boolean isDebitedFirst()`

Money left the customer before the provider was called (a transfer out, a bill), so a failure must give it back.

`public UUID getId()`

GETTERS

payment/entity/PaymentEvent.java

`public class PaymentEvent`

One status change of a payment. Written by the database itself (trg_payments_audit), so no code path can move a payment without leaving a record; the application only reads them.

payment/enums

payment/enums/PaymentResolution.java

`public enum PaymentResolution`

How operations settle a payment automation could not (NEEDS_REVIEW), decided by two officers (maker-checker) on evidence from outside PayCore.

```
transfer out  COMPLETE  the beneficiary was paid: SUCCEEDED, the debit stands
(and bill)
              REVERSE   they were not: the debit is reversed, FAILED
card top-up   CREDIT    the card was charged: credit the customer, SUCCEEDED
              REFUND    the card was charged: refund it instead, FAILED
              CLOSE     the card was never charged: FAILED, nothing moves
```

`public boolean succeeds()`

Whether the payment ends SUCCEEDED (otherwise FAILED).

payment/enums/PaymentStatus.java

`public enum PaymentStatus`

Where a payment is in its life. The same rules are enforced by the database (V26, trg_payments_guard).

```
CREATED ------> PROCESSING ----> SUCCEEDED
   |               |   |
   |               |   +-------> FAILED
   |               v
   |            UNKNOWN -------> SUCCEEDED / FAILED
   |               |
   |               v
   +--> FAILED  NEEDS_REVIEW --> SUCCEEDED / FAILED
```

`CREATED,`

Recorded (a transfer is already debited); not yet sent to the provider.

`PROCESSING,`

Sent to the provider; its answer is not recorded yet.

`UNKNOWN,`

The provider did not answer, or PayCore stopped before recording the answer. Never treated as failed: the provider may have acted. A status check with the provider settles it.

`NEEDS_REVIEW,`

Automatic checks gave up, or the money could not be credited. Operations must look.

`FAILED;`

Final. A failed transfer's money has been returned to the customer.

`public boolean isOpen()`

Not final, so the provider may still be asked what happened.

payment/enums/PaymentType.java

`public enum PaymentType`

Payments that cross PayCore's boundary through a provider. Transfers between PayCore accounts never leave PayCore and are ledger-only.

`OUTBOUND_TRANSFER,`

To an account at another bank, through the bank rail.

`CARD_FUNDING,`

A customer's card top-up, through the funding provider.

`BILL_PAYMENT`

Airtime, data, electricity or TV, through a bills aggregator.

payment/exception

payment/exception/InvalidPaymentTransitionException.java

`public class InvalidPaymentTransitionException extends RuntimeException`

A payment was asked to move somewhere its state machine does not allow, e.g. a SUCCEEDED payment to FAILED. Always a bug or a lost race, never a customer error.

payment/exception/PaymentNotFoundException.java

`public class PaymentNotFoundException extends RuntimeException`

Also thrown when the payment exists but belongs to someone else, so a customer cannot learn which payments exist.

payment/repository

payment/repository/PaymentRepository.java

`List<Payment> findByTypeAndProviderAndCreatedAtGreaterThanEqualAndCreatedAtLessThan(`

A provider's payments of one type created in [from, to), for reconciliation.

`Optional<Payment> findByIdForUpdate(@Param("paymentId") UUID paymentId);`

SELECT ... FOR UPDATE: whoever changes a payment holds its row, so the request thread and a status check never apply two answers.

`SELECT p.id FROM Payment p`

Payments left in the given states since before the cut-off: a request that stopped (crash, restart) between two steps.

Unknown outcomes whose next status check is due.

payment/service

payment/service/PaymentRecoveryScheduler.java

`public class PaymentRecoveryScheduler`

Runs the payment recovery sweep every `paycore.payments.recovery.interval` (default 30 seconds). On by default; tests switch it off and run sweeps themselves.

payment/service/PaymentRecoveryService.java

`public class PaymentRecoveryService`

Settles payments whose outcome PayCore does not know. Never assumes: the provider is asked, and its answer decides.

```
1. CREATED for too long      the request stopped before the provider was
                             called, so nothing was sent: a transfer's
                             money goes back
2. PROCESSING for too long   the request stopped mid-call: the provider
                             may have acted, so the outcome is UNKNOWN
3. UNKNOWN, check due        ask the provider (status query), with
                             exponential backoff between checks; after
                             max-checks without an answer, NEEDS_REVIEW
```

Safe to run on several instances at once: every step re-checks the payment under its row lock, and claiming a check pushes the next one into the future, so two instances never act on the same answer.

`public record Sweep(int abandoned, int markedUnknown, int checked)`

What one sweep did.

`public Sweep recoverDue()`

One pass over everything that is due.

`public PaymentResponse checkNow(UUID paymentId)`

An operations officer asking the provider again, now: for an UNKNOWN payment before its next check is due, or one in NEEDS_REVIEW.

`private Optional<Payment> abandonUnsent(UUID paymentId, Instant staleBefore)`

Only debit-first payments start CREATED; each goes back its own way.

`private static int forEach(List<UUID> paymentIds, Function<UUID, Optional<?>> action)`

One failure must not stop the sweep; it is retried on the next one.

payment/service/PaymentService.java

`public class PaymentService`

Every change to a payment, each one its own short database transaction that locks the payment row. The orchestrators (BankTransferService, FundingService, PaymentRecoveryService) call the provider between these steps, never inside one, so no lock is held while a provider answers.

Where a payment change and a ledger posting must happen together they share one database transaction: a transfer's payment is created with its debit, a failed transfer is marked FAILED with its reversal, a top-up is marked SUCCEEDED with its credit. A crash can therefore leave a payment unfinished, but never money moved without its payment, or a payment finished without its money.

Transitions are idempotent where a race makes them legitimately repeatable (the request thread and a status check both learning that a payment succeeded): moving to the state it is already in does nothing.

`public Payment initiateOutbound(`

TRANSFERS TO OTHER BANKS

Debits the customer and records the payment, together. If a concurrent duplicate of this request (same key) was debited first, returns its payment instead: that is the one being paid.

`public Payment initiateBill(`

Debits the customer for a bill and records its payment, together. As for a transfer, a concurrent duplicate that was debited first wins.

`public Payment failOutbound(UUID paymentId, String reason)`

The rail rejected the transfer, or never received it: the debit is reversed and the payment FAILED, together. Never on a payment that succeeded.

`public Optional<Payment> failOutboundIfUnsent(UUID paymentId, Instant staleBefore, String reason)`

A transfer debited but still CREATED since before the cut-off: the request stopped before the rail was called (the call only happens after PROCESSING is committed), so nobody was paid and the money goes back. Checked under the payment's lock, so a request thread that is just now sending it cannot be overtaken. Empty if it moved on.

`public Payment startFunding(`

CARD TOP-UPS

Records a top-up before the provider is asked to charge. A second request with the same key fails here, on the unique key, rather than charging the customer twice.

`public Payment completeFunding(UUID paymentId, FundingTerms terms, String providerReference)`

The provider confirmed the charge: credits the customer and marks the payment SUCCEEDED, together. The ledger's own idempotency (the customer's key, and the provider's payment reference) makes a repeat credit nothing.

`public Payment failFunding(UUID paymentId, String reason, String providerReference)`

The provider declined the charge, or never received it. Nothing was credited.

`public Payment markProcessing(UUID paymentId)`

ANY PAYMENT

About to call the provider.

`public Payment succeed(UUID paymentId)`

The provider confirmed it (a transfer: its debit already stands).

`public Payment succeed(UUID paymentId, String billToken)`

A bill the biller delivered, with what it gave back (a meter token).

`public Payment markUnknown(UUID paymentId)`

The provider did not answer; the first status check is scheduled.

`public Optional<Payment> markUnknownIfStale(UUID paymentId, Instant staleBefore)`

A payment the request thread left PROCESSING, presumably because PayCore stopped mid-call: its outcome is unknown, check now. Does nothing if it moved on since it was found.

`public Optional<Payment> claimStatusCheck(UUID paymentId, boolean manual)`

Claims the status check that is due, so no other instance makes the same check: counts it and schedules the next one (backing off), in case this one gets no answer.

Empty if there is nothing to check: the payment is final, or (for an automatic check) another instance already claimed it. Operations may also re-check a payment in review.

@throws InvalidPaymentTransitionException when a manual check is asked for a payment still in its request (CREATED, PROCESSING)

`public Payment stillUnknown(UUID paymentId, String reviewReason)`

A status check got no definite answer. Once the checks are used up the payment goes to review; until then the next one is already scheduled.

`public Payment lockForResolution(UUID paymentId)`

Locks a payment operations are resolving, until the caller's transaction ends.

`public Payment failReversed(UUID paymentId, String reason, UUID reversalTransactionId)`

A transfer operations found was never paid: FAILED, with the staff reversal (maker-checker) that returned the money.

`public Payment refunded(UUID paymentId, String refundReference, String reason)`

The charge was given back to the card (by operations, or found refunded by a status check): FAILED, nothing credited.

`public Payment escalate(UUID paymentId)`

An operations officer takes an UNKNOWN payment out of automation, e.g. after a customer complaint, so it can be resolved under maker-checker instead of waiting for the remaining status checks.

`public Payment requireReview(UUID paymentId, String reason)`

Automation stops; operations must look. reason is customer-safe.

`public Optional<Payment> findByKey(UUID customerId, String idempotencyKey)`

READS

`public PaymentResponse getOwn(UUID customerId, UUID paymentId)`

404 unless it is the customer's own.

`public List<PaymentEventResponse> events(UUID paymentId)`

The payment's history, oldest first.

`private Payment lock(UUID paymentId)`

HELPERS

`private String generateReference()`

PAY-20261005-8F3A2C91D0B47E15: the UTC date, then 64 random bits, like ledger references; the unique constraint is the backstop.

pin/config

pin/config/PinProperties.java

`private int maxAttempts = 3;`

Wrong PINs in a row before the PIN locks and must be reset.

pin/controller

pin/controller/TransactionPinController.java

`public class TransactionPinController`

The caller's transaction PIN. Setting, changing and resetting it all need a trusted device.

`public CodeSentResponse sendResetCode(@AuthenticationPrincipal CurrentUser currentUser)`

Emails a code for resetting the PIN. At most one a minute.

pin/dto

pin/dto/PinRequests.java

`public final class PinRequests`

Request bodies for the transaction PIN endpoints. A PIN is exactly four digits.

pin/dto/PinStatusResponse.java

`public record PinStatusResponse(boolean set, boolean locked, int attemptsRemaining)`

Whether the customer has a PIN, and whether it still works.

pin/entity

pin/entity/TransactionPin.java

`public class TransactionPin`

A customer's 4-digit transaction PIN, BCrypt-hashed. Every guess first reserves an attempt (failedAttempts goes up before the hash is compared), so guesses sent in parallel cannot get past the limit; a right guess resets the count.

`public void reserveAttempt()`

Counted as failed until the guess turns out right.

`public void attemptFailed(int maxAttempts)`

The reserved attempt stays counted; the last one locks the PIN.

`public void replace(String pinHash)`

A new PIN, after a change or a reset: unlocked, no failed attempts.

pin/service

pin/service/TransactionAuthorizationService.java

`public class TransactionAuthorizationService`

What a customer must show before money leaves their account: something they have (a trusted device) and something they know (the transaction PIN). Every endpoint that pays money out calls this first.

The device comes first, so a new, unconfirmed device cannot even use up the PIN's attempts.

`public static final String PIN_HEADER = "X-Transaction-PIN";`

Where clients put the PIN, keeping it out of the payment itself.

`public void authorize(CurrentUser user, String rawPin)`

@throws DeviceNotTrustedException when the request's device is not trusted @throws RuntimeException the PIN's exceptions, see `TransactionPinService#verify`

pin/service/TransactionPinService.java

`public class TransactionPinService`

The transaction PIN: set once from a trusted device, entered for every payment, changed with the current PIN, reset with a code sent to the customer's email.

BCrypt runs outside any database transaction, as at login: it is slow on purpose, and inside a transaction it would hold a pooled connection (and here a row lock) for the whole comparison.

`public PinStatusResponse set(UUID customerId, UUID sessionId, String rawPin)`

The first PIN. Only from a trusted device.

`if (pinRepository.existsById(customerId))`

Checked again: a concurrent set may have won. The primary key stops a second row either way.

`public PinStatusResponse change(UUID customerId, UUID sessionId, String currentPin, String newPin)`

A new PIN, proven with the current one (a wrong one counts).

`public CodeSentResponse sendResetCode(UUID customerId, UUID sessionId)`

Emails a code that resets the PIN. Only from a trusted device.

`public PinStatusResponse reset(UUID customerId, UUID sessionId, String code, String newPin)`

A new PIN, proven with the emailed code; also unlocks a locked PIN. The code is consumed in its own transaction, so a wrong guess stays counted.

`public void verify(UUID customerId, String rawPin)`

Checks a PIN entered for a payment.

@throws TransactionPinRequiredException when none was entered @throws TransactionPinNotSetException when the customer has none @throws TransactionPinLockedException after too many wrong ones @throws InvalidTransactionPinException when it is wrong

`String hash =`

1. Reserve an attempt under the row lock and commit it, so this guess is counted even if the request dies during the comparison.

`boolean matches = passwordEncoder.matches(rawPin.trim(), hash);`

2. Compare, holding nothing.

`int remaining =`

3. Settle the reserved attempt.

`static boolean isWeak(String pin)`

Not one digit repeated (0000) and not a run up or down (1234, 9876): the PINs tried first.

reconciliation/controller

reconciliation/controller/ReconciliationController.java

`public class ReconciliationController`

Reconciling PayCore's books with each provider's settlement report. Operations officers run it and work the breaks (LEDGER_REQUEST); a supervisor closes them (LEDGER_APPROVE); staff who can see all accounts can read the results.

`public ResponseEntity<ReconciliationRunResponse> run(`

201 with the run's summary; GET the run for its items.

`public List<ReconciliationRunResponse> runs()`

The most recent runs, newest first.

`public ReconciliationItemResponse redeliver(`

For a MISSING_LOCAL inbound transfer: the rail delivers it again and the customer is credited.

`public ReconciliationItemResponse resolve(`

A supervisor closes a break once it is fixed or explained.

reconciliation/dto

reconciliation/dto/ReconciliationItemResponse.java

`public record ReconciliationItemResponse(`

One reconciliation item: each side's amount and status (null when that side has no record), the result, what is wrong (detail), and for a break its resolution. paymentId, ledgerTransactionId and accountId point at PayCore's records, for fixing it.

reconciliation/dto/ReconciliationRunDetailResponse.java

`public record ReconciliationRunDetailResponse(`

A run and its items, breaks first.

reconciliation/dto/ReconciliationRunResponse.java

`public record ReconciliationRunResponse(`

A run's summary. providerTotal is the money the provider moved that day, paycoreTotal what PayCore's records say moved; they differ by the breaks. scheduled is true for the automatic daily run.

reconciliation/dto/ResolveBreakRequest.java

`public record ResolveBreakRequest(`

How a break was fixed or explained, e.g. "Reversal TXN-... approved".

reconciliation/dto/RunReconciliationRequest.java

`public record RunReconciliationRequest(`

Reconcile one provider for one business day (default: today, Lagos).

reconciliation/entity

reconciliation/entity/ReconciliationItem.java

`public class ReconciliationItem`

One line of a reconciliation run: a reference both sides share (or that only one side has), each side's amount and outcome, and the result. A break stays OPEN until it is fixed and resolved with a note; nothing else about an item ever changes (the database enforces it, V30).

`public record Side(Long amountMinor, String status)`

One side of the comparison: an amount and an outcome, or nothing.

`public record Links(UUID paymentId, UUID ledgerTransactionId, UUID accountId)`

PayCore's records the item points at; any may be null.

`public void resolve(UUID resolvedBy, String note)`

Closes the break: fixed, or explained, by resolvedBy.

reconciliation/entity/ReconciliationRun.java

`public class ReconciliationRun`

One comparison of one provider's settlement report for one business day with PayCore's books. Made once and never changed: running the same day again makes a new run. Its items say what matched and what broke.

reconciliation/enums

reconciliation/enums/ItemKind.java

`public enum ItemKind`

What one reconciliation item is about.

`OUTBOUND,`

A transfer from a PayCore account to another bank.

`INBOUND,`

A transfer from another bank to a PayCore account.

`CARD`

A card top-up.

reconciliation/enums/ProviderKind.java

`public enum ProviderKind`

Which provider a reconciliation run compares PayCore with.

`BANK_RAIL,`

The interbank rail: transfers out and in.

`CARD`

The card processor: top-ups.

reconciliation/enums/ReconciliationResult.java

`public enum ReconciliationResult`

How one provider line and PayCore's record of it compare.

```
MATCHED           both agree on the reference, the amount and the outcome
MISSING_LOCAL     the provider moved money PayCore has no record of
MISSING_PROVIDER  PayCore recorded money the provider never moved
AMOUNT_MISMATCH   both have it, for different amounts
STATUS_MISMATCH   both have it, with different outcomes
```

Anything but MATCHED is a break.

reconciliation/enums/ResolutionStatus.java

`public enum ResolutionStatus`

NOT_REQUIRED for a match; a break is OPEN until a supervisor RESOLVES it.

reconciliation/repository

reconciliation/repository/ReconciliationItemRepository.java

`SELECT i FROM ReconciliationItem i`

Breaks first (by result), then by reference.

`Optional<ReconciliationItem>`
`findFirstByKindAndExternalReferenceAndResultAndResolutionStatusOrderByResolvedAtDesc(`

The most recent resolution of the same break in an earlier run, for carry-over.

reconciliation/service

reconciliation/service/ReconciliationScheduler.java

`public class ReconciliationScheduler`

Reconciles yesterday every morning with every configured provider (paycore.reconciliation.schedule, default 06:30 Lagos time), once the providers have published their settlement reports. Officers can run any day again from the back office. Tests switch it off.

reconciliation/service/ReconciliationService.java

`public class ReconciliationService`

Compares a provider's settlement report for one business day with PayCore's books, in both directions:

```
every line of the report      found in PayCore, same amount, same outcome?
every PayCore record that day found in the report?
```

Bank rail: outbound sessions against transfer payments, inbound sessions against inbound ledger credits. Card processor: charges against top-up payments. The two sides are joined on the reference they share (NIP session id, merchant reference), never on amounts or times.

Records near midnight can land on different days on each side, so the side that is looked up is searched a day either way of the window; only the side being reconciled is limited to the business day.

Reconciliation reads and reports; it never moves money itself. A break is fixed through the usual controls (the rail re-delivering a lost notification, a status check, maker-checker reversal or adjustment) and then resolved with a note by a supervisor.

`static final Duration LOOKUP_MARGIN = Duration.ofDays(1);`

How far either side of the business day a counterpart is looked for.

`public LocalDate today()`

RUNNING

The business day in PayCore's time zone (Lagos), e.g. today.

`public ReconciliationRun run(ProviderKind kind, LocalDate businessDate, UUID startedBy)`

Reconciles one provider for one business day. startedBy is the officer, or null for the scheduled run. Breaks already resolved in an earlier run of the same references are carried over as resolved.

`private void carryOver(ReconciliationItem item)`

A break resolved in an earlier run stays resolved, with that note.

`private Tally reconcileRail(BankRail rail, Instant from, Instant to)`

BANK RAIL

`for (RailSettlementLine line : report)`

---- the rail's lines for the day ----

`for (Payment payment : paymentRepository.findByTypeAndProviderAndCreatedAtGreaterThanEqualAndCreatedAtLessThan(`

---- PayCore's records for the day the rail does not have ----

`if (line != null)`

The rail has it just outside the day (midnight): compare there.

`if (status.equals(PaymentStatus.FAILED.name()))`

A transfer that failed and was returned without reaching the rail is consistent on both sides: nothing happened.

`private String outboundStatus(Payment payment)`

SUCCEEDED, FAILED, or still open; a debit since reversed counts as FAILED.

`private Tally reconcileCard(FundingProvider processor, Instant from, Instant to)`

CARD PROCESSOR

`if (payment.getStatus() == PaymentStatus.FAILED)`

Declined or never arrived: nothing to settle on either side.

`public ReconciliationItem redeliver(UUID itemId, UUID officerId)`

WORKING THE BREAKS

For an inbound transfer the rail settled but PayCore never heard of: asks the rail to deliver its notification again, and credits it exactly as if it had arrived the first time (idempotent by session). The provider confirms the money; no officer types an amount.

`public ReconciliationItem resolve(UUID itemId, UUID supervisorId, String note)`

A supervisor closes a break once it is fixed or explained.

`public List<ReconciliationRun> recentRuns(int limit)`

READS

`private ReconciliationItem lockItem(UUID itemId)`

HELPERS

`private static void countPaycore(Tally tally, String status, long amountMinor)`

Money PayCore's records say moved: settled ones only.

`private record Draft(`

An item before its run exists.

`private static final class Tally`

What a run has found so far.

`void add(Draft draft)`

One item per reference and kind.

security

security/AccessTokenService.java

`public class AccessTokenService`

Issues short-lived access tokens. Claims are limited to what authorization needs: subject, login session id, roles, permissions, iat and exp. Never add profile, contact, financial or KYC data here.

`public static final String SESSION_CLAIM = "sid";`

The login session the token belongs to, so logging out, suspension and closure end the token at once instead of when it expires.

security/CurrentUser.java

`public record CurrentUser(`

The authenticated principal resolved from the access token: the customer (`sub`) and the login session it was issued for (`sid`), which names the device the request comes from.

security/CurrentUserJwtAuthenticationConverter.java

`public class CurrentUserJwtAuthenticationConverter`

Turns a verified JWT into an authentication whose principal is `CurrentUser`. Roles become ROLE_* authorities and permissions become plain authorities, so both hasRole and hasAuthority work.

The signature alone would keep a token valid until it expires, so the login session it names is also checked: after logout, logout-all, suspension or closure the token stops working on the next request, not minutes later.

security/JwtConfiguration.java

`static final String DEV_SECRET_PREFIX = "local-dev-only";`

Prefix of the development-only fallback secret in application.properties. It is public in the repository, so it must never sign production tokens.

`if (secret != null && secret.startsWith("${") && secret.endsWith("}"))`

@ConfigurationProperties binding leaves an unresolvable placeholder such as ${PAYCORE_JWT_SECRET} as literal text instead of failing, so a missing environment variable must be caught here.

security/JwtProperties.java

`private String secret;`

HMAC-SHA256 signing secret. Must be at least 32 bytes.

verification/config

verification/config/OneTimeCodeProperties.java

`private Duration timeToLive = Duration.ofMinutes(10);`

How long a code works.

`private int maxAttempts = 5;`

Wrong guesses before a code stops working.

`private Duration resendCooldown = Duration.ofSeconds(60);`

Minimum wait between two codes for the same purpose.

`private int maxPerDay = 10;`

Most codes for one purpose a customer can be sent in 24 hours.

verification/entity

verification/entity/OneTimeCode.java

`public class OneTimeCode`

A 6-digit code sent to the customer's email. Only a hash is stored. It works once, before it expires, for its purpose and subject (e.g. the device being confirmed), and only until too many wrong guesses.

`public void markUsed()`

Used, or replaced by a newer code. Either way it no longer works.

verification/enums

verification/enums/OneTimeCodePurpose.java

`public enum OneTimeCodePurpose`

What a one-time code proves. A code issued for one purpose never works for another.

`DEVICE_BINDING("confirm a new device"),`

The customer confirms a new device from their email.

`PIN_RESET("reset your transaction PIN");`

The customer resets a forgotten or locked transaction PIN.

`public String action()`

Fits "Use this code to ...".

verification/exception

verification/exception/InvalidOneTimeCodeException.java

`public class InvalidOneTimeCodeException extends RuntimeException`

Wrong, expired, used up or never sent: every kind of bad code gets the same answer.

verification/repository

verification/repository/OneTimeCodeRepository.java

`SELECT c FROM OneTimeCode c`

The newest code for the purpose, locked, so two guesses at once cannot both slip under the attempt limit.

`UPDATE OneTimeCode c`

Retires every open code for the purpose, so only the newest works.

verification/service

verification/service/OneTimeCodeService.java

`public class OneTimeCodeService`

Emails 6-digit codes and checks them. A new code retires the previous one for the same purpose; sending is rate limited so the endpoints that call this cannot flood a mailbox.

`public Instant send(Customer customer, OneTimeCodePurpose purpose, UUID subjectId)`

Sends a fresh code for the purpose. subjectId ties it to one thing (the device being confirmed); null when there is nothing to tie.

@return when the code expires

`TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization()`

Only once the code is committed, and without holding a connection while the mail server answers.

`public void consume(UUID customerId, OneTimeCodePurpose purpose, UUID subjectId, String rawCode)`

Uses the newest code for the purpose. A wrong guess is counted and stays counted: callers must not roll back on `InvalidOneTimeCodeException`.

`private String hash(UUID customerId, String code)`

Salted with the customer, so equal codes for two customers differ.

Backend tests

account

account/AccountIntegrationTest.java

`class AccountIntegrationTest`

The account API end to end: authentication, permissions, ownership, the KYC gate, staff status changes and the audit history.

`void anonymousCallersShouldBeRejected() throws Exception`

AUTHENTICATION AND THE KYC GATE

`void verifiedCustomerShouldOpenAndReadTheirAccount() throws Exception`

OPENING AND READING OWN ACCOUNTS

`assertThat(jdbcTemplate.queryForObject(`

Opening is audited.

`void supportShouldReadButNotChangeAccounts() throws Exception`

STAFF

`statusChange(adminToken, accountId, "freeze", "Too early")`

A pending account cannot be frozen until it is activated.

`statusChange(adminToken, accountId, "activate", "Approving myself")`

Segregation of duties: nobody activates their own account.

`private String openAccountWithNewCustomer() throws Exception`

HELPERS

Registers holder@example.com with verified KYC and opens an NGN account for them; returns the account id.

`jdbcTemplate.update(`

KYC review itself is covered by KycFlowIntegrationTest.

account/entity

account/entity/AccountTest.java

`void shouldActivatePendingAccount()`

---------- activate ----------

`void shouldFreezeActiveAccount()`

---------- freeze / unfreeze ----------

`void shouldClosePendingAccount()`

---------- close ----------

account/repository

account/repository/AccountRepositoryIntegrationTest.java

`class AccountRepositoryIntegrationTest`

The database rules must hold on their own, without the service's checks, because concurrent requests can pass application checks together.

account/service

account/service/AccountServiceTest.java

`void shouldOpenPendingAccountForCustomer()`

OPENING

`verify(customerRepository).getReferenceById(customerId);`

A reference only: the auth check has already vetted the customer for this request, and KYC is checked at activation.

`verifyNoMoreInteractions(accountRepository);`

No "does it exist?" queries: the unique constraints decide.

`void shouldRetryWithNewNumberAfterCollision()`

ACCOUNT NUMBER COLLISIONS

`verify(transactionManager, times(2)).getTransaction(any());`

Each attempt had its own transaction; the failed one rolled back.

`void shouldReturnOwnAccount()`

READING

`void shouldActivatePendingAccountOfVerifiedCustomer()`

ACTIVATION

`void shouldFreezeAccountAndRecordWhoAndWhy()`

STAFF STATUS CHANGES

`private Account givenStoredAccount()`

HELPERS

account/service/NubanAccountNumberGeneratorTest.java

`assertThat(NubanAccountNumberGenerator.checkDigit("058", "123456789"))`

Worked by hand with weights 3,7,3,3,7,3,3,7,3,3,7,3: 0,5,8 | 1..9 -> 0+35+24+3+14+9+12+35+18+21+56+27 = 254 -> 10 - 4 = 6

`assertThat(NubanAccountNumberGenerator.checkDigit("999", "000000001"))`

9,9,9 | 0..0,1 -> 27+63+27+3 = 120 -> (10 - 0) % 10 = 0

`assertThat(seen).hasSizeGreaterThan(9_990);`

10^9 serials: a handful of collisions in 10k draws would already mean the randomness is broken.

authorization/controller

authorization/controller/AdminRoleIntegrationTest.java

`private void bootstrapAdmin(UUID customerId)`

The first admin has to be created outside the API, the same way an operator would seed it.

authorization/repository

authorization/repository/RoleRepositoryIntegrationTest.java

`void shouldSeedOperationsRolePermissions()`

Operations officers request and approve money corrections; nobody else touches money, and they cannot manage customers or roles.

banktransfer

banktransfer/BankTransferIntegrationTest.java

`"paycore.rails.provider=simulated",`

Money in and out of PayCore through the (simulated) interbank rail, end to end against PostgreSQL: the signed inbound webhook, the Test Bank simulator, name enquiry, outbound transfers with automatic reversal of rejected payments, idempotency, concurrency, and the settlement account staying equal to what PayCore owes its customers.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void signedNotificationShouldCreditTheAccountAgainstSettlement() throws Exception`

INBOUND: THE RAIL WEBHOOK

`mockMvc.perform(authorized(get("/api/v1/ledger/accounts/{id}/statement", esther.accountId()), esther.token()))`

The customer sees who sent it.

`webhook(notification("SESSION-0002", esther.accountNumber(), "9000", null))`

The same session with a different amount is not believed.

`String signature = InboundWebhookService.sign(SECRET, body);`

Signed, then tampered with.

`void customerShouldSendThemselvesMoneyFromTheirTestBankAccount() throws Exception`

INBOUND: THE TEST BANK SIMULATOR

`fromTestBank(esther, esther.accountNumber(), "2000.01")`

Test Bank only sends pretend money up to its limit per account.

`void nameEnquiryShouldNameTheAccountHolder() throws Exception`

NAME ENQUIRY

`nameEnquiry(esther, "999001", testBank.accountNumberFor(UUID.fromString(esther.customerId())))`

Their own Test Bank account carries their own name.

`void outboundTransferShouldDebitTheCustomerAndPayThroughTheRail() throws Exception`

OUTBOUND

`assertThat(balance(esther)).isEqualByComparingTo("1000");`

The customer has their money back; the history shows both.

`assertThat(jdbcTemplate.queryForMap(`

The rail made the reversal, not staff: no staff note, no approver.

`sendOut(esther, "4123456789", "100.99", "out-0002")`

A retry reports the same failed transfer and pays nothing.

`sendOut(esther, "4123456789", "1000.01", "out-0004")`

Over budget: refused before anything is sent.

`sendOut(esther, "0001234567", "10", "out-0005")`

Unknown beneficiary: refused before anything is debited.

`List<Callable<Object>> tasks = new ArrayList<>();`

Two ₦800 transfers at once: only one fits.

`sendOut(john, "4123456789", "50.99", "out-0010");`

rejected and reversed

`assertThat(jdbcTemplate.queryForObject(`

And the whole ledger balances.

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private Holder funded(String name, String amount) throws Exception`

An active customer with money that arrived from another bank.

`private BigDecimal settlementBalance()`

Money PayCore holds at its bank: debits minus credits on settlement.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

bills

bills/BillPaymentIntegrationTest.java

`"paycore.bills.provider=simulated",`

Bill payments end to end through the simulated aggregator: the debit, what the biller delivers (a meter token), refusals reversed, lost answers settled by requery, prices set by the biller, and the same PIN, device and limits as any payment.

Recovery runs on demand: anything is stale at once, a check is due at once, and "not found" is believed at once.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void billersShouldBeListedWithWhatEachNeeds() throws Exception`

WHAT CAN BE PAID

`void airtimeShouldBeDebitedAndDelivered() throws Exception`

PAYING

`mockMvc.perform(authorized(get("/api/v1/payments/{id}", paymentId), esther))`

The token stays with the payment.

`assertThat(balance(esther)).isEqualByComparingTo("7999.02");`

The money stays reserved while nobody knows.

`void billShouldNeedThePin() throws Exception`

THE SAME RULES AS ANY PAYMENT

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private Holder holder(String name) throws Exception`

A tier 3 customer with an ACTIVE NGN account, a trusted device and a PIN.

bills/provider

bills/provider/VtpassBillProviderTest.java

`class VtpassBillProviderTest`

The VTpass adapter against canned HTTP responses; it never calls VTpass.

`void readsShouldSendTheApiKeyAndPublicKey()`

AUTHENTICATION

`void deliveredShouldCarryTheTransactionId()`

ANSWERS

`void productsShouldBeReadEvenWhenVtpassMisspellsVariations()`

PRODUCTS AND VERIFICATION

`private void respond(ResponseCreator response)`

HELPERS

common/config

common/config/ForwardedHeadersIntegrationTest.java

`"PAYCORE_JWT_SECRET=test-only-prod-profile-secret-0123456789abcdef")`

Runs the prod profile on a real Tomcat (MockMvc bypasses Tomcat, so it cannot exercise RemoteIpValve). The test client connects over loopback, which is inside the default trusted-proxy range, so it plays the role of the load balancer.

`String sessionId =`

The client sent "X-Forwarded-For: 6.6.6.6"; the proxy appended the address it actually saw. The right-most untrusted entry wins.

client -> edge proxy (10.0.0.5) -> load balancer (loopback)

`assertThat(restTemplate.exchange(`

The KYC profile was created at registration, so it only needs starting.

`assertThat(jdbcTemplate.queryForObject(`

Keyed on the real client's /64, not the proxy and not the forgery.

common/config/RequiredSecretsConfigurationTest.java

`class RequiredSecretsConfigurationTest`

Sets spring.datasource.password directly (without loading the project's config files), so the result does not depend on whether this machine has a paycore-local.properties.

common/config/UntrustedPeerForwardedHeadersIntegrationTest.java

`"server.tomcat.remoteip.internal-proxies=10\\.1\\.2\\.3",`

Prod profile with the trusted-proxy list narrowed to a single load balancer address (as SERVER_TOMCAT_REMOTEIP_INTERNALPROXIES would do in production). The test client on loopback is now an ordinary, untrusted client, so its X-Forwarded-For must be ignored.

common/net

common/net/ClientNetworkTest.java

`long start = System.nanoTime();`

Each of these would trigger a DNS lookup if passed to InetAddress.getByName; a slow or failing lookup would show up as a long runtime. They must be rejected before parsing.

`assertThat(ClientNetwork.key(input)).isEqualTo(ClientNetwork.UNKNOWN);`

Hostnames are never resolved; everything unparseable shares one bucket, which is still rate limited.

common/persistence

common/persistence/DatabasePrivilegeIntegrationTest.java

`"paycore.rails.provider=simulated",`

The production database setup (V29): Flyway migrates as the owner, the API runs as paycore_app with only the privileges it needs, and paycore.database.require-least-privilege=true would stop it from starting otherwise. Proves the whole money flow works under those privileges, and that the application's own user cannot do what the ledger triggers cannot stop: delete, truncate, rewrite entries, disable triggers, or change the schema.

`private JdbcTemplate jdbcTemplate;`

Connected as paycore_app, like everything else in the application.

`assertThat(jdbcTemplate.queryForObject("SELECT current_user", String.class)).isEqualTo("paycore_app");`

It started at all, with require-least-privilege on.

`mockMvc.perform(authorized(post("/api/v1/simulator/test-bank/transfers"), token)`

In from Test Bank, a top-up, out to another bank, and a timeout the simulator records: every table and both schemas written.

`String customerId = jdbcTemplate.queryForObject(`

Revoking a role is the one delete the application makes.

`private JdbcTemplate ownerJdbc()`

HELPERS

`private String customerWithActiveAccount(String name) throws Exception`

Registers, verifies KYC, opens and activates an account; returns the access token.

`mockMvc.perform(authorized(post("/api/v1/security/pin"), token)`

Money moves only with a transaction PIN, from a trusted device (the first one a customer signs in from is).

common/persistence/UuidV7RekeyMigrationIntegrationTest.java

`class UuidV7RekeyMigrationIntegrationTest`

Applies migrations up to V7, adds data that references the seeded v4 ids, then applies V8 and checks every id is v7 and no link was lost. Runs Flyway directly (no Spring context) so the database can be inspected between versions.

customer/controller

customer/controller/CustomerControllerTest.java

`private LoginSessionRepository loginSessionRepository;`

Needed by CurrentUserJwtAuthenticationConverter, which this slice loads.

customer

customer/EmailVerificationIntegrationTest.java

`class EmailVerificationIntegrationTest`

Email verification end to end: the link sent at registration, activation, one-time and expiring links, resend limits, and email changes.

`String token = login("ada@example.com");`

Signing in works before verification; the customer is pending.

`assertThat(jdbcTemplate.queryForObject(`

Only the hash is stored, never the token itself.

`resend(token)`

The registration link was just sent: wait a minute.

`verify(first).andExpect(status().isBadRequest());`

Only the newest link works.

`login("ada.new@example.com");`

The password login follows the new address...

`register("ada@example.com", "08012340002");`

...and the old one is free for someone else to register.

`jdbcTemplate.update("UPDATE email_verification_tokens SET used_at = NULL");`

Even if it had not been replaced by the new link.

`private void ageTokens()`

Moves every link back past the resend cooldown.

customer/service

customer/service/CustomerServiceTest.java

`void shouldAssignCustomerRoleOnRegistration()`

REGISTRATION

`verify(roleAssignmentService).assignRole(`

System-assigned, so no acting admin is recorded.

`assertThatThrownBy(() ->`

The exception rolls back the @Transactional registration, so the saved customer is not committed.

`verify(eventPublisher)`

KYC listens for this to create the customer's profile.

`verify(identityService)`

Password is no longer part of Customer. It must be passed to IdentityService instead.

The encoded password must be sent to IdentityService.

`verify(identityService, never())`

The raw password must never be sent to IdentityService.

The insert failed first, so nothing else was written.

`var order = inOrder(passwordEncoder, transactionManager, customerRepository);`

BCrypt is slow on purpose; no pooled connection may wait on it.

`verify(customerRepository, never()).existsByEmail(anyString());`

Duplicates come from the unique constraints, not extra queries.

`assertThat(captor.getValue().getEmail())`

The unique constraint only catches duplicates if every email is stored in the same form.

`void shouldGetCustomerProfile()`

GET CUSTOMER

`void shouldUpdateCustomerFirstName()`

PROFILE UPDATE

`void shouldSuspendActiveCustomer()`

SUSPENSION

`void shouldReactivateSuspendedCustomer()`

REACTIVATION

`void shouldCloseCustomerAccount()`

CLOSURE

demo

demo/DemoModeIntegrationTest.java

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void customerAndAdminShouldCompleteTheDemoJourney() throws Exception`

The whole walkthrough the demo UI guides visitors through, with the simulated provider and a NIN instead of a BVN.

`mockMvc.perform(`

BVN has its own allowance.

`String adminToken = accessToken(ADMIN_EMAIL, ADMIN_PASSWORD);`

Admin actions on anyone else still work.

`demoRecipientSeeder.seed();`

Seeding again reuses the recipient and its account.

`mockMvc.perform(post("/api/v1/simulator/test-bank/transfers")`

Money arrives from another bank; nobody at PayCore can type it in.

`String recipientAccount = recipientAccountId();`

Nobody can take the shared recipient out of use for everyone else.

`mockMvc.perform(post("/api/v1/ops/requests/reversals")`

The admin cannot move money at all...

`String officer = accessToken(DemoStaff.OFFICER.email(), ADMIN_PASSWORD);`

...but the two shared operations officers can reverse a transfer into the recipient: one requests, the other approves.

`mockMvc.perform(post("/api/v1/admin/accounts/{id}/freeze", visitorAccount)`

And freezing an ordinary customer's account is unaffected.

`String body = JsonPath.read(outbox, "$[0].body");`

The link in it verifies the reader.

`mockMvc.perform(`

The demo recipient is seeded verified (tier 3, no balance cap), so only the new customer has not started.

funding

funding/FundingIntegrationTest.java

`"paycore.funding.provider=simulated",`

Customer top-ups through the simulated provider, end to end against PostgreSQL: the double-entry posting, declines, idempotency, the per-account limit (also under concurrency), ownership and account states, and the database's own guarantees for provider deposits.

The limit is NGN 1,000 here so it is quick to reach.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void topUpShouldCreditTheAccountAgainstSettlement() throws Exception`

A TOP-UP

`assertThat(jdbcTemplate.queryForMap(`

The customer initiated it; no staff note, the provider instead.

`assertThat(settlementBalance()).isEqualByComparingTo("250.50");`

PayCore's settlement account holds the money it now owes Esther.

`mockMvc.perform(authorized(get("/api/v1/ledger/accounts/{id}/statement", esther.accountId()), esther.token()))`

The customer's own statement shows it like any other credit.

`void retryShouldReturnTheSameTopUpAndChargeOnce() throws Exception`

IDEMPOTENCY

`mockMvc.perform(authorized(post("/api/v1/ledger/accounts/{id}/transfers", esther.accountId()), esther.token())`

A transfer key is not a top-up key.

`void topUpsShouldStopAtTheLimit() throws Exception`

THE LIMIT

`receiveFromOtherBank(esther, "5000", "session-0001");`

Transfers in, from other banks or other customers, do not use up the limit.

`reverseThroughOperations(topUp, "Chargeback");`

A reversed top-up (a chargeback, maker-checker approved) frees its share again.

`assertThat(jdbcTemplate.queryForList("SELECT status FROM payments", String.class))`

A top-up refused after its card was charged (it passed the limit check, then lost the race for the account lock) is not lost: it waits in review for operations to refund it.

`void anotherCustomersAccountShouldNotBeFound() throws Exception`

OWNERSHIP, STATES AND INPUT

`void oneProviderPaymentCanOnlyBeCreditedOnce() throws Exception`

DATABASE GUARANTEES

`assertThatThrownBy(() -> insertProviderDeposit(admin.customerId(), "key-staff-0001", null, null, null))`

Staff deposits still need their note.

`private record Holder(`

HELPERS

`private Holder activeHolder(String name) throws Exception`

A customer with verified KYC and an ACTIVE NGN account, set directly (KYC review and activation have their own tests).

`private void receiveFromOtherBank(Holder holder, String amount, String sessionId)`

Money in from another bank, as the rail would report it.

`private void reverseThroughOperations(String transactionId, String reason) throws Exception`

One operations officer requests the reversal, another approves it.

`private List<Object> runTogether(List<Callable<Object>> tasks) throws Exception`

Starts every task at the same moment and returns each result, or the exception it threw.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

funding/provider

funding/provider/InMemoryCardPaymentRecords.java

`class InMemoryCardPaymentRecords implements CardPaymentRecords`

The simulated processor's records without a database, for unit tests.

funding/provider/SimulatedFundingProviderTest.java

`assertThat(provider.refund(collection.merchantReference())).startsWith("SIMREF-");`

It was charged all the same: it can be refunded.

`void aRestartedSimulatorShouldRememberEarlierCharges()`

What a restart looks like: a new simulator over the same records.

funding/service

funding/service/FundingServiceTest.java

`class FundingServiceTest`

The order of the steps and what happens when the provider says no or cannot answer. The ledger and payment sides run against PostgreSQL in FundingIntegrationTest and PaymentIntegrationTest.

`void aProviderFailureShouldLeaveThePaymentUnknownNotFailed()`

A timeout is not a "no": the customer may have been charged. The payment is UNKNOWN, a status check will settle it, and the customer is told it is pending rather than that nothing happened.

identity/controller

identity/controller/AuthenticationControllerTest.java

`private GoogleSignInService googleSignInService;`

Google sign-in is covered by GoogleSignInIntegrationTest.

`private LoginSessionRepository loginSessionRepository;`

Needed by CurrentUserJwtAuthenticationConverter, which this slice loads.

`SecurityContextHolder.getContext().setAuthentication(`

Filters are disabled in this slice, so populate the context that @AuthenticationPrincipal reads from directly.

identity/controller/AuthenticationIntegrationTest.java

`kycProfileRepository.deleteAll();`

Registration creates a KYC profile for every customer.

identity/controller/RefreshTokenIntegrationTest.java

`refresh(tokenA)`

Replaying A is treated as theft.

`assertThat(refreshTokenRepository`

The revocation is committed despite the error response.

`refresh(tokenB)`

B was legitimately issued but is now unusable: re-authenticate.

identity/google

identity/google/GoogleSignInIntegrationTest.java

`class GoogleSignInIntegrationTest`

Sign in and sign up with Google, with Google's token check stubbed (JwtGoogleIdTokenVerifierTest covers the real one): new customers, returning customers, linking by email, and the pre-account takeover defence.

`mockMvc.perform(get("/api/v1/kyc/status").header("Authorization", "Bearer " + token))`

A normal customer: CUSTOMER role, a KYC profile, no password.

`assertThat(events.stream(EmailSentEvent.class)).isEmpty();`

Google already verified the address: no link is sent.

`login("ada.obi@gmail.com").andExpect(status().isOk());`

Both ways in now work.

`String customerId = register("ada.obi@gmail.com");`

Someone registered Ada's address with a password but never proved they own it...

`google("ada-token")`

...then the real Ada signs in with Google.

`login("ada.obi@gmail.com")`

The password no longer works, and the earlier session is over.

`private ResultActions google(String credential) throws Exception`

identity/google/JwtGoogleIdTokenVerifierTest.java

`class JwtGoogleIdTokenVerifierTest`

The same issuer, audience and expiry checks as production, on tokens signed with a local key instead of Google's.

identity/repository

identity/repository/IdentityRepositoryIntegrationTest.java

`assertThat(`

Social identity factory will be added when Google/Apple authentication is implemented.

This test documents the database design:

UNIQUE(customer_id, provider)

rather than:

UNIQUE(customer_id)

identity/service

identity/service/AuthenticationServiceTest.java

`var order = inOrder(passwordEncoder, transactionManager, loginSessionService);`

BCrypt is slow on purpose; no pooled connection may wait on it.

`void shouldIssueNewAccessTokenOnRefresh()`

REFRESH

`void shouldRevokeSessionAndRefreshTokensOnLogout()`

LOGOUT

identity/service/RefreshTokenServiceTest.java

`private void stubSaveAssigningId()`

Mimics persist: the UUIDv7 generator assigns the id on save.

kyc/controller

kyc/controller/KycControllerTest.java

`private LoginSessionRepository loginSessionRepository;`

Needed by CurrentUserJwtAuthenticationConverter, which this slice loads.

`void authenticate()`

Filters are disabled in this slice, so populate the context that @AuthenticationPrincipal reads from directly.

`verify(kycService).verifyBvn(`

The connection's address is used; a client-supplied X-Forwarded-For header is ignored.

`mockMvc.perform(get("/api/v1/kyc/{kycId}/bvn-attempts", kycId))`

No paging params: defaults page=0, size=20.

kyc/entity

kyc/entity/KycProfileTest.java

`assertThat(profile.getReviewReason()).isNull();`

The earlier request's reason no longer applies.

kyc

kyc/KycFlowIntegrationTest.java

`class KycFlowIntegrationTest`

REGISTER -> LOGIN -> CREATE KYC -> START -> UPLOAD DOCUMENT -> BVN CHECK -> SUBMIT -> START REVIEW -> VERIFIED / REJECTED / ADDITIONAL_INFO, over real HTTP, JWT, PostgreSQL and file storage. Only the KYC provider is mocked: builds never call Dojah.

`private static final byte[] PDF =`

A tiny but valid-looking PDF header; the upload only sniffs magic bytes.

`uploadDocument(customer, "NATIONAL_ID")`

Nothing can be added before the customer starts.

`String storageKey =`

The file is on disk under a generated name; only metadata is in the database.

`when(kycProvider.verifyBvn(any()))`

First attempt: typo in the name, so the provider says FAILED.

`verifyBvn(customer)`

PASSED is one verification signal, not a finished KYC package.

`uploadDocument(customer, "PASSPORT")`

Submitted packages are frozen.

`assertThat(jdbcTemplate.queryForList(`

Both attempts are kept; the BVN itself is stored nowhere.

`authorized(post("/api/v1/kyc/{id}/approve", kycId), customer)`

A normal customer cannot approve, not even their own KYC.

`authorized(post("/api/v1/kyc/{id}/approve", kycId), admin)`

Cannot approve before a reviewer has picked it up.

`authorized(get("/api/v1/kyc/tier"), customer)`

One identity number and a document earn tier 1.

`verifyBvn(customer)`

A number already proven cannot be checked again.

`authorized(post("/api/v1/kyc/start"), customer)`

REJECTED is final.

`uploadDocument(customer, "PROOF_OF_ADDRESS")`

The customer has to resume before changing anything.

`assertThat(Long.parseLong(retryAfter))`

Roughly 24h until the first failure leaves the window.

`verify(kycProvider, times(3)).verifyBvn(any());`

The 4th request never reached the provider and was not recorded.

`authorized(post("/api/v1/kyc/{id}/bvn-attempts/reset", kycId), customer)`

Customers cannot lift their own limit.

`authorized(`

Paging: 3 attempts in pages of 2. The summary is the same on both pages even though page 1 holds only the oldest attempt.

`authorized(get("/api/v1/kyc/{id}/bvn-attempts", kycId), customer)`

Customers cannot see the admin view either.

`authorized(get("/api/v1/kyc/{id}/bvn-attempts", kycId), admin)`

What the admin sees before resetting: limited, 3 counted.

After the reset: history kept, but none of it counts any more.

`verifyBvn(customer)`

Fresh attempts: the next failure leaves 2.

`assertThat(jdbcTemplate.queryForObject(`

Nothing was deleted: 3 old failures + 1 new one, and the reset is recorded against the admin.

`when(kycProvider.verifyBvn(any()))`

A slow provider widens the window in which requests could race.

A FAILED check does not count.

`private String startKyc(String accessToken) throws Exception`

Creates and starts the caller's KYC; returns its id.

`private String createKycUnderReview(String customer, String reviewer)`

Customer starts, passes BVN, uploads a document and submits; the reviewer then picks it up.

kyc/KycNetworkRateLimitIntegrationTest.java

`"paycore.kyc.bvn.ip-max-attempts=3",`

Per-network BVN rate limit over real HTTP, JWT and PostgreSQL, with a low limit (3 checks per network per hour) so it is reachable without tripping the per-customer limit. Only the KYC provider is mocked.

`for (int i = 1; i <= 3; i++)`

Three different accounts, one check each: under every per-customer limit, but they use up the network's budget.

`String fourth = customerWithKyc("acct4@example.com", "08011111134");`

A fourth fresh account from the same network is refused.

`assertThat(Long.parseLong(retryAfter)).isBetween(3_500L, 3_600L);`

About an hour until the oldest check leaves the window.

`verify(kycProvider, times(3)).verifyBvn(any());`

The blocked request never reached the provider.

`assertThat(jdbcTemplate.queryForList(`

Every recorded attempt carries its network.

`verifyBvn(fourth, OTHER_NETWORK)`

The same account from a different network is unaffected.

`for (int i = 1; i <= 3; i++)`

Rotating addresses inside one /64 does not escape the limit.

`List<String> accounts = new ArrayList<>();`

Six different accounts, so the per-customer row lock does not serialize them: only the network lock can keep the count right.

`private String customerWithKyc(String email, String phone) throws Exception`

Registers, logs in and creates a KYC profile; returns the access token.

`mockMvc.perform(`

The KYC profile was created at registration.

kyc/provider

kyc/provider/DojahKycProviderTest.java

`class DojahKycProviderTest`

Exercises the Dojah adapter against canned HTTP responses; it never calls Dojah. See DojahSandboxIntegrationTest for the real sandbox.

kyc/provider/DojahSandboxIntegrationTest.java

`class DojahSandboxIntegrationTest`

Calls the real (free) Dojah Sandbox with its documented test BVN. Skipped unless DOJAH_APP_ID and DOJAH_SECRET_KEY are set, so normal builds never touch the network.

`assertThat(result.provider()).isEqualTo("DOJAH");`

Sandbox data is mock data, so only assert we got a real verdict.

kyc/repository

kyc/repository/KycVerificationRepositoryIntegrationTest.java

`kycVerificationRepository.saveAndFlush(`

Other verification types are not part of BVN history.

`assertThat(last.getContent().getFirst().getCreatedAt())`

The last page holds the oldest attempt.

`kycVerificationRepository.saveAndFlush(KycVerification.create(`

Two customers, same network, any result: all count.

`KycVerification old = KycVerification.create(`

Excluded: too old, another network, another verification type.

`assertThat(kycVerificationRepository.lockClientNetwork("203.0.113.7"))`

Re-entrant within the same transaction; released at rollback.

kyc/service

kyc/service/KycServiceTest.java

`void shouldCreateKycProfile()`

CREATE / GET

`void shouldStartKyc()`

START / SUBMIT

`assertThat(response.reviewReason()).isEqualTo("More please");`

The customer can still see what the reviewer asked for.

`void shouldVerifyBvnSuccessfully()`

BVN

`void shouldReportRemainingAttemptsAfterFailure()`

RETRY LIMIT

`assertThat(response.remainingAttempts()).isEqualTo(1);`

1 earlier failure + this one = 2 of 3.

`verifyNoInteractions(kycProvider);`

Refused before contacting the provider or recording anything.

`void shouldRecordNormalizedNetworkWithAttempt()`

NETWORK (IP) LIMIT

`void shouldResetBvnAttemptsAsAdmin()`

RETRY LIMIT RESET

`verifyNoInteractions(kycVerificationRepository);`

History is untouched: nothing is deleted.

`void shouldShowAttemptsForCustomerBelowTheLimit()`

ADMIN VIEW OF BVN ATTEMPTS

`assertThat(view.retryAfter())`

Same rule verifyBvn uses: oldest counted failure + window.

`assertThat(view.attempts()).hasSize(3);`

History is still visible.

`private void stubHistory(KycVerification... newestFirst)`

Stubs both queries the admin view makes, over the same attempts: the page of history (returned as one page) and the counted-failures query, which applies the real repository predicate (FAILED, after `since`, oldest first) so the counting logic is exercised.

`when(kycVerificationRepository`

Page 1 of size 1 holds only the middle attempt...

...but the summary still sees all three counted failures.

`void shouldApproveKycUnderReview()`

REVIEW

`private void stubProfileInProgress()`

Most BVN tests need a profile the customer has already started.

`private void stubProfile()`

BVN checks load the profile with a row lock; other operations use the plain lookup. Each test only exercises one of them.

ledger/entity

ledger/entity/LedgerTransactionTest.java

`class LedgerTransactionTest`

The double-entry rules: positive entries, one currency, debits equal credits, and the INITIATED -> POSTED -> REVERSED lifecycle.

`assertThatThrownBy(() -> LedgerEntry.debit(transfer("USD"), esther, dollars))`

A USD transaction cannot touch an NGN account.

`assertThatThrownBy(() -> LedgerTransaction.createAdjustment(`

Four eyes: the maker cannot approve their own correction.

`private LedgerTransaction posted()`

A posted transfer with an id, ready to be reversed.

ledger

ledger/LedgerIntegrationTest.java

`class LedgerIntegrationTest`

The ledger end to end against PostgreSQL: balanced entries, derived balances, ownership, account states, idempotency, database constraints and concurrent transfers.

`private Holder admin;`

An ADMIN (with LEDGER_POST) who funds accounts with deposits.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void transferShouldPostBalancedEntriesAndMoveTheBalances() throws Exception`

A TRANSFER

`fund(esther, 10_000_000);`

₦100,000

`assertThat(jdbcTemplate.queryForList(`

Stored in kobo, positive, and debits equal credits.

`void anonymousCallersShouldBeRejected() throws Exception`

SECURITY

`mockMvc.perform(authorized(post("/api/v1/ledger/accounts/{id}/transfers", esther.accountId()), mallory.token())`

Mallory names Esther's account as the source.

`for (Holder involved : List.of(esther, john))`

Sender and recipient.

`mockMvc.perform(authorized(get("/api/v1/ledger/transactions/{id}", id), mallory.token()))`

Anyone else: indistinguishable from a transaction that does not exist.

`void sourceAccountMustBeActive(String status) throws Exception`

ACCOUNT STATE, CURRENCY AND FUNDS

`assertThat(transactionCount()).isEqualTo(1);`

only the funding

`fund(esther, 100_000);`

₦1,000

`transfer(esther, john.accountNumber(), "1000.00", "transfer-0011", null)`

Exactly the balance is fine.

`transfer(esther, esther.accountNumber(), "10", "transfer-0012", null)`

To yourself.

`transfer(esther, "0000000000", "10", "transfer-0013", null)`

To an account that does not exist.

`transfer(esther, admin.accountNumber(), "10.005", "transfer-0014", null)`

More decimal places than kobo: rejected, never rounded.

`transfer(esther, admin.accountNumber(), "0", "transfer-0015", null)`

Zero and negative amounts, malformed account number, short key.

`void retryWithTheSameKeyShouldReturnTheOriginalAndMoveMoneyOnce() throws Exception`

IDEMPOTENCY

`assertThat(transactionCount()).isEqualTo(2);`

funding + one transfer

`transfer(esther, john.accountNumber(), "300", "transfer-reuse", null)`

Different amount, recipient or description under the same key.

`String second =`

John's identical key is his own: a new transfer, not Esther's replayed.

`void databaseShouldEnforceLedgerInvariants() throws Exception`

DATABASE CONSTRAINTS

`assertThatThrownBy(() -> insertEntry(transactionId, esther.accountId(), "CREDIT", 0, "NGN"))`

Amounts are positive; direction is the entry type.

`assertThatThrownBy(() -> insertEntry(transactionId, esther.accountId(), "CREDIT", 100, "USD"))`

An entry's currency must be its transaction's and its account's.

`assertThatThrownBy(() -> insertEntry(UUID.randomUUID().toString(), esther.accountId(), "CREDIT", 100, "NGN"))`

Every entry belongs to a real transaction and account.

`String reference = jdbcTemplate.queryForObject(`

References are unique.

`insertBalancedTransaction("TXN-UNIQUE-REF-1", "same-key-0001", esther);`

Idempotency keys are unique per initiating customer. (Inbound transfers have no initiator; their rail session id is unique instead.)

`void concurrentTransfersMustNotSpendTheSameMoneyTwice() throws Exception`

CONCURRENCY

`fund(esther, 10_000_000);`

₦100,000

`List<Callable<Object>> tasks = List.of(`

Two ₦80,000 transfers at once: only one can fit.

`List<Callable<Object>> tasks = new ArrayList<>();`

A -> B and B -> A at the same time lock the same two rows; the fixed lock order is what keeps this from deadlocking.

`assertThat(jdbcTemplate.queryForObject(`

The whole ledger still balances: every debit has its credit.

`fund(esther, 1_000_000);`

₦10,000

`List<Callable<Object>> tasks = new ArrayList<>();`

A client that times out and retries hard: the same request, same key, ten times at once.

`fund(esther, 10_000_000);`

₦100,000

`List<Callable<Object>> tasks = new ArrayList<>();`

Forty ₦3,000 transfers at once: exactly 33 fit (₦99,000).

Two separate approved requests (different keys) race each other.

`void booksShouldBalanceAfterAMixedConcurrentWorkload() throws Exception`

The trial balance after a mixed burst of money in, transfers inside PayCore and money out: every transaction balances, the whole ledger nets to zero, and the settlement account holds exactly what PayCore owes its customers.

`void postedEntriesShouldNeverChangeEvenWhenReversed() throws Exception`

IMMUTABLE HISTORY (V25)

`assertThat(jdbcTemplate.queryForList(snapshot, String.class, transfer)).isEqualTo(before);`

The original's entries are exactly as they were; the reversal is a new transaction with its own two entries.

`assertThatThrownBy(() -> jdbcTemplate.update(`

Entries are append-only.

A transaction's facts never change, and it is never deleted.

Its status only moves forward.

`assertThatThrownBy(() -> transactionTemplate.executeWithoutResult(status ->`

Debit 100, credit 99: each row is valid on its own, the set is not.

`assertThatThrownBy(() -> insertTransaction("TXN-EMPTY-1", "empty-0001"))`

A posted transaction with no entries at all.

`void statementShouldShowOpeningClosingTotalsAndRunningBalances() throws Exception`

STATEMENTS

`backdate(fund(esther, 100_000), "2026-08-15T10:00:00Z");`

+1000 before

`backdate(transfer, "2026-09-02T09:00:00Z");`

-200

`backdate(fund(esther, 50_000), "2026-09-10T12:00:00Z");`

+500

`backdate(withdrawal, "2026-09-20T08:00:00Z");`

-100

`backdate(fund(esther, 5_000), "2026-10-01T09:00:00Z");`

+50 after

`statement(esther, "?from=2026-09-01&to=2026-09-30&size=2&page=1")`

Page 1 starts from the opening balance plus page 0, without it.

`statement(esther, "?from=2026-09-01&to=2026-09-30&size=2&page=5")`

Past the end: no lines, same totals.

`backdate(fund(esther, 1_000), "2026-08-31T23:30:00Z"); // ₦10, 1 Sep in Lagos`

23:30 UTC is 00:30 the next day in Lagos (UTC+1).

`backdate(fund(esther, 1_000), "2026-08-31T23:30:00Z");`

₦10, 1 Sep in Lagos

`backdate(fund(esther, 10_000), "2026-09-30T23:30:00Z");`

₦100, 1 Oct in Lagos

`statement(esther, "")`

No dates: month to date, which includes everything just posted.

`mockMvc.perform(authorized(`

System accounts have no statement here.

`"?from=2026-09-10&to=2026-09-01",`

backwards

`"?from=2025-01-01&to=2026-09-30",`

longer than a year

`"?from=01-09-2026",`

not an ISO date

`private record Holder(`

HELPERS

`private Holder activeHolder(String name) throws Exception`

A customer with verified KYC and an ACTIVE NGN account. KYC review and activation have their own tests; here they are set directly.

`private Holder staff(Holder holder, String role) throws Exception`

Grants a role directly (role management has its own tests) and signs in again so the token carries its permissions.

`private String fund(Holder holder, long kobo)`

Puts money into an account the way it really arrives: a transfer from another bank, reported by the rail. Returns the transaction id.

`private String sendToOtherBank(Holder holder, String amount, String key)`

Money out to another bank (the rail is not involved at this level).

`private void reverseThroughOperations(String transactionId) throws Exception`

A reversal requested by one operations officer and approved by another.

`private void backdate(String transactionId, String instant)`

Moves a transaction's entries to a fixed moment, for statements. Entries are append-only, so this is a fixture-only maintenance edit.

`private void asLedgerMaintenance(Runnable work)`

Runs the work in one database transaction with the V25 escape hatch on.

`private void insertBalancedTransaction(String reference, String idempotencyKey, Holder holder)`

A posted transaction and its two entries, committed together: the database refuses a posted transaction that does not balance.

`private String transferId(Holder from, Holder to, String amount, String key) throws Exception`

Transfers and returns the transaction id.

`private BigDecimal settlementBalance() throws Exception`

The NGN settlement account's balance: money PayCore holds.

`private List<Object> runTogether(List<Callable<Object>> tasks) throws Exception`

Starts every task at the same moment and returns each result, or the exception it threw.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

ledger/StaffNoteMigrationIntegrationTest.java

`class StaffNoteMigrationIntegrationTest`

Applies migrations up to V20, when staff reasons were stored as the customer-facing description, then applies V21 and checks every reason moved to staff_note and customers are left with a neutral description. Runs Flyway directly (no Spring context) so the database can be inspected between versions.

`assertThat(row(withdrawal))`

A reason that was never recorded still satisfies the new rule.

`assertThat(row(transfer))`

Customer transfers are the customer's own words: untouched.

limits

limits/KycTierLimitsIntegrationTest.java

`"paycore.rails.provider=simulated",`

KYC tiers end to end: what each tier may send in a day and hold, moving up to tier 2 by verifying the second identity number, and to tier 3 with a proof of address that compliance approves.

`void tierOneShouldSendAtMostFiftyThousandADay() throws Exception`

WHAT EACH TIER MAY SEND

`sendToTestBank(esther, "49999.99", "daily-rev-01").andExpect(status().isCreated())`

Test Bank rejects amounts ending in .99; the debit is reversed.

`void transferShouldNotTakeARecipientPastTheirBalanceCap() throws Exception`

WHAT EACH TIER MAY HOLD

`.andExpect(jsonPath("$.message").value("The recipient's account cannot receive this amount."));`

Never the recipient's balance or tier.

`void verifyingTheSecondIdentityNumberShouldEarnTierTwo() throws Exception`

MOVING UP

`verify(esther, "nin", "70123456789").andExpect(status().isConflict());`

Nothing more to prove with numbers.

`mockMvc.perform(authorized(post("/api/v1/kyc/tier-upgrades/{id}/approve", upgradeId), esther))`

The customer cannot approve their own request.

`mockMvc.perform(authorized(post("/api/v1/kyc/tier-upgrades/{id}/approve", upgradeId), admin))`

Decided once.

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private Holder holder(String name, int tier) throws Exception`

A verified customer at the given tier with an ACTIVE NGN account and a PIN.

`private Holder staff(Holder holder) throws Exception`

Grants ADMIN (with KYC_REVIEW) and signs in again for a token that carries it.

`private void startedWithBvn(Holder holder)`

The holder was approved on a passed BVN check alone.

operations

operations/OperationsIntegrationTest.java

`class OperationsIntegrationTest`

Maker-checker corrections end to end against PostgreSQL: reversals and manual adjustments are requested by one operations officer and posted only when a different officer approves them. Nobody else can move money by hand, and no officer can touch their own account.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void reversalShouldWaitForASecondOfficerThenMoveTheMoneyBack() throws Exception`

REVERSALS

`assertThat(balance(esther)).isEqualByComparingTo("700");`

Nothing has moved yet.

`assertThat(jdbcTemplate.queryForMap(`

The reversal records both officers and the reason.

`assertThatThrownBy(() -> jdbcTemplate.update(`

The database refuses it as well.

`List<Callable<Object>> tasks = List.of(`

Two checkers approve at the same moment.

`requestReversal(maker, transfer, "Again")`

Once reversed, it cannot be reversed again.

`transfer(john, esther, "300", "transfer-0004");`

John spends it before the checker gets to it.

`mockMvc.perform(authorized(get("/api/v1/ops/requests/{id}", requestId), checker.token()))`

Still pending: nothing half-done.

`jdbcTemplate.update("UPDATE accounts SET status = 'FROZEN' WHERE id = ?::uuid", john.accountId());`

Fraud: John's account is frozen and the money pulled back.

`void adjustmentsShouldCreditOrDebitAgainstSettlement() throws Exception`

ADJUSTMENTS

`approve(checker, adjustmentId(maker, esther, "DEBIT", "5000", "Too much"))`

A debit beyond the balance is refused at approval.

`void onlyOperationsOfficersCanRequestOrApprove() throws Exception`

WHO MAY DO WHAT

`mockMvc.perform(authorized(post("/api/v1/admin/ledger/accounts/{id}/deposits", esther.accountId()), admin.token())`

The old manual deposit endpoint is gone.

`fund(maker, "1000");`

Officers are customers too, with their own accounts.

`String requestId = adjustmentId(maker, checker, "CREDIT", "100", "Bonus");`

The maker files one for the checker's account; the checker cannot approve it.

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private Holder withAccount(Holder holder) throws Exception`

Opens and activates an NGN account for the holder.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

payment

payment/PaymentBackfillMigrationIntegrationTest.java

`class PaymentBackfillMigrationIntegrationTest`

Applies migrations up to V25, records the provider payments PayCore made before payments were tracked (a transfer that went through, one that was rejected and reversed, a card top-up, and money that never touched a provider), then applies V26 and checks each provider payment got its settled payment and nothing else did. Runs Flyway directly (no Spring context) so the database can be inspected between versions.

`.containsEntry("external_reference", FundingService.merchantReference(customer, "topup-key-0001"));`

The same reference FundingService gives the provider.

`assertThat(jdbc.queryForObject(`

Money that arrived from another bank never was a PayCore payment.

`private UUID posted(`

A posted transaction and its two entries, committed together (V25 refuses anything less).

`private UUID reversal(UUID original, UUID initiatedBy, UUID debitAccount, UUID creditAccount, long amountMinor)`

The rail's reversal of a rejected transfer: the original moves to REVERSED.

payment/PaymentIntegrationTest.java

`"paycore.rails.provider=simulated",`

Payments end to end against PostgreSQL, through the simulated rail and card provider: the state machine, provider timeouts (paid but the answer lost; never arrived), PayCore stopping mid-payment, duplicate requests, status checks that give up, and what staff and customers can see.

Recovery runs here on demand, with no waiting: anything is stale at once, a check is due at once, and "not found" is believed at once.

`private static final String BENEFICIARY = "4123456789";`

Name enquiry at Test Bank answers for this account.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void aSuccessfulTransferShouldBeTrackedFromCreatedToSucceeded() throws Exception`

TRANSFERS TO OTHER BANKS

`void aTimeoutAfterTheRailPaidShouldStayDebitedUntilAStatusCheckConfirmsIt() throws Exception`

The rail paid the beneficiary but its answer was lost. Reversing would pay twice, so the debit stands as PENDING until a status check finds the payment went through.

`sendOut(esther, "200.98", "pay-out-0003")`

A customer retry while it is unknown: the same payment, no second debit.

The customer now sees the settled outcome.

`void aTimeoutBeforeTheRailShouldBeReversedOnceTheRailHasNoRecordOfIt() throws Exception`

The request never reached the rail: the status check finds nothing, and the money goes back.

`void aTransferDebitedButNeverSentShouldBeReturned() throws Exception`

PayCore debited the customer, then stopped before calling the rail.

`void aTransferLeftProcessingShouldBeSettledByAskingTheRail() throws Exception`

The rail paid, then PayCore stopped before recording the answer.

`void checksThatNeverGetAnAnswerShouldEndInReviewAndOperationsCanCheckAgain() throws Exception`

The rail cannot answer status queries either. After max-checks the payment goes to operations, still debited (the beneficiary may have been paid); an officer's later check settles it with the rail's answer.

`assertThat(recoveryService.recoverDue().checked()).isZero();`

Nothing more happens automatically.

`doCallRealMethod().when(rail).status(anyString());`

The rail is back: an officer asks again and its answer decides.

`void aTopUpShouldBeTrackedAndLinkedToItsCredit() throws Exception`

CARD TOP-UPS

`void aTopUpChargedButUnansweredShouldBeCreditedByAStatusCheck() throws Exception`

The provider charged the card but its answer was lost. The customer is told it is pending (202), never that nothing happened; a status check finds the charge and credits it, once.

`mockMvc.perform(authorized(get("/api/v1/payments/{id}", paymentId), esther.token()))`

The customer can follow it.

`topUp(esther, "300.98", "pay-fund-0002").andExpect(status().isAccepted());`

A retry meanwhile charges nothing.

`topUp(esther, "300.98", "pay-fund-0002").andExpect(status().isCreated());`

A retry now gets the credit itself, still only once.

`void aTopUpChargedBeforePayCoreStoppedShouldBeCreditedOnRecovery() throws Exception`

The provider took the money, then PayCore stopped before crediting it.

`void aChargeThatCannotBeCreditedShouldGoToReview() throws Exception`

Charged, but by the time the charge is confirmed the account cannot take the money. The customer has paid and nothing was credited: operations must credit or refund it, so it waits in review.

`void customersShouldOnlySeeTheirOwnPaymentsAndNeverCheckThem() throws Exception`

ACCESS

`mockMvc.perform(authorized(post("/api/v1/admin/payments/{id}/status-check", paymentId), officer.token()))`

Final: the check changes nothing.

`Payment created = paymentService.initiateOutbound(`

Still in its request: refused.

`void databaseShouldEnforceThePaymentStateMachine() throws Exception`

DATABASE GUARANTEES (V26)

`assertThatThrownBy(() -> jdbcTemplate.update(`

Final is final.

What was asked for never changes, and nothing is deleted.

`Payment created = paymentService.initiateOutbound(`

Skipping a step is refused too.

`assertThatThrownBy(() -> jdbcTemplate.update(`

One key, one payment.

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private List<String> history(Payment payment)`

"from>to" for each status change, oldest first.

`private Holder funded(String name, String amount) throws Exception`

An active customer with money that arrived from another bank.

`private BigDecimal settlementBalance()`

Money PayCore holds at its bank: debits minus credits on settlement.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

payment/PaymentResolutionIntegrationTest.java

`"paycore.rails.provider=simulated",`

Payments automation could not settle, resolved by operations under maker-checker: escalation, every resolution, four eyes, the provider's answer overruling staff, and the simulators' records surviving a restart. Against PostgreSQL, through the simulated rail and processor; amounts ending in .96 are ones whose status the provider never reports.

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions CASCADE");`

Ledger rows are append-only (V25); TRUNCATE is the table owner's reset.

`void anUnconfirmableTransferShouldBeCompletedByTwoOfficers() throws Exception`

TRANSFERS

The rail paid but will never say so. Operations escalate it, confirm with the rail's support desk, and two officers complete it: the debit stands and nothing new is posted.

`requestResolution(maker, paymentId, "COMPLETE").andExpect(status().isConflict())`

Not yet in review: nothing to resolve.

`assertThat(paymentService.get(UUID.fromString(paymentId)).getStatus()).isEqualTo(PaymentStatus.NEEDS_REVIEW);`

Nothing changes until a second officer approves; the maker cannot.

`assertThat(jdbcTemplate.queryForMap(`

A staff reversal: who asked, who approved, and why.

`void aResolutionTheRailContradictsShouldBeRefused() throws Exception`

The rail says it paid: staff cannot reverse it, however sure they are.

`mockMvc.perform(authorized(post("/api/v1/admin/payments/{id}/status-check", paymentId), maker.token()))`

A status check applies the rail's answer instead.

`void aChargeTheProcessorWillNotConfirmShouldBeCreditedByTwoOfficers() throws Exception`

CARD TOP-UPS

`void aChargeThatCannotBeCreditedShouldBeRefundedToTheCard() throws Exception`

Charged, then the account was frozen, so the credit failed and the payment went to review. Closing it ("not charged") is refused, the processor says it was charged; crediting a frozen account fails and leaves the request pending; a refund settles it.

`requestResolution(maker, paymentId, "REFUND").andExpect(status().isConflict())`

One open resolution at a time.

`mockMvc.perform(authorized(get("/api/v1/payments/{id}", paymentId), esther.token()))`

The customer sees why.

`requestResolution(maker, paymentId, "COMPLETE").andExpect(status().isBadRequest());`

A top-up is never "completed": that is for transfers.

`String own = topUp(maker, "20.96", "res-fund-0004");`

An officer's own top-up: neither they nor a colleague's approval of their request.

`void aRestartedSimulatorShouldStillKnowWhatItDid() throws Exception`

SIMULATORS SURVIVE A RESTART (V28)

`SimulatedBankRail rail = new SimulatedBankRail(testBank, railSessions);`

New simulator instances over the same tables, as after a restart.

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private String sendOut(Holder holder, String amount, String key) throws Exception`

Sends money out and returns the payment id.

`private String topUp(Holder holder, String amount, String key) throws Exception`

Tops up with a card whose answer is lost, and returns the payment id.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

pin

pin/TransactionSecurityIntegrationTest.java

`class TransactionSecurityIntegrationTest`

Device binding and the transaction PIN end to end: money leaves an account only from a trusted device with the right PIN, a new device is confirmed by email and then caps spending, and PIN guesses are counted even when sent all at once.

`void transferShouldNeedAPinThatIsSetAndEntered() throws Exception`

THE TRANSACTION PIN

`transfer(esther, john, "100", PIN, "lock-0004")`

Locked means locked: the right PIN does not get through either.

`void parallelGuessesShouldNotGetPastTheAttemptLimit() throws Exception`

Each guess reserves an attempt before BCrypt runs, so ten wrong guesses at once still lock the PIN after exactly three.

`void firstDeviceShouldBeTrustedAtOnce() throws Exception`

DEVICE BINDING

`mockMvc.perform(authorized(post("/api/v1/security/pin/reset-code"), onLaptop))`

An unconfirmed device cannot touch the PIN either.

`transfer(onPhone, john, "100", PIN, "phone-0001").andExpect(status().isCreated());`

The phone is unaffected.

`confirm(onLaptop, "000000").andExpect(status().isBadRequest())`

A wrong code is refused and counted; the right one still works.

`transfer(onLaptop, john, "25000", PIN, "cool-0001")`

₦20,000 in total until the cooling-off ends, from any device.

`jdbcTemplate.update("UPDATE customer_devices SET cooling_off_until = now() - interval '1 minute'");`

Once it ends, only the balance limits what can be sent.

`mockMvc.perform(authorized(delete("/api/v1/devices/{id}", laptopId), onLaptop))`

A pending device cannot remove others (or itself).

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private Holder holder(String name, String device) throws Exception`

A customer with an ACTIVE NGN account, signed in from the given device.

`private String latestCode(Holder holder)`

The code in the newest email to the holder.

reconciliation

reconciliation/ReconciliationIntegrationTest.java

`"paycore.rails.provider=simulated",`

Reconciliation end to end against PostgreSQL, through the simulated rail and card processor: a clean day, then every kind of break the simulators can produce, how each is found, fixed and closed, and that a run is evidence nobody can rewrite.

```
Test Bank .95   inbound notification lost      MISSING_LOCAL    re-delivered by the rail
transfer  .93   returned after acceptance      STATUS_MISMATCH  reversed (maker-checker)
card      .94   charged N1.00 more             AMOUNT_MISMATCH  resolved by a supervisor
card      .92   not in the settlement report   MISSING_PROVIDER
card      .96   still unknown in PayCore       STATUS_MISMATCH
```

`jdbcTemplate.execute("TRUNCATE ledger_entries, ledger_transactions, reconciliation_runs CASCADE");`

Every day starts empty on both sides: PayCore's books and the providers' own records.

`void aDayWhereBothSidesAgreeShouldHaveNoBreaks() throws Exception`

A CLEAN DAY

`sendOut(esther, "20.99", "rec-out-0002");`

rejected and reversed: both sides say FAILED

`void aLostInboundNotificationShouldBeFoundAndDeliveredAgain() throws Exception`

BREAKS AND HOW THEY ARE FIXED

Test Bank paid, but PayCore never heard: the money sits in the settlement bank account, credited to nobody. Reconciliation finds it; the rail delivers the notification again and the customer is credited, once.

`mockMvc.perform(authorized(post("/api/v1/ops/reconciliation/items/{id}/redeliver", item.get("id")), esther.token()))`

Customers cannot touch reconciliation.

`mockMvc.perform(authorized(post("/api/v1/ops/reconciliation/items/{id}/redeliver", item.get("id")), maker.token()))`

Again: nothing more to deliver, and the day now matches.

`void aTransferReturnedAfterAcceptanceShouldBeFoundAndReversed() throws Exception`

The rail accepted the transfer, so the customer was debited and told it succeeded; the beneficiary bank returned it later. Only the settlement report shows it. Two officers reverse the debit, and the day matches.

`String request = JsonPath.read(`

The fix moves money, so it goes through maker-checker.

`void anOverchargedTopUpShouldBeResolvedOnceAndStayResolved() throws Exception`

The processor took N1.00 more than PayCore credited. A supervisor resolves it with a note; the next run carries the resolution over instead of raising it again.

`void aRunShouldNeverBeRewritten() throws Exception`

RUNS ARE EVIDENCE

`private record Holder(String email, String customerId, String accountId, String accountNumber, String token)`

HELPERS

`private Map<String, Object> onlyBreak(ResultActions run) throws Exception`

Runs, expects exactly one break, and returns it.

`private Holder funded(String name, String amount) throws Exception`

Money from another rail ("TESTRAIL"), so it stays out of the simulated rail's reconciliation.

`private void withTransactionPin(String token) throws Exception`

Money moves only from a trusted device with the transaction PIN. A test customer signs in from TEST_DEVICE_ID, which as their first device is trusted, and has TEST_PIN (already set on a second login).

security

security/ProdJwtSecretConfigurationTest.java

`class ProdJwtSecretConfigurationTest`

Loads the real application.properties / application-prod.properties (through Spring Boot's config-data processing) with only the JWT configuration, so the secret rules are checked without a database.

`assumeTrue(`

A PAYCORE_JWT_SECRET exported in the developer's shell would make this case unreachable; skip rather than give a false result.

`runner.withPropertyValues(`

Guards against relying on the 32-byte check by coincidence: a long unresolved placeholder must still be reported as missing.

security/SecurityIntegrationTest.java

`register("proxied@example.com", "08011111119", "http://localhost:5173");`

A dev proxy forwards the browser's Origin header to another host.

`String expired =`

Signed with the real key for a live session, so expiry is the only thing wrong. Ten minutes is well past the decoder's 60 second clock skew.

`getOwnProfile(accessToken).andExpect(status().isUnauthorized());`

Every session's access token stops working at once.

The access token is still signed and unexpired, but no longer works.

`refresh(refreshToken).andExpect(status().isUnauthorized());`

Sessions ended at suspension stay ended after reactivation.

src/test/resources/db

src/test/resources/db/test-app-role.sql

`CREATE ROLE paycore_app LOGIN PASSWORD 'app-password' NOINHERIT;`

DatabasePrivilegeIntegrationTest: the API's own database user, as scripts/db/create-app-role.sql creates it in production. Flyway runs as the container's superuser (the owner) and grants it its privileges.

Frontend

paycore-ui/.env.example

`VITE_API_BASE_URL=`

Origin of the PayCore API, without a trailing slash. Leave empty in development: Vite proxies /api to http://localhost:8080.

paycore-ui/src/api/client.ts

`const API_BASE = (import.meta.env.VITE_API_BASE_URL ?? '').replace(/\/$/, '');`

Empty in development (Vite proxies /api); the API origin in production.

`export interface Session`

---------- Session: tokens live in sessionStorage, so each browser tab is its own sign-in and closing the tab forgets them. ----------

`(end of block)`

Storage blocked: the session still works for this page load.

`const DEVICE_KEY = 'paycore.deviceId';`

---------- Device id: a random id this browser keeps for itself (localStorage, so every tab is the same device). Sent when signing in; the server only stores its hash, and trusts the device once it is confirmed. ----------

`fallbackDeviceId ??= crypto.randomUUID();`

Storage blocked: this page load is one device.

`export type AuthMode = 'bearer' | 'none' | { token: string; label: string };`

---------- API inspector: every call is recorded so the UI can show what the backend actually did (status, auth header, timing, body). ----------

`function redact(value: unknown): unknown`

Hides passwords, BVNs and NINs and shortens tokens before they are displayed.

`export class ApiError extends Error`

---------- Requests ----------

`formSummary?: unknown;`

Show this instead of the real multipart body in the inspector.

`if (first.status === 401 && auth === 'bearer' && session?.refreshToken)`

An expired access token is renewed once with the refresh token and the request is repeated. Deliberate unauthenticated calls are never retried.

`function refreshTokens(): Promise<boolean>`

Single-flight: parallel 401s share one refresh (refresh tokens rotate).

`export interface AccessClaims`

---------- JWT claims (decoded for display only; the server verifies the signature) ----------

`const missing = error.body.missing ?? [];`

KYC_INCOMPLETE already names the missing items in its message.

paycore-ui/src/api/demo.ts

`let cached: Promise<DemoInfo | null> | null = null;`

Fetched once per page load; null when demo mode is off on the server.

`export function useDemoInfo()`

undefined while loading, null when demo mode is off.

paycore-ui/src/api/types.ts

`export type CustomerStatus = 'PENDING_VERIFICATION' | 'ACTIVE' | 'SUSPENDED' | 'CLOSED';`

Mirrors the Spring Boot DTOs. Instants arrive as ISO-8601 strings.

`providerReference?: string;`

On PAYMENT_DECLINED: the provider's reference for the declined payment.

`attemptsRemaining?: number;`

On INVALID_PIN: wrong PINs left before it locks.

`limit?: string;`

On LIMIT_EXCEEDED: which limit, what can still be sent, and until when.

`coolingOffUntil: string | null;`

Set while spending is capped after this device was confirmed.

`adminPassword: string;`

Also the password of the shared operations officers.

`kycProvider: string;`

'SIMULATED' or 'DOJAH'

`recipientName: string | null;`

A shared account visitors can send money to; null until seeded.

`fundingProvider: string | null;`

'SIMULATED' when card top-ups are on; null otherwise.

`bankRailProvider: string | null;`

'SIMULATED' when transfers to and from Test Bank are on; null otherwise.

`export type TransactionType =`

---------- Ledger. Amounts arrive as JSON numbers in major units (20000.5 = ₦20,000.50); the backend keeps them in kobo and never rounds. ----------

`accountNumber: string | null;`

null for PayCore's own settlement account

`accountName: string;`

The account holder; "PayCore" for its settlement account.

`export interface Counterparty`

The account at another bank on the other side of a bank transfer.

`export interface Transaction`

What customers see. Has no staff note, by design.

`provider: string | null;`

The card processor or bank rail that carried the money.

`providerReference: string | null;`

Its own reference: a payment reference or NIP session id.

`export interface StaffTransaction extends Transaction`

What staff see: the same, plus the internal note, who initiated and who approved.

`export interface OutboundTransfer extends Transaction`

A transfer to another bank, with the rail's verdict.

`export interface Payment`

A payment through an external provider (GET /api/v1/payments/{id}).

`bill: { billerId: string; productCode: string | null; productName: string | null; token: string | null } | null;`

Bill payments only; token is what the biller gave back (a meter token).

`export interface PaymentEvent`

One step in a payment's history (fromStatus null for its creation).

`export interface FundingAllowance`

How much more can be added to an account by card.

`export interface TestBankAccount`

The customer's own (pretend) account at Test Bank.

`export interface OpsRequest`

---------- Operations: maker-checker corrections. ----------

`export interface ReconciliationRun`

One comparison of a provider's settlement report for a day with PayCore's books.

`maxBalance: number | null;`

Null when the tier has no balance cap.

`products: boolean;`

Pick a bundle or bouquet; its price is the biller's.

`verified: boolean;`

The number is checked with the biller first.

`export interface BillPayment extends Transaction`

The debit for a bill (a Transaction) plus what the biller did.

paycore-ui/src/auth/AuthContext.tsx

`isStaff: boolean;`

ADMIN, OPERATIONS or SUPPORT: uses the back office.

`isCustomer: boolean;`

Has the CUSTOMER role: uses the banking app.

`can: (permission: string) => boolean;`

Whether the access token carries the permission.

`let signOutRedirect: string | null = null;`

Where a deliberate sign-out should land; read by RequireAuth.

`const logout = useCallback(async (redirectTo?: string) =>`

Forgets the tokens locally at once, then revokes the LoginSession and its refresh tokens server-side. Clearing the session makes a protected page redirect, so `redirectTo` tells RequireAuth where to send the visitor instead of the generic sign-in page.

`(end of block)`

Already signed out locally; nothing else to do if the API is down.

paycore-ui/src/banking.ts

`const listeners = new Set<() => void>();`

What every banking screen needs: the customer's naira account, its balance and their KYC status. Screens that move money call refresh() afterwards; every mounted screen then reloads.

`export function refreshBanking()`

Ask every screen showing the account to reload it (after money moved).

`account: Account | null;`

The customer's naira account (the first one that is not closed), if any.

`balance: Balance | null;`

Null until the account is active.

`export function useStatement(accountId: string | null | undefined, query = '', refreshKey = 0)`

The account's statement for the month so far (or a date range).

paycore-ui/src/components/ApiInspector.tsx

`export function ApiInspector()`

Docked panel listing every request the UI made: method, path, whether a bearer token was sent, the status the backend returned and the body. Passwords, BVNs and NINs are masked and tokens shortened before display.

paycore-ui/src/components/JourneyBar.tsx

`export function JourneyBar()`

The guided walkthrough: customer phase, then admin phase, then back to the customer to see the decision. Completed steps are ticked.

paycore-ui/src/components/Layout.tsx

`function GuideButton()`

The walkthrough, behind a button instead of on every screen.

`export function PublicLayout()`

---------- Public pages: landing, sign in, register. ----------

`const CUSTOMER_NAV: { to: string; label: string; icon: string; end?: boolean; tabBar?: boolean }[] = [`

---------- The banking app: sidebar on desktop, tab bar on phones. ----------

tabBar: false keeps an item out of the five-slot phone tab bar (Home links to it instead).

`export function StaffLayout()`

---------- The back office: compliance (admin) and operations (maker-checker). ----------

`export function ScreenTitle({ title, children, back }: { title: string; children?: ReactNode; back?: string })`

A screen title inside the app shells.

paycore-ui/src/components/PinInput.tsx

`export function PinInput(`

Four masked digit boxes; calls onChange with the digits typed so far.

paycore-ui/src/components/StatementView.tsx

`export function StatementView(`

An account statement: period totals and every line with its running balance. Empty dates mean the month to date, which is what the API defaults to. Works against the customer endpoint or the staff one.

`path: string;`

/api/v1/ledger/accounts/{id}/statement or the /admin equivalent

`refreshKey?: number;`

Change it to reload, e.g. after a transfer.

`lineAction?: (line: StatementLine) => ReactNode;`

Staff only: something to show at the end of a line, e.g. Reverse.

`const [applied, setApplied] = useState({ from: '', to: '' });`

Dates only apply when the form is submitted, so typing does not refetch.

paycore-ui/src/components/TransactionList.tsx

`export function lineTitle(line: StatementLine)`

The line's headline, as a banking app writes it.

`export function TransactionList(`

Statement lines, newest first, grouped by day.

paycore-ui/src/components/ui.tsx

`export function HttpStatus({ status }: { status: number | null })`

Shows an HTTP status the way the inspector does.

`export function formatAccountNumber(value: string)`

0123456789 -> 012 345 6789, the way Nigerian account numbers are read out.

`export function formatMoney(amount: number, currency: string)`

20000.5, 'NGN' -> ₦20,000.50

`export function accountLabel(accountNumber: string | null)`

Entries on PayCore's settlement account have no account number.

`export function newIdempotencyKey(prefix: string)`

A fresh idempotency key for one money-moving request. Resending the same request with the same key returns the original result instead of moving the money again.

`export function RequireAuth({ staff = false, children }: { staff?: boolean; children: ReactNode })`

staff: the back office (ADMIN, OPERATIONS, SUPPORT). Otherwise the banking app, which staff-only accounts (operations officers) cannot use.

`export function initials(name: string | null | undefined)`

Initials for an avatar: "Esther Agboniro" -> "EA".

`export function CopyButton({ text, label = 'Copy' }: { text: string; label?: string })`

Copies text and briefly says so.

`(end of block)`

Clipboard blocked (e.g. not https); the number is on screen anyway.

paycore-ui/src/journey.ts

`export type StepId =`

The guided walkthrough. Progress is kept per browser in localStorage so a visitor can switch between the customer and the staff accounts and still see where they are.

`customerEmail: string | null;`

The customer account this visitor created, to sign back in later.

`(end of block)`

Progress just won't survive a reload.

paycore-ui/src/main.tsx

`function StaffHome()`

Compliance lands on KYC reviews; operations officers on their approvals queue.

paycore-ui/src/pages/admin/AccountMoney.tsx

`export function AccountMoney({ account }: { account: Account })`

Staff view of one customer account's money: its balance and statement. Staff cannot deposit or withdraw. An operations officer can request an adjustment, posted only once a second officer approves it.

paycore-ui/src/pages/admin/Customers.tsx

`const ACTIONS: Record<Account['status'], AccountAction[]> =`

Which staff actions the account's current status allows.

paycore-ui/src/pages/admin/FinishCard.tsx

`export function FinishCard()`

Closes the compliance loop: back to the visitor's own customer account.

paycore-ui/src/pages/admin/Ledger.tsx

`export function Ledger()`

Staff view of the ledger: what PayCore holds, and any transaction by reference with its internal note. Staff cannot move money here; an operations officer can only request a reversal, which a second officer must approve.

paycore-ui/src/pages/admin/Operations.tsx

`export function Operations()`

The maker-checker queue. Officers file reversals (from Transactions) and adjustments (from Customers); a different officer approves or rejects them here. Nothing moves until a second person approves.

paycore-ui/src/pages/admin/Payments.tsx

`const RESOLUTIONS: Record<Payment['type'], { value: PaymentResolution; label: string; effect: string }[]> =`

What each resolution does, for the officer choosing one.

`export function Payments()`

Payments through the bank rail and the card processor, and the ones automation could not settle. Officers can ask the provider again, take an unknown payment into review, and request a resolution, which a second officer approves under Approvals. Nobody decides alone, and never against what the provider says.

paycore-ui/src/pages/admin/Reconciliation.tsx

`function lagosToday()`

Today in Lagos, as yyyy-mm-dd: the business day PayCore reconciles.

`export function Reconciliation()`

Reconciliation: each provider's settlement report for a day against PayCore's books. Matches need nothing; breaks are worked here (the rail re-delivering a lost notification, a maker-checker reversal, a status check) and closed by a supervisor with a note.

paycore-ui/src/pages/admin/TierUpgrades.tsx

`export function TierUpgrades()`

Verified customers asking for a higher tier; compliance decides.

paycore-ui/src/pages/app/AddMoney.tsx

`}, [account.id]);`

eslint-disable-next-line react-hooks/exhaustive-deps

`const credit = await api<Transaction | { error: string; message: string; sessionId: string }>(`

201: credited. 202 NOTIFICATION_LOST (.95): Test Bank paid, but the rail never told PayCore; reconciliation finds it.

`async function refreshPending()`

Where a pending top-up stands now.

`}, [account.id]);`

eslint-disable-next-line react-hooks/exhaustive-deps

`const result = await api<Transaction | Payment>(`/api/v1/ledger/accounts/${account.id}/fundings`,`

201: the deposit. 202: the provider gave no answer, so the payment is still being confirmed (the card may have been charged).

paycore-ui/src/pages/app/BankHome.tsx

`(end of block)`

Not remembered; fine.

`function Onboarding({ account, kycStatus }: { account: Account | null; kycStatus: string | null })`

Before the account can be used: verify identity, open the account, wait for compliance to activate it.

paycore-ui/src/pages/app/Bills.tsx

`const NUMBER_PATTERNS: Record<BillCategory, RegExp> =`

Same shapes the server accepts (BillCategory on the backend).

`product: BillProduct | null;`

Bundle or bouquet; null for airtime and electricity.

`meterType: string | null;`

"prepaid" / "postpaid" for electricity.

`amount: draft.product ? null : draft.amount,`

The provider sets the price of bundles and bouquets.

`const [pin, setPin] = useState('');`

Kept in memory only, for the "resend the same request" check.

`useEffect(() =>`

A new category starts on its first biller.

`}, [category, billers]);`

eslint-disable-next-line react-hooks/exhaustive-deps

`useEffect(() =>`

A new biller clears everything chosen for the previous one.

Meters and smartcards are checked with the biller as soon as the number is complete.

`async function checkStatus()`

Where the payment behind a pending bill stands now.

`async function resend()`

The identical request again, same idempotency key.

paycore-ui/src/pages/app/PinAndDevices.tsx

`function ThisDevice({ device }: { device: Device | null })`

`type PinMode = 'view' | 'change' | 'reset';`

`function DeviceList({ canRemove }: { canRemove: boolean })`

`function DemoCode({ sentAt }: { sentAt: CodeSent })`

In the demo, emails go to an outbox instead of a real inbox.

paycore-ui/src/pages/app/Transactions.tsx

`function monthRange(back: number)`

First and last day of the month `back` months ago, as ISO dates (local).

paycore-ui/src/pages/app/Transfer.tsx

`const [pin, setPin] = useState('');`

Kept in memory only, for the "resend the same request" check.

`useEffect(() =>`

Name enquiry as soon as the number is complete, like every banking app.

`async function checkStatus()`

Where the payment behind a pending transfer stands now.

`async function resend()`

The identical request again, same idempotency key.

paycore-ui/src/pages/Kyc.tsx

`export const TEST_NUMBERS: Record<IdType, string> = { BVN: '22222222222', NIN: '70123456789' };`

Test numbers known to both the simulated provider and Dojah's sandbox.

`export function sampleDocument(type: KycDocumentType): Promise<Blob>`

A clearly fake ID card drawn in the browser, as a PNG.

paycore-ui/src/pages/Login.tsx

`const mode = params.get('as');`

'staff' | 'customer' | null

paycore-ui/src/pages/Register.tsx

`navigate('/login?registered=1');`

The email is prefilled from the walkthrough state, never put in the URL.

paycore-ui/src/pages/Security.tsx

`function tamperedToken(): AuthMode`

Changes one character in the middle of the signature so the HMAC no longer verifies. (The last base64url character carries padding bits, so editing it may not change the decoded signature at all.)

paycore-ui/src/security.ts

`const listeners = new Set<() => void>();`

Whether this browser may move money: it must be a trusted device, and the customer must have a transaction PIN.

`export function refreshSecurity()`

Ask every screen that shows the PIN or device state to reload it.

`device: Device | null;`

This browser; null if the session was opened without a device id.

`ready: boolean;`

Trusted device and an unlocked PIN: payments can go ahead.

paycore-ui/src/styles.css

`.shell`

---------- Shell ----------

`.journey`

---------- Journey ----------

Wraps on wide screens so every phase stays visible; scrolls on phones.

`.page-head`

---------- Page head ----------

`.card`

---------- Cards & layout ----------

`.form`

---------- Forms & buttons ----------

`.badge`

---------- Badges, chips, notices ----------

`.kv`

---------- Key/value ----------

`.hero`

---------- Home ----------

`.kv-wide`

---------- Dashboard ----------

`.kyc-flow`

---------- KYC ----------

`.security .card:first-child`

---------- Security ----------

`.filters`

---------- Admin ----------

`.inspector`

---------- API inspector ----------

`.big-money`

---------- Ledger: balances, transfers, statements ----------

`:root`

==========

Banking app shell: public site, customer app, back office.

==========

`.public-bar`

---------- Public site ----------

`.app-frame`

---------- App frame ----------

`.drawer-backdrop`

---------- Drawer (demo guide) ----------

`.panel`

---------- Panels ----------

`.home-grid`

---------- Home ----------

`.txn-list section + section`

---------- Transactions ----------

`.onboarding`

---------- Onboarding ----------

`.bank-picker`

---------- Transfer, add money, receipts ----------

`.profile-head`

---------- Profile ----------

`.ops-list`

---------- Back office ----------

`.inspector:not(.open)`

---------- API inspector: a pill until opened ----------

`.only-mobile`

---------- Phones ----------

`.pin-input`

---------- Transaction PIN entry ----------

`.limit-row`

---------- Limits ----------

paycore-ui/vite.config.ts

`const api = process.env.PAYCORE_API_URL ?? 'http://localhost:8080';`

In development the UI calls /api on its own origin and Vite forwards it to Spring Boot, so no CORS setup is needed locally. PAYCORE_API_URL changes where it forwards to. In production set VITE_API_BASE_URL to the API's origin instead (see README.md).

Configuration, build and scripts

Dockerfile

`FROM eclipse-temurin:21-jdk AS build`

Builds and runs the PayCore API. Sized for small hosts such as Render's free plan (512 MB RAM). Configuration comes from environment variables; see README.md.

---- Build ----

`COPY mvnw pom.xml ./`

Download dependencies first so they are cached until pom.xml changes.

`RUN ./mvnw -q -B package -DskipTests \`

Tests need Docker (Testcontainers), which is not available here; run them locally or in CI before deploying.

`FROM eclipse-temurin:21-jre`

---- Run ----

`ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+UseSerialGC -Xss512k -XX:TieredStopAtLevel=1"`

Keep the JVM within a 512 MB container.

`CMD ["sh", "-c", "exec java -Dserver.port=${PORT:-8080} -jar app.jar"]`

Hosts like Render pass the port to listen on in $PORT.

paycore-local.properties.example

`PAYCORE_DB_PASSWORD=`

Copy to paycore-local.properties (git-ignored) and fill in your own values. Loaded by spring.config.import in application.properties when the app or tests run from the project root. Real environment variables override it. In production, set these as environment variables / from a secret manager instead of using this file.

Required: password for the local PostgreSQL user (default user: postgres, default URL: jdbc:postgresql://localhost:5432/paycore).

`DOJAH_APP_ID=`

Optional overrides PAYCORE_DB_URL=jdbc:postgresql://localhost:5432/paycore PAYCORE_DB_USERNAME=postgres

Optional locally (a development fallback is used); required in prod. Generate with: openssl rand -base64 48 PAYCORE_JWT_SECRET=

Dojah sandbox credentials from https://app.dojah.io (Developers > Configuration). Without them, BVN checks return 502 KYC_PROVIDER_UNAVAILABLE.

`(end of file)`

Developer Preview demo mode (see paycore-ui/README.md). Seeds a shared admin whose password is shown to every visitor of the demo UI. PAYCORE_DEMO_ENABLED=true PAYCORE_DEMO_ADMIN_PASSWORD= Let customers top up their own accounts with fake card payments (demo only: the balances are not backed by real money). Default: none. PAYCORE_FUNDING_PROVIDER=simulated PAYCORE_FUNDING_MAX_TOTAL_PER_ACCOUNT=500000 Transfers to and from other banks through a simulated rail ("Test Bank"). PAYCORE_RAILS_PROVIDER=simulated Signs inbound transfer webhooks; blank disables the webhook. PAYCORE_RAILS_WEBHOOK_SECRET= Bill payments: "simulated" for the demo, or "vtpass" with sandbox keys from https://sandbox.vtpass.com (Settings > API keys). PAYCORE_BILLS_PROVIDER=simulated VTPASS_API_KEY= VTPASS_PUBLIC_KEY= VTPASS_SECRET_KEY= Only needed when the UI runs on a different origin than the API. PAYCORE_CORS_ALLOWED_ORIGINS=http://localhost:5173

Verification emails. Without SMTP they are only written to the log. Local Mailpit (docker run -p 1025:1025 -p 8025:8025 axllent/mailpit), inbox at http://localhost:8025: SPRING_MAIL_HOST=localhost SPRING_MAIL_PORT=1025 Where the web app runs; verification links point there. PAYCORE_PUBLIC_URL=http://localhost:5173

Sign in with Google: client id of a "Web application" OAuth client in Google Cloud Console, with http://localhost:5173 as an authorised JavaScript origin. No client secret is needed. PAYCORE_GOOGLE_CLIENT_ID=

pom.xml

`<dependency>`

Verification emails over SMTP when spring.mail.host is set

OpenAPI description at /v3/api-docs and Swagger UI at /swagger-ui.html

Tests

`<plugin>`

Tests run payment recovery and reconciliation themselves, not on a timer.

`<paycore.demo.enabled>false</paycore.demo.enabled>`

A developer's paycore-local.properties may turn the demo or providers on; tests that need them say so.

scripts/db/create-app-role.sql

`CREATE ROLE paycore_owner LOGIN PASSWORD :owner_password;`

Creates the two PayCore database users for production. Run once, as a user with CREATEROLE (e.g. the managed database's admin), before the first deploy:

psql "$ADMIN_URL" -v owner_password="'...'" -v app_password="'...'" \ -f scripts/db/create-app-role.sql

Then configure the API with

PAYCORE_DB_MIGRATION_USERNAME=paycore_owner (Flyway: owns the tables) PAYCORE_DB_MIGRATION_PASSWORD=... PAYCORE_DB_USERNAME=paycore_app (the running API) PAYCORE_DB_PASSWORD=... PAYCORE_DB_REQUIRE_LEAST_PRIVILEGE=true (refuse to start otherwise)

Flyway grants paycore_app its privileges after every migration (V29__least_privilege_app_role.sql, afterMigrate.sql). If the tables already exist under another owner, move them first:

REASSIGN OWNED BY <old_owner> TO paycore_owner;

`GRANT CREATE, USAGE ON SCHEMA public TO paycore_owner;`

The owner creates PayCore's schemas and tables; the app creates nothing.

`(end of file)`

If the database already has PayCore's tables and functions, grant now rather than waiting for the next migration: SELECT paycore_apply_app_grants('paycore_app');

scripts/try-accounts.ps1

`param(`

Walks through the account flow against a locally running PayCore API:

customer registers -> opens account (PENDING) admin tries to activate -> refused, KYC not verified customer completes KYC -> admin approves it admin activates (ACTIVE) -> freezes -> unfreezes customer lists accounts; admin reads the audit history plus: duplicate account (409) and another customer's account (404)

Start the API first, in demo mode with simulated KYC (see README), then:

powershell -ExecutionPolicy Bypass -File scripts\try-accounts.ps1 powershell -ExecutionPolicy Bypass -File scripts\try-accounts.ps1 -Api http://localhost:8080

`function CallExpectingError([string]$Method, [string]$Path, [string]$Token, $Body)`

Calls that are supposed to fail: returns "<status> <error code>".

`try`

Windows PowerShell 5.1 leaves ErrorDetails empty; read the body.

`$customer = Register "John" "Doe"`

The simulated KYC provider knows one person: John Doe, born 1990-01-01.

scripts/try-payments.ps1

`param(`

Walks through payments whose outcome is unknown, against a locally running PayCore API with the simulated card processor and bank rail:

card top-up .98 charged, answer lost -> 202 pending -> status check credits it transfer .96 paid, rail never confirms -> escalated -> COMPLETE (maker-checker) card top-up .96 charged, never confirmable -> escalated -> REFUND (maker-checker) transfer .98 rail says paid -> a REVERSE request is refused (the provider wins) plus: the maker approving their own request (403 FOUR_EYES_REQUIRED)

Start the API first in demo mode with all simulators, e.g.

$env:PAYCORE_DEMO_ENABLED="true"; $env:PAYCORE_DEMO_ADMIN_PASSWORD="admin_user_test" $env:PAYCORE_KYC_PROVIDER="simulated"; $env:PAYCORE_FUNDING_PROVIDER="simulated" $env:PAYCORE_RAILS_PROVIDER="simulated"; .\mvnw spring-boot:run

then:

powershell -ExecutionPolicy Bypass -File scripts\try-payments.ps1 powershell -ExecutionPolicy Bypass -File scripts\try-payments.ps1 -Api http://localhost:8080

`[string]$StaffPassword = "admin_user_test"`

The demo operations officers share the demo admin's password.

`function CallExpectingError([string]$Method, [string]$Path, [string]$Token, $Body)`

Calls that are supposed to fail: returns "<status> <error code>".

src/main/resources/application-prod.properties

`paycore.jwt.secret=${PAYCORE_JWT_SECRET}`

Production profile: activate with SPRING_PROFILES_ACTIVE=prod.

---------- JWT signing secret ---------- Required, with no fallback: if PAYCORE_JWT_SECRET is not set the app refuses to start ("paycore.jwt.secret is not configured: set the PAYCORE_JWT_SECRET environment variable") instead of silently signing tokens with the development secret from application.properties. The development secret is also rejected under this profile. It must be at least 32 bytes; generate one with openssl rand -base64 48 and keep it in your secret manager, not in any file in this repository.

`server.forward-headers-strategy=native`

---------- Running behind a reverse proxy / load balancer ---------- Without this, every request appears to come from the proxy's address, so all customers would share one per-IP BVN allowance and login sessions would record the proxy instead of the customer.

NATIVE uses Tomcat's RemoteIpValve. It honours X-Forwarded-For and X-Forwarded-Proto ONLY when the direct peer is a trusted proxy; from anyone else the headers are ignored, so clients cannot spoof their IP. It walks X-Forwarded-For right to left, skipping trusted proxies, and uses the first untrusted address as the client, so a forged left-most entry is never picked.

`(end of file)`

Which peers are trusted to set X-Forwarded-For. The defaults are:

internal-proxies: private ranges (10/8, 172.16/12, 192.168/16, 127/8, 169.254/16, 100.64/10, ::1, fc00::/7, fe80::/10). Suits a load balancer inside a private network, as long as nothing else on that network can reach the app directly. trusted-proxies: none.

Narrow or extend them per environment with environment variables (Java regexes matched against the peer's IP), without editing this file:

SERVER_TOMCAT_REMOTEIP_INTERNALPROXIES=10\.0\.1\.\d{1,3} only your load balancer's subnet (recommended) SERVER_TOMCAT_REMOTEIP_TRUSTEDPROXIES=173\.245\.4[89]\.\d{1,3}|... public proxies/CDNs in front of it, e.g. Cloudflare's published ranges SERVER_TOMCAT_REMOTEIP_REMOTEIPHEADER=X-Real-IP only if your proxy does not send X-Forwarded-For

A proxy that appends the peer address to X-Forwarded-For (the standard behaviour of nginx $proxy_add_x_forwarded_for, AWS ALB, GCP LB) is safe: the right-most untrusted entry is the address the proxy itself saw, and anything the client wrote further left is ignored.

What must hold: the app is reachable ONLY through the trusted proxies. If a client can connect directly from an address matching the patterns above, it can choose its own X-Forwarded-For.

src/main/resources/application.properties

`spring.config.import=optional:file:./paycore-local.properties`

Secrets never live in this file. Each one is read from an environment variable. For local development, put them in paycore-local.properties in the project root (git-ignored; copy paycore-local.properties.example). Real environment variables take precedence over that file.

`spring.flyway.user=${PAYCORE_DB_MIGRATION_USERNAME:${spring.datasource.username}}`

Flyway can run as a different user that owns the tables, so the API's own user (above) holds only the privileges it needs (V29). Defaults to the API's user, as in local development.

`paycore.database.require-least-privilege=${PAYCORE_DB_REQUIRE_LEAST_PRIVILEGE:false}`

true: refuse to start unless the API's database user is least-privileged (not the owner, no DELETE/TRUNCATE on ledger and payment tables). See scripts/db/create-app-role.sql.

`spring.jpa.open-in-view=false`

Keep database sessions to the service transaction, not the whole web request, so a connection is never held while a response is written.

`spring.datasource.hikari.maximum-pool-size=${PAYCORE_DB_POOL_SIZE:10}`

Connection pool. Each app instance holds at most this many connections; (instances x pool size) must stay under the database's connection limit. Size it to the database, not to the traffic: more connections than the database has cores mostly adds contention. Requests that cannot get a connection within the timeout fail fast instead of piling up.

`paycore.security.session.expiration=12h`

Session lifetime (banking practice): signed out after 15 minutes without activity, and after 12 hours at most. The access token must stay well below the idle timeout, because refreshing it is what counts as activity.

`paycore.jwt.secret=${PAYCORE_JWT_SECRET:local-dev-only-paycore-jwt-secret-change-me-0123456789}`

JWT: override the secret via PAYCORE_JWT_SECRET outside local development.

`paycore.kyc.provider=${PAYCORE_KYC_PROVIDER:dojah}`

KYC provider: "dojah" (default; its free sandbox uses mock data) or "simulated" (in-process test records, no vendor account needed; demo only).

`dojah.base-url=${DOJAH_BASE_URL:https://sandbox.dojah.io}`

Dojah. Never commit real keys.

`paycore.kyc.bvn.max-failed-attempts=3`

BVN retry limit: at most N FAILED checks per rolling window (then HTTP 429).

`paycore.kyc.bvn.ip-max-attempts=10`

Per-IP limit: at most N BVN checks (any result) per network per window, across all customers.

`paycore.kyc.documents.storage-dir=${PAYCORE_KYC_DOCUMENT_DIR:data/kyc-documents}`

KYC document uploads (PDF/PNG/JPEG). Files go to local disk in development; only metadata and the storage key are stored in PostgreSQL.

`paycore.demo.enabled=${PAYCORE_DEMO_ENABLED:false}`

Developer Preview: seeds a shared demo admin and publishes its sign-in details at GET /api/v1/demo so visitors can try the admin flow. Only for the public demo on sandbox data; never enable it with real customers.

`paycore.funding.provider=${PAYCORE_FUNDING_PROVIDER:none}`

Customer top-ups through a payment provider: "none" (default; only staff deposit) or "simulated" (confirms fake card payments at once, amounts ending in .99 are declined; demo only, the balances are not real money).

`paycore.rails.provider=${PAYCORE_RAILS_PROVIDER:none}`

Transfers to and from other banks: "none" (default; only transfers inside PayCore) or "simulated" (a pretend interbank rail connected to "Test Bank"; payments ending in .99 are rejected and reversed; demo only).

`paycore.bills.provider=${PAYCORE_BILLS_PROVIDER:none}`

Bill payments (airtime, data, electricity, TV): "none" (default), "simulated" (a pretend aggregator; amounts ending in .99 are declined and reversed; demo only) or "vtpass" (https://www.vtpass.com, sandbox by default). VTpass keys come from its dashboard; never commit them.

`paycore.rails.webhook-secret=${PAYCORE_RAILS_WEBHOOK_SECRET:}`

Signs inbound transfer notifications from the rail (HMAC-SHA256). Blank disables the webhook.

`paycore.rails.simulator-inbound-limit-per-account=${PAYCORE_TESTBANK_LIMIT_PER_ACCOUNT:1000000}`

The most the Test Bank simulator sends to one account in total.

`paycore.funding.max-total-per-account=${PAYCORE_FUNDING_MAX_TOTAL_PER_ACCOUNT:500000}`

The most provider top-ups may add to one account in total, in major units.

`paycore.cors.allowed-origins=${PAYCORE_CORS_ALLOWED_ORIGINS:}`

Browser origins allowed to call the API (comma-separated), e.g. the Developer Preview UI: https://paycore-ui.example.com

`paycore.accounts.supported-currencies=NGN`

Accounts. Currencies customers can open accounts in (ISO 4217), and the 3-digit institution code behind the NUBAN check digit of every account number. Changing the code invalidates existing account numbers.

`paycore.ledger.statement-time-zone=${PAYCORE_STATEMENT_TIME_ZONE:Africa/Lagos}`

Ledger. Statements cover whole calendar days in this time zone, so a transfer at 23:30 UTC appears on the next day's statement in Lagos.

`paycore.email-verification.public-url=${PAYCORE_PUBLIC_URL:http://localhost:5173}`

Email verification. Links point at the web app: {public-url}/verify-email?token=...

`management.health.mail.enabled=false`

SMTP is off unless spring.mail.host is set (SPRING_MAIL_HOST); emails are then only logged. Locally, Mailpit: SPRING_MAIL_HOST=localhost and SPRING_MAIL_PORT=1025, inbox at http://localhost:8025. A mail outage must not mark the API itself unhealthy.

`paycore.devices.cooling-off=24h`

Device binding and the transaction PIN. Money moves only from a trusted device (the first one a customer uses, or one confirmed with an emailed code) and with the PIN. Confirming a new device caps the customer's total spending for the cooling-off period.

`paycore.limits.tier1.daily-send=50000`

Limits by KYC tier (naira, major units), after the CBN's three-tier KYC: how much a customer may send per day (statement time zone) and hold. An empty max-balance means no cap.

`paycore.google.client-id=${PAYCORE_GOOGLE_CLIENT_ID:}`

Sign in with Google: the client id of a "Web application" OAuth client (Google Cloud Console > APIs & Services > Credentials). Empty = off.

`paycore.payments.recovery.enabled=${PAYCORE_PAYMENT_RECOVERY_ENABLED:true}`

Payments whose outcome is unknown (provider timeout, or PayCore stopped mid-request) are settled by asking the provider, never by assuming. A background sweep runs every interval; see PaymentProperties.

`paycore.reconciliation.scheduled-runs.enabled=${PAYCORE_RECONCILIATION_ENABLED:true}`

Reconciliation: every morning, yesterday is reconciled with each provider's settlement report (cron, in the statement time zone). Officers can run any day from the back office at any time.